

CNN

概要

- 卷积
 - 平移不变性和局部性
 - 卷积层
 - 填充和步幅
- 多个输入和输出通道
- 池化

卷和



分类图像中的狗和猫

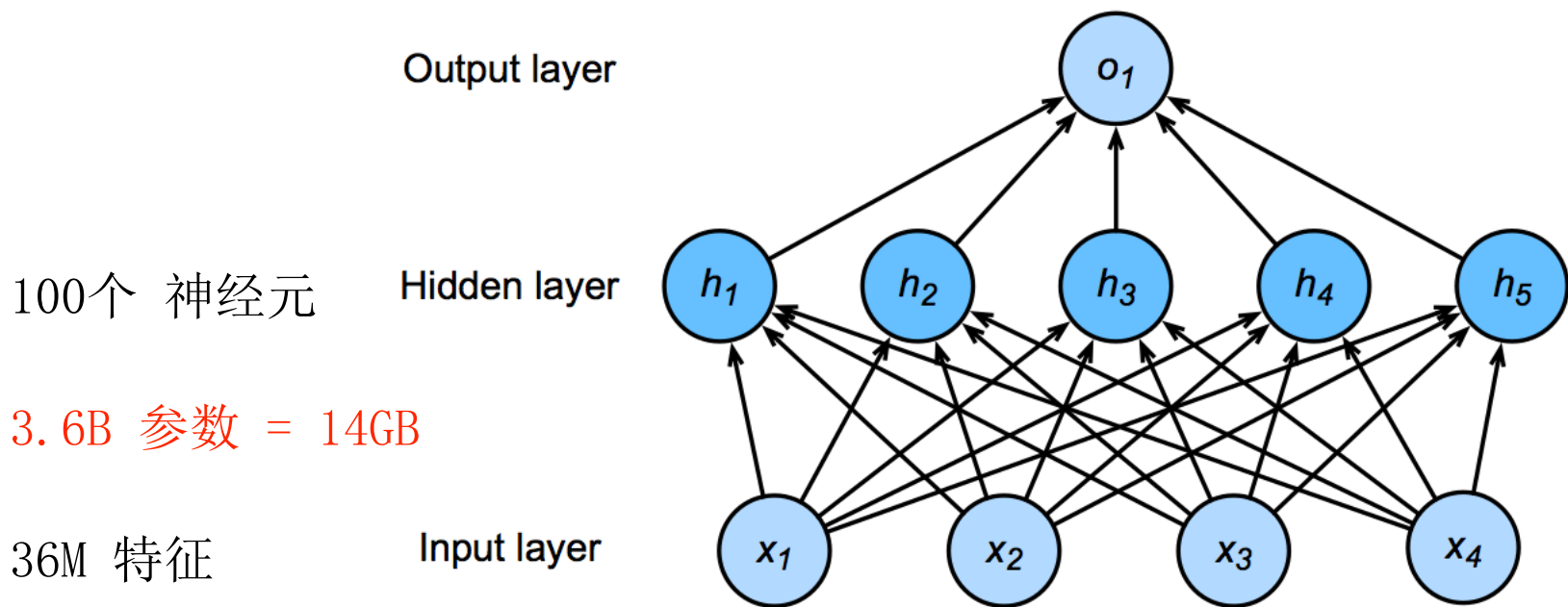
- 使用好的相机
- RGB图像具有 36M 个元素
- 使用 100 个神经元单隐含层的 MLP 模型：
 - 有 36 亿个参数
 - 超过地球上的狗和猫的数量 (900M 狗+ 600M 猫)



Dual
12MP
wide-angle and
telephoto cameras

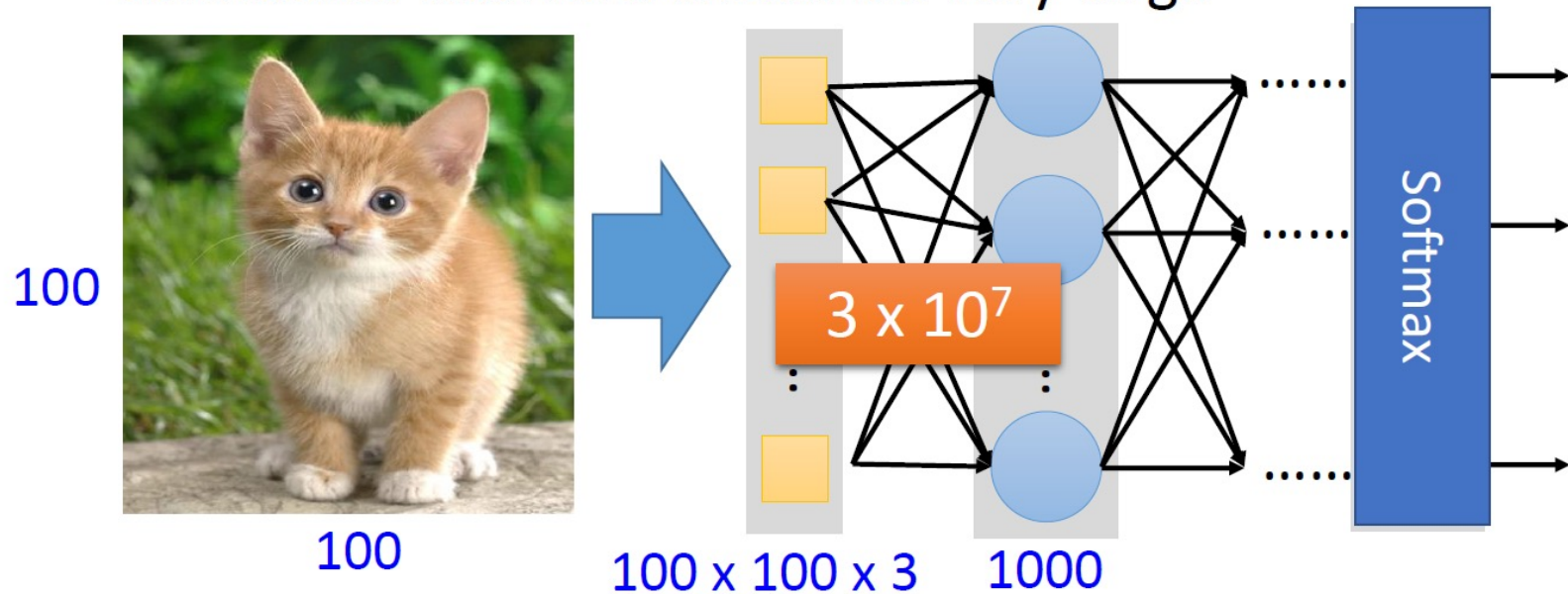


单隐含层网络



$$\mathbf{h} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$$

- When processing image, the first layer of fully connected network would be very large



Can the fully connected network be simplified by considering the properties of image recognition?

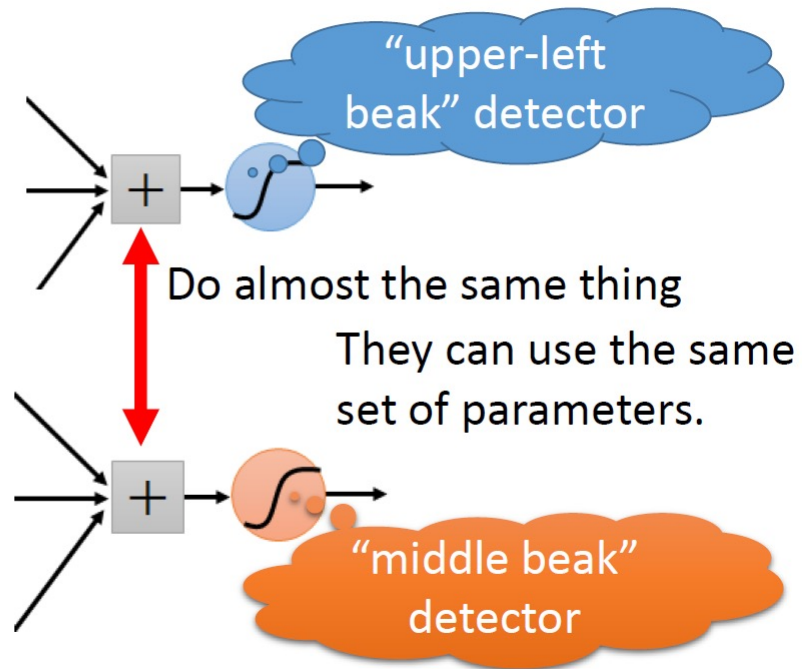
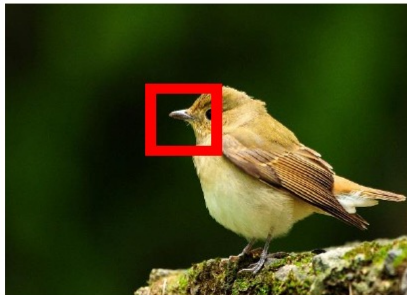
- Some patterns are much smaller than the whole image

A neuron does not have to see the whole image to discover the pattern.

Connecting to small region with less parameters



- The same patterns appear in different regions.



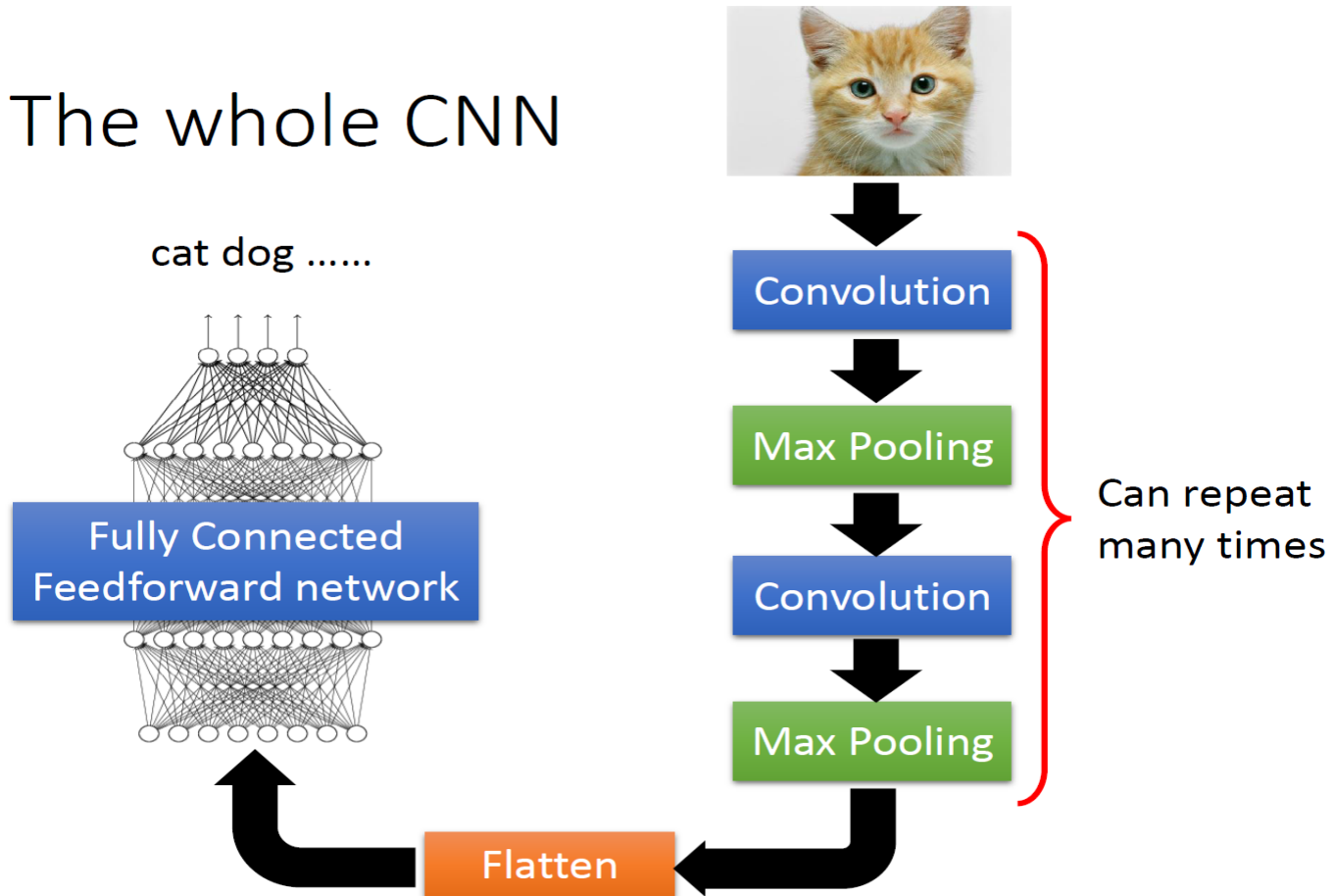
- Subsampling the pixels will not change the object



We can subsample the pixels to make image smaller

➡ Less parameters for the network to process the image

The whole CNN



The whole CNN

Property 1

- Some patterns are much smaller than the whole image

Property 2

- The same patterns appear in different regions.

Property 3

- Subsampling the pixels will not change the object



Convolution

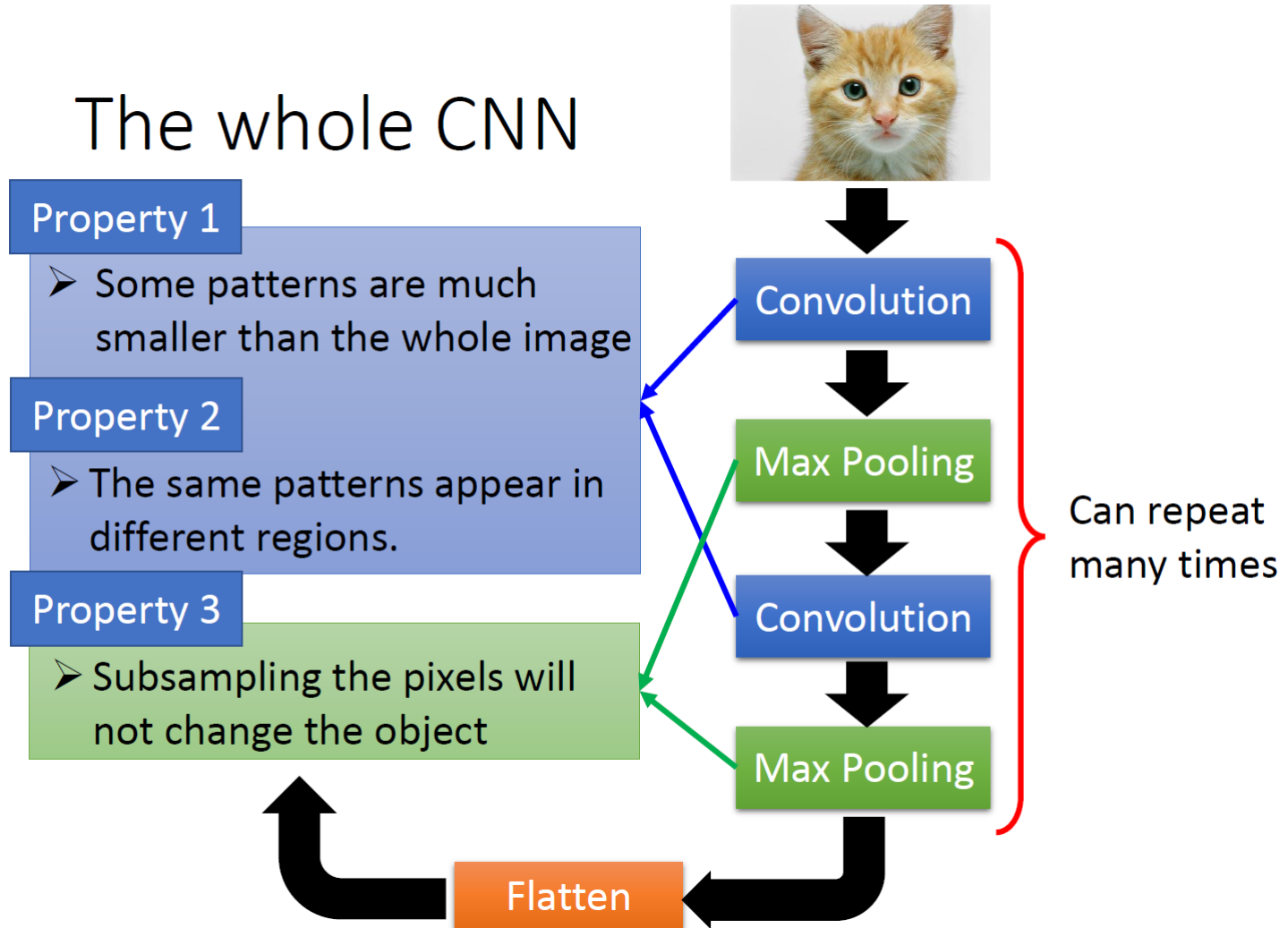
Max Pooling

Convolution

Max Pooling

Can repeat many times

Flatten



- ✓ 平移不变性
- ✓ 局部性



重新思考 - 稠密层

- 将输入和输出变形为矩阵(宽度, 高度)
- 将权重变形为4-D张量(h, w) 到 (h', w')

- $$h_{i,j} = \sum_{k,l} w_{i,j,k,l} x_{k,l} = \sum_{a,b} v_{i,j,a,b} x_{i+a,j+b}$$

v是w的重索引 k,l

$$v_{i,j,a,b} = w_{i,j,i+a,j+b}$$

原则 #1 - 平移不变性

$$h_{i,j} = \sum_{a,b} v_{i,j,a,b} x_{i+a,j+b}$$

- x 的变化也导致 h 的变化
- v 不应该依赖于 (i, j) , 所以

$$v_{i,j,a,b} = v_{a,b}$$

$$h_{i,j} = \sum_{a,b} v_{a,b} x_{i+a,j+b}$$

原则 #2 - 局部性

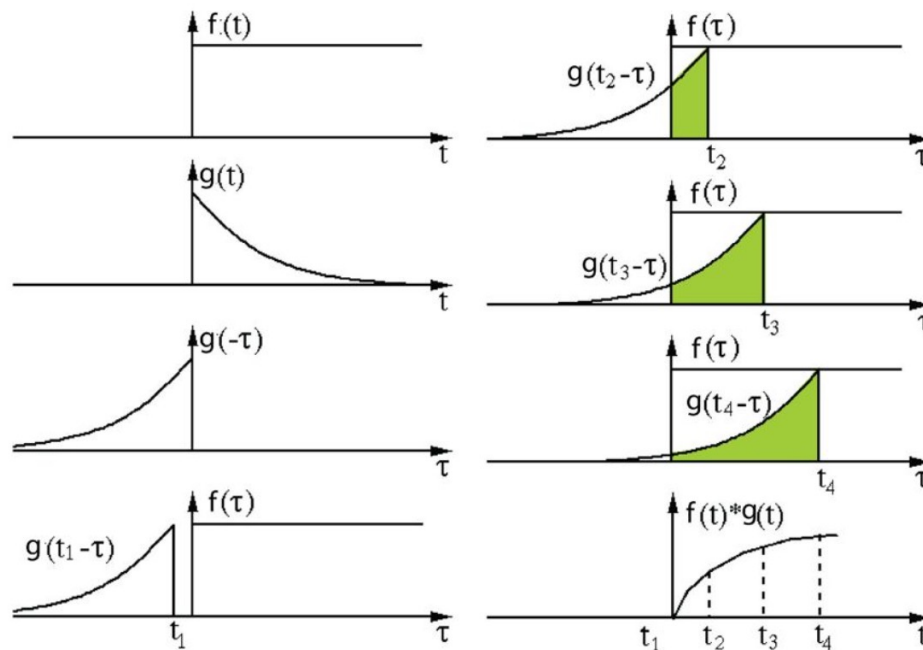
$$h_{i,j} = \sum_{a,b} v_{a,b} x_{i+a,j+b}$$

- 当评估 $h(i,j)$ 时，我们不应该用远离 $x(i, j)$ 的参数
- 外部参数 $|a|, |b| > \Delta$ 消失, $v_{a,b} = 0$

$$h_{i,j} = \sum_{a=-\Delta}^{\Delta} \sum_{b=-\Delta}^{\Delta} v_{a,b} x_{i+a,j+b}$$

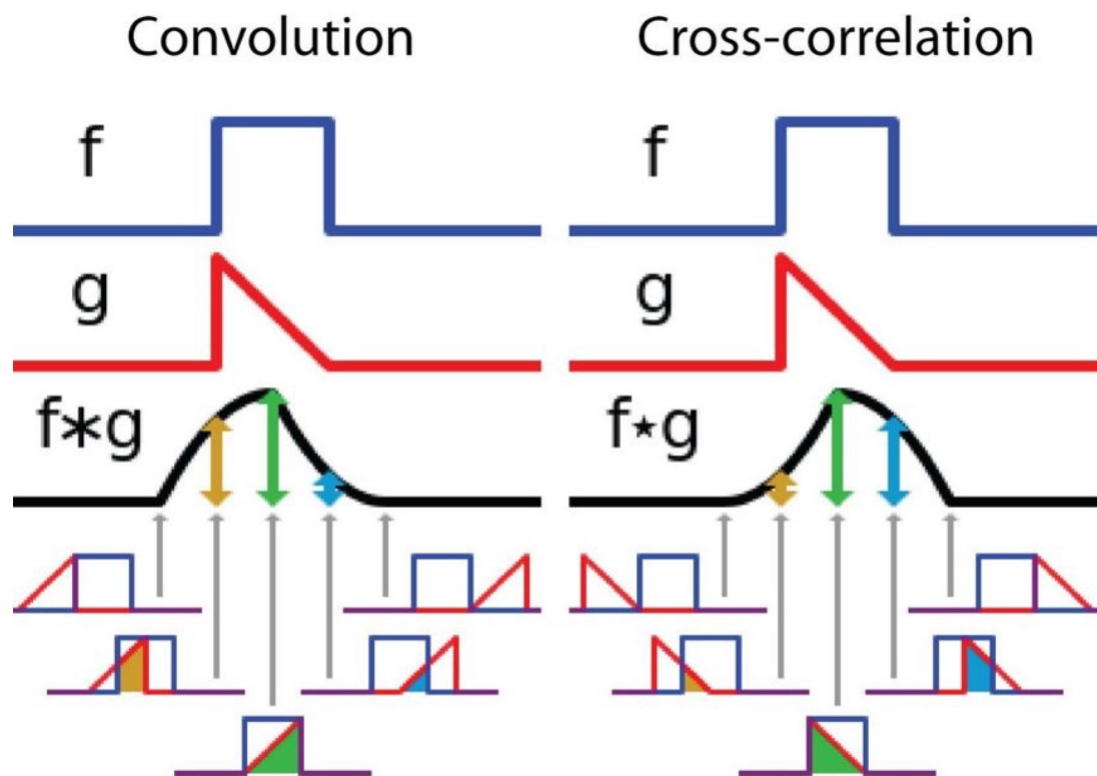
$$(f * g)(t) = \int_{-\infty}^{\infty} f(\tau)g(t - \tau)d\tau$$

卷积：两个函数中一个函数经过反转和位移后再相乘得到的积的积分



信号处理中的卷积。过滤器 g 经过反转，然后再沿水平轴滑动。在每一个位置，我们都计算 f 和反转后的 g 之间相交区域的面积。这个相交区域的面积就是特定位置出的卷积值。

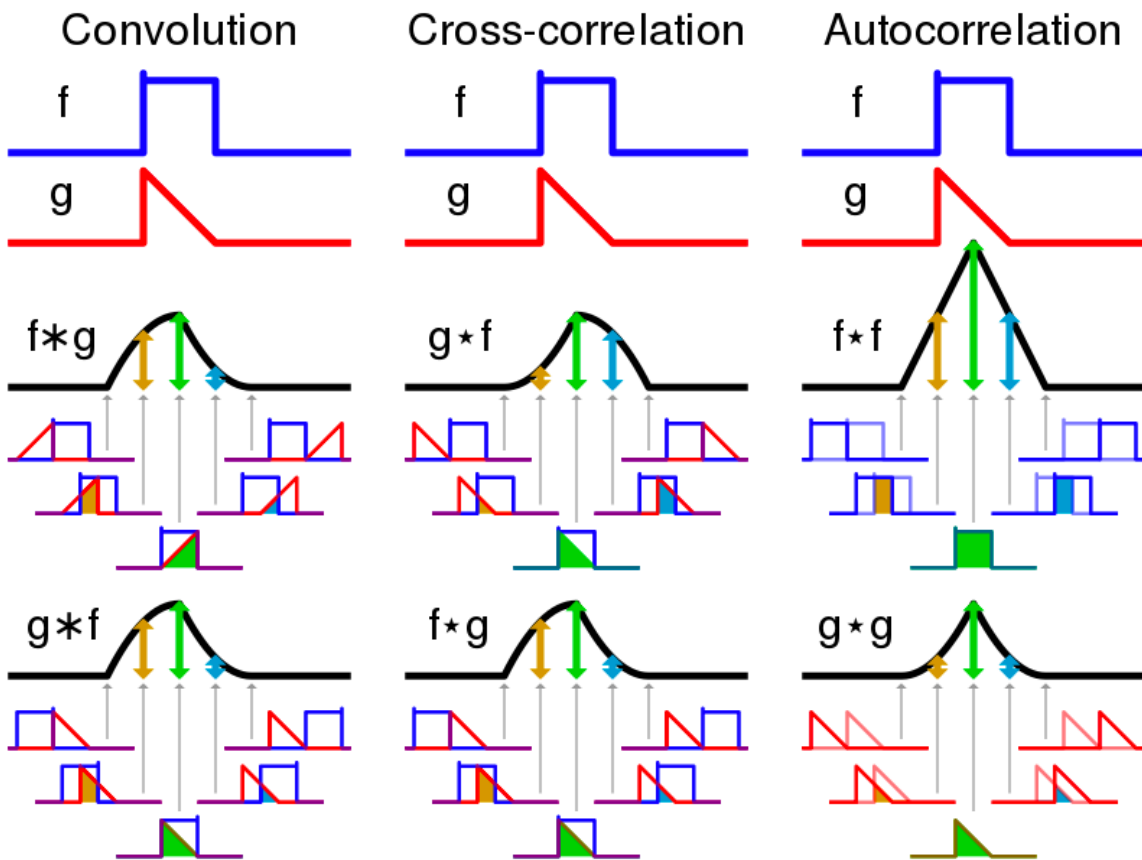
互相关是两个函数之间的滑动点积或滑动内积。互相关中的过滤器不经过反转，而是直接滑过函数 f 。 f 与 g 之间的交叉区域即是互相关。



信号处理中卷积与互相关之间的差异

在深度学习中，卷积中的滤波器不经过反转。严格来说，这是互相关。我们本质上是执行逐元素乘法和加法。但在深度学习中，直接将其称之为卷积更加方便。因为滤波器的权重是在训练阶段学习到的。如果上面例子中的反转函数 g 是正确的函数，那么经过训练后，学习得到的滤波器看起来就会像是反转后的函数 g 。因此，在训练之前，没必要像在真正的卷积中那样首先反转滤波器。

卷积层



二维交叉相关

Input

0	1	2
3	4	5
6	7	8

Kernel

0	1
2	3

*

=

Output

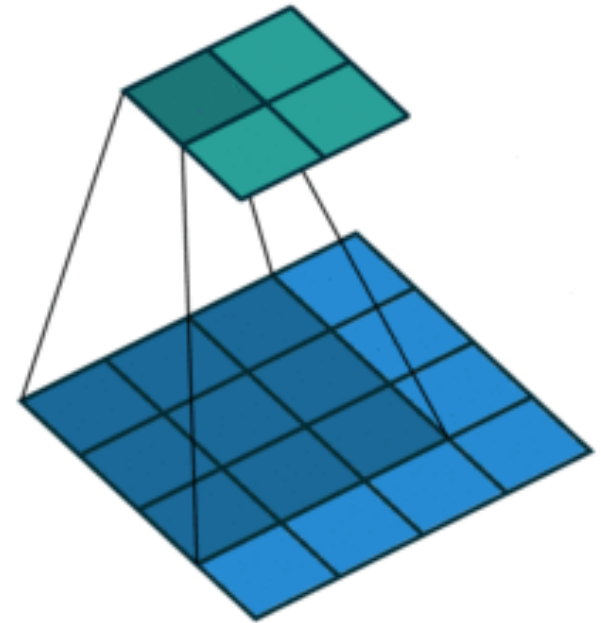
19	25
37	43

$$0 \times 0 + 1 \times 1 + 3 \times 2 + 4 \times 3 = 19,$$

$$1 \times 0 + 2 \times 1 + 4 \times 2 + 5 \times 3 = 25,$$

$$3 \times 0 + 4 \times 1 + 6 \times 2 + 7 \times 3 = 37,$$

$$4 \times 0 + 5 \times 1 + 7 \times 2 + 8 \times 3 = 43.$$



(vdumoulin@ Github)

二维卷积层

0	1	2
3	4	5
6	7	8

 *

0	1
2	3

 =

19	25
37	43

- $\mathbf{X}: n_h \times n_w$ 输入矩阵
- $\mathbf{W}: k_h \times k_w$ 核矩阵
- b : 偏差标量
- $\mathbf{Y}: (n_h - k_h + 1) \times (n_w - k_w + 1)$ 输出矩阵

$$\mathbf{Y} = \mathbf{X} \star \mathbf{W} + b$$

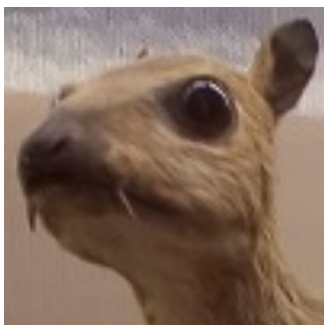
- $\mathbf{W}$ 和 b 是可学习的参数

例子

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

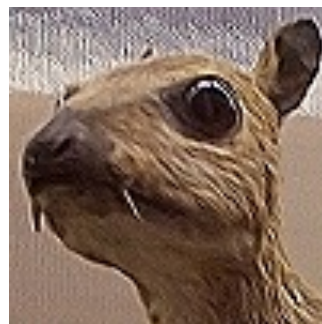


边缘检测



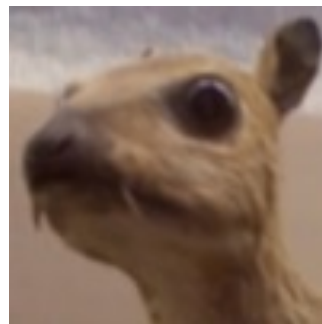
(维基百科)

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$



锐化

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

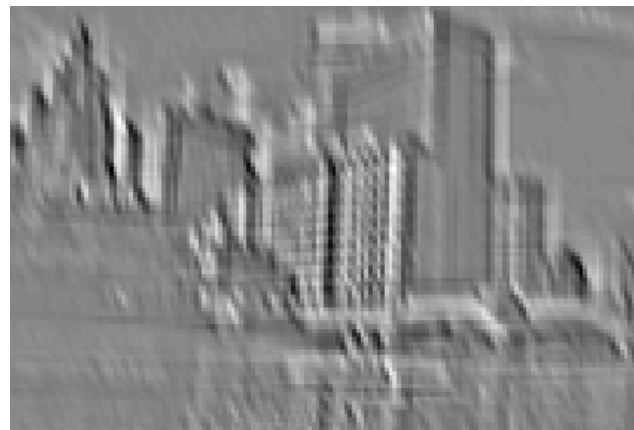


高斯模糊

例子



(Rob Fergus)



互相关与卷积

- 2-D 互相关

$$y_{i,j} = \sum_{a=1}^h \sum_{b=1}^w w_{a,b} x_{i+a,j+b}$$

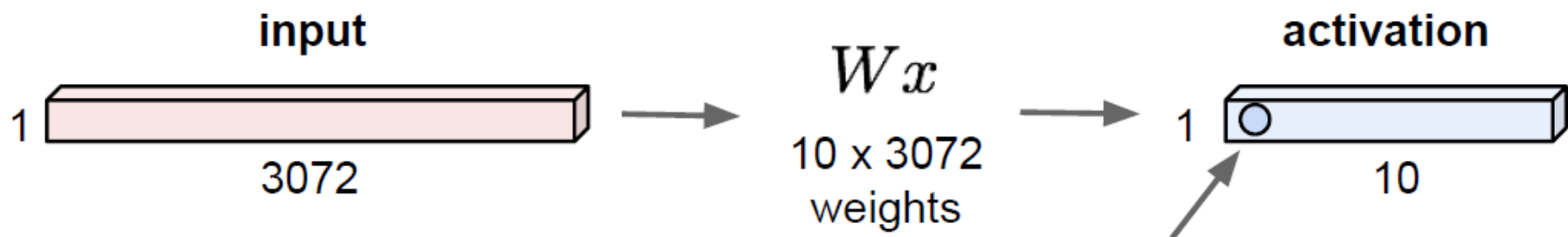
- 2-D 卷积

$$y_{i,j} = \sum_{a=1}^h \sum_{b=1}^w w_{-a,-b} x_{i+a,j+b}$$

- 在对称性方面没有差别

全连接层：

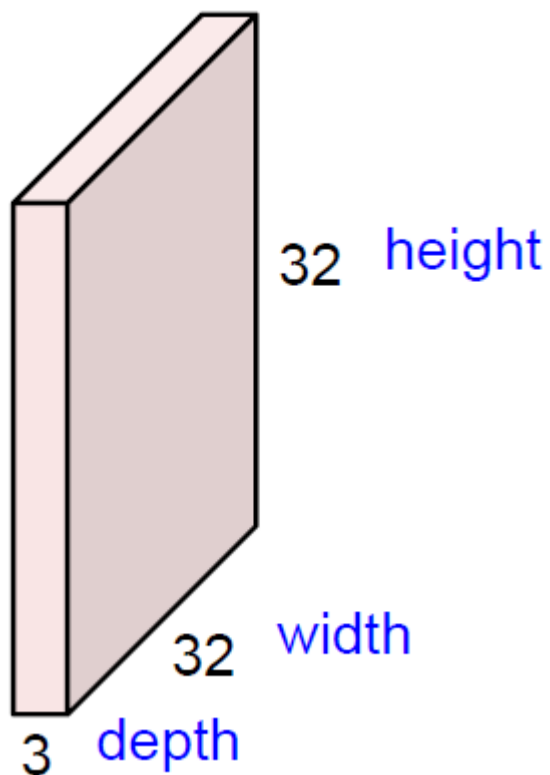
32x32x3 图像拉伸为 3072x1

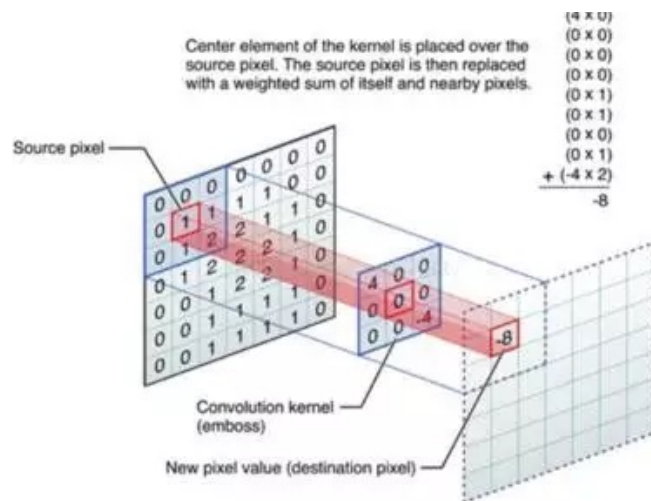


1 number:
在W行和输入之间取点积的结果(一个3072维点积结果)

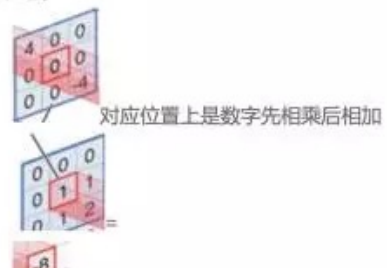
卷积层：

32x32x3 图像保留空间结构





分解下上图

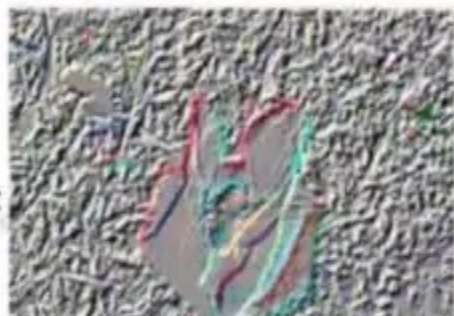




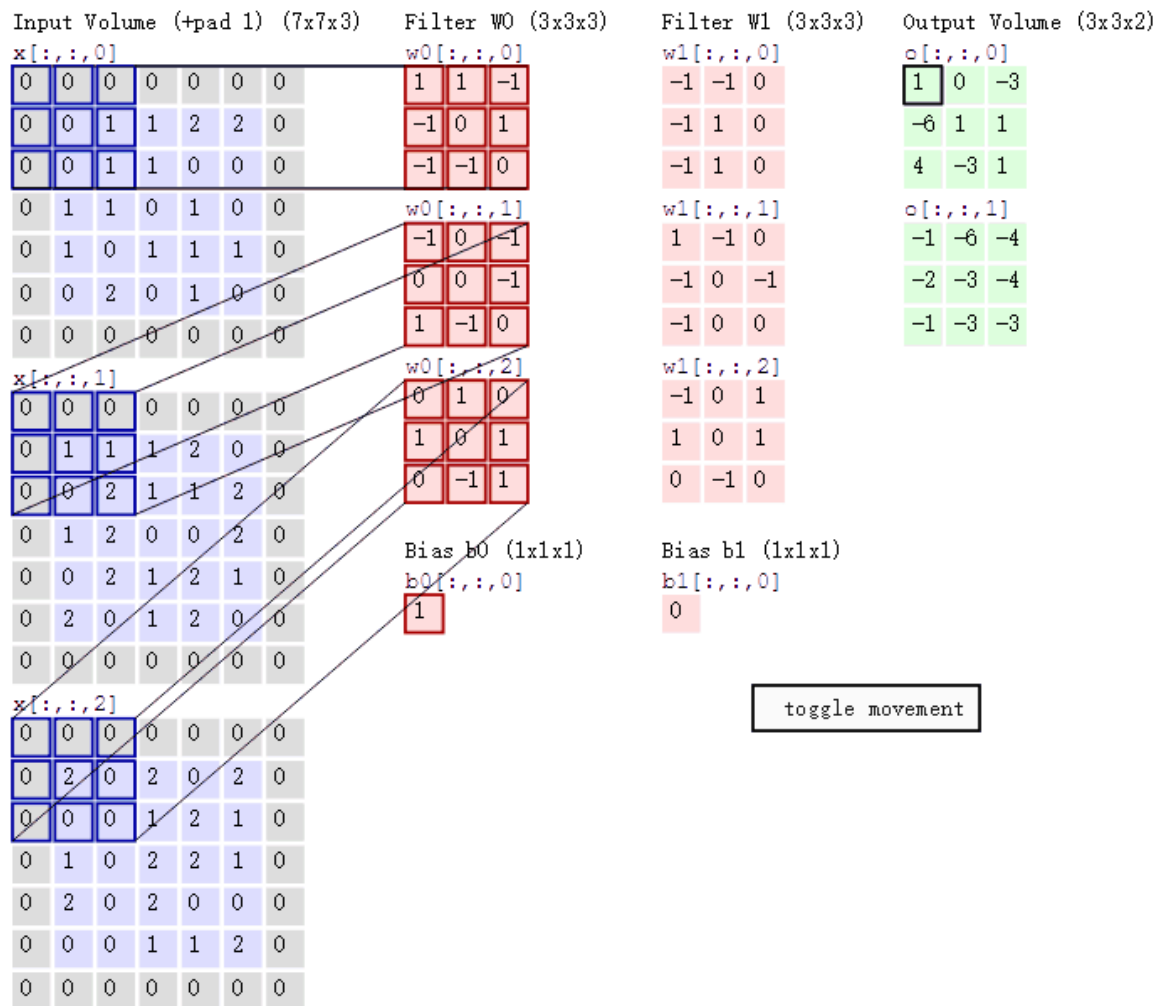
$$\begin{matrix} * & \begin{matrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{matrix} & = \end{matrix}$$



$$\begin{matrix} * & \begin{matrix} -1 & -1 & 0 \\ -1 & 0 & 1 \\ 0 & 1 & 1 \end{matrix} & = \end{matrix}$$



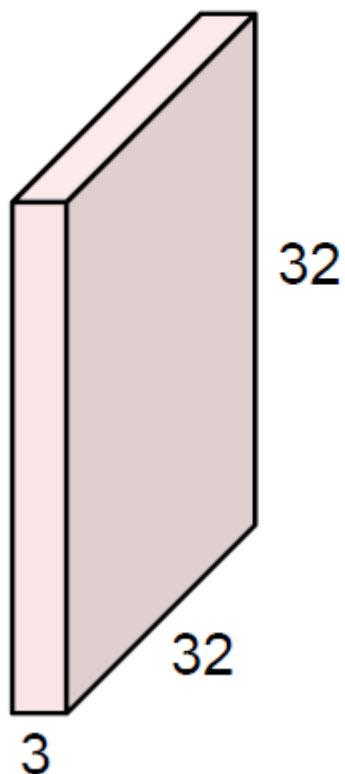
卷积



卷积层

滤波器总是延伸输入体积的全部深度

32x32x3 image

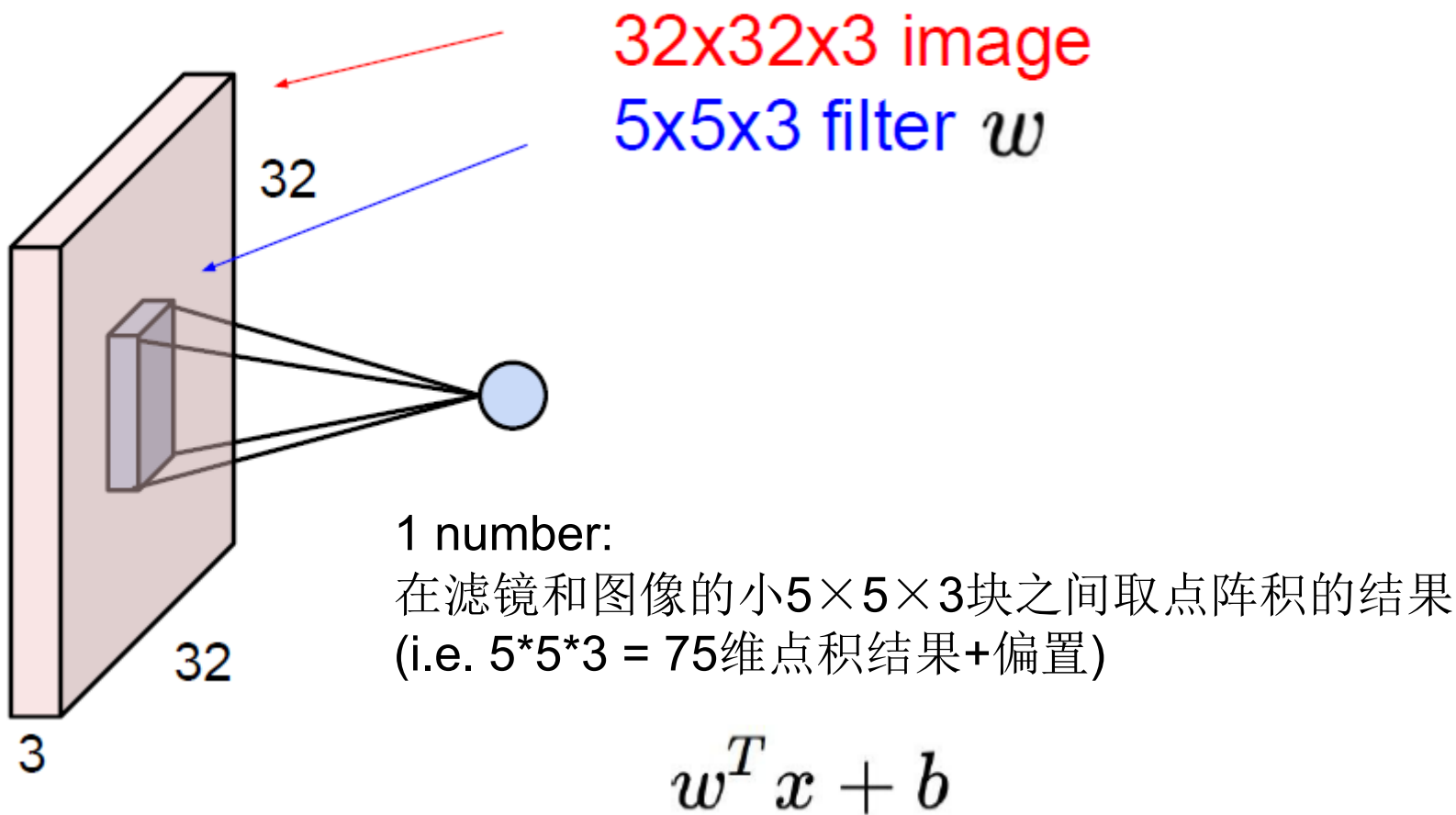


5x5x3 filter

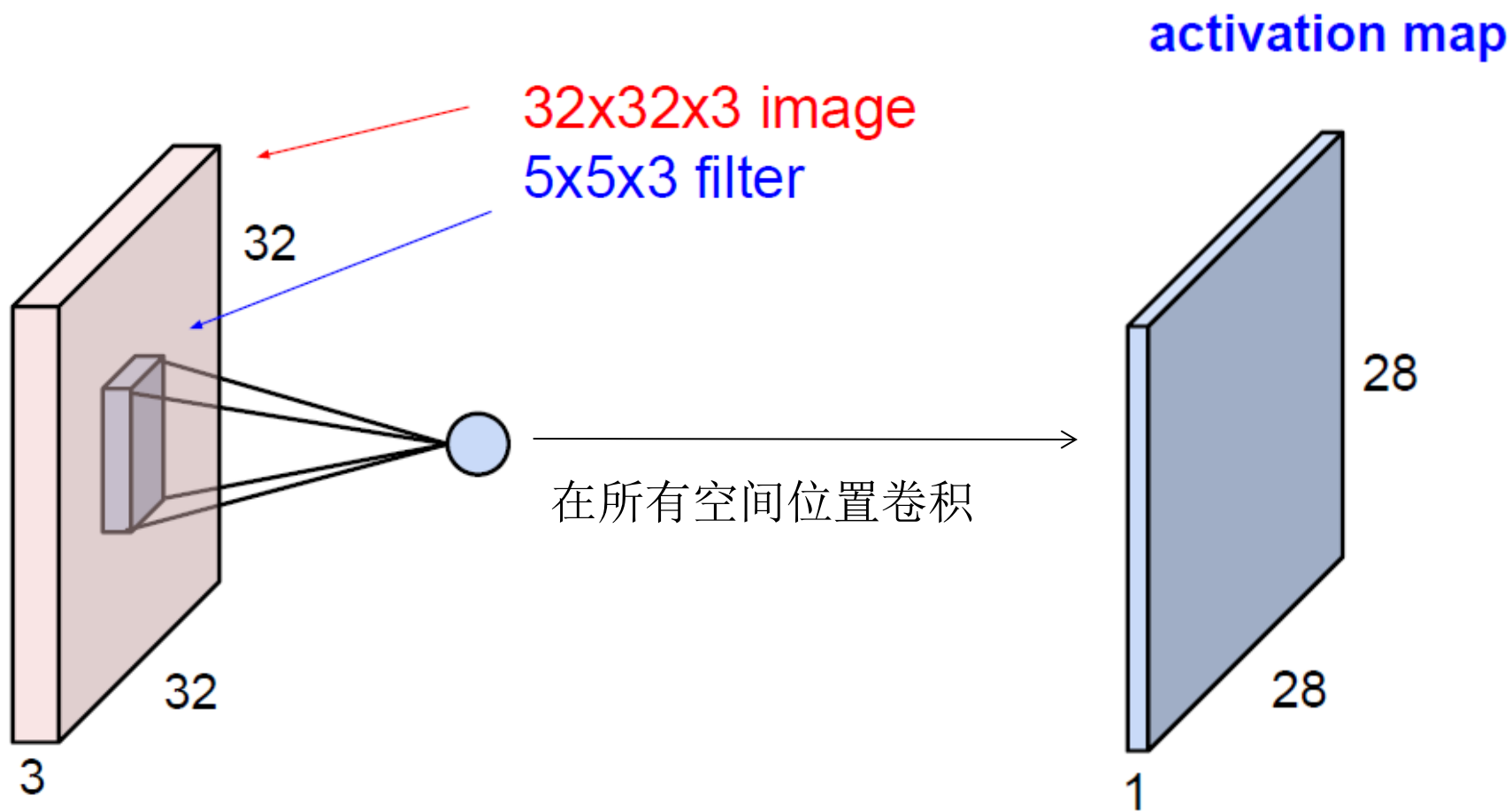


Convolve the filter with the image
i.e. “slide over the image spatially,
computing dot products”

卷积层

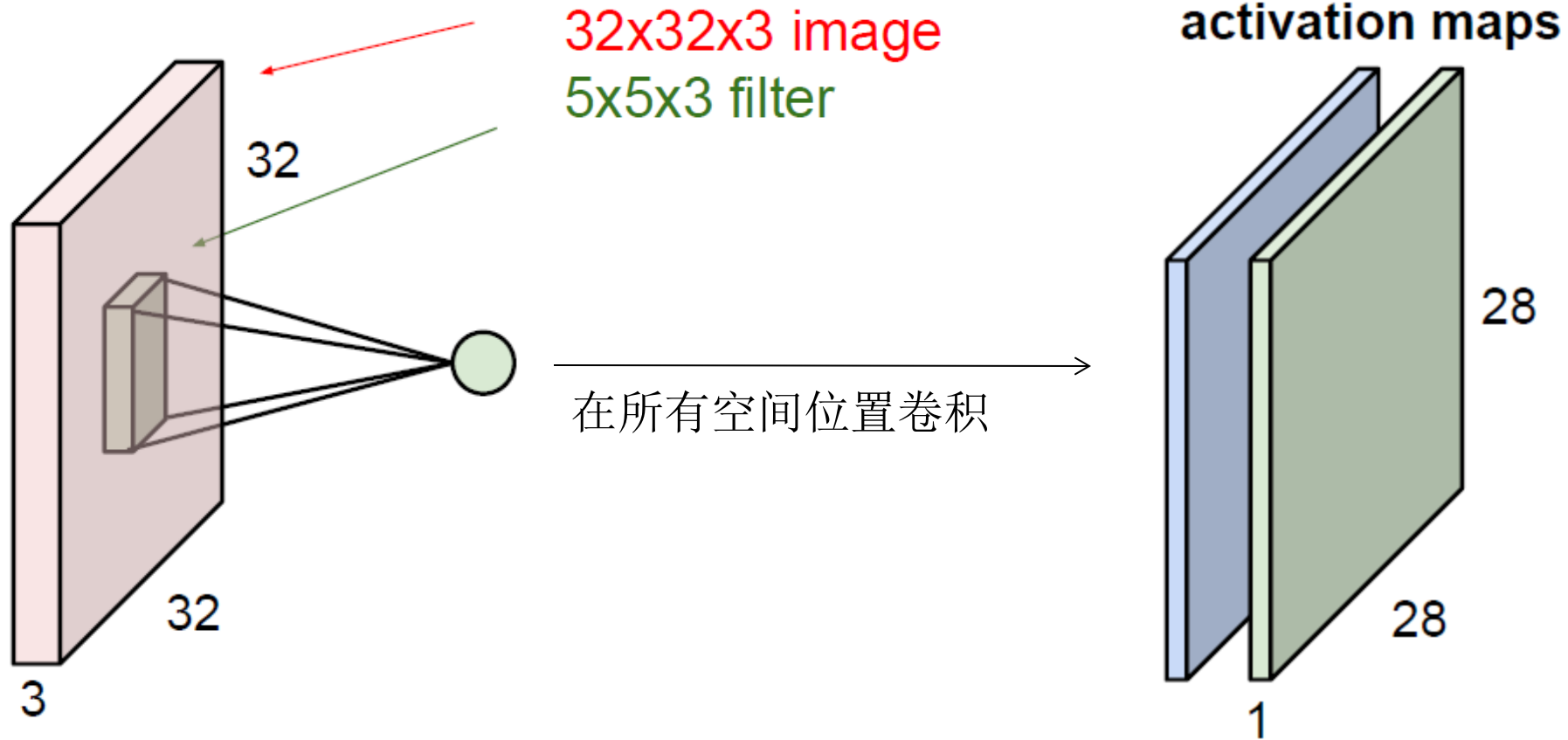


卷积层

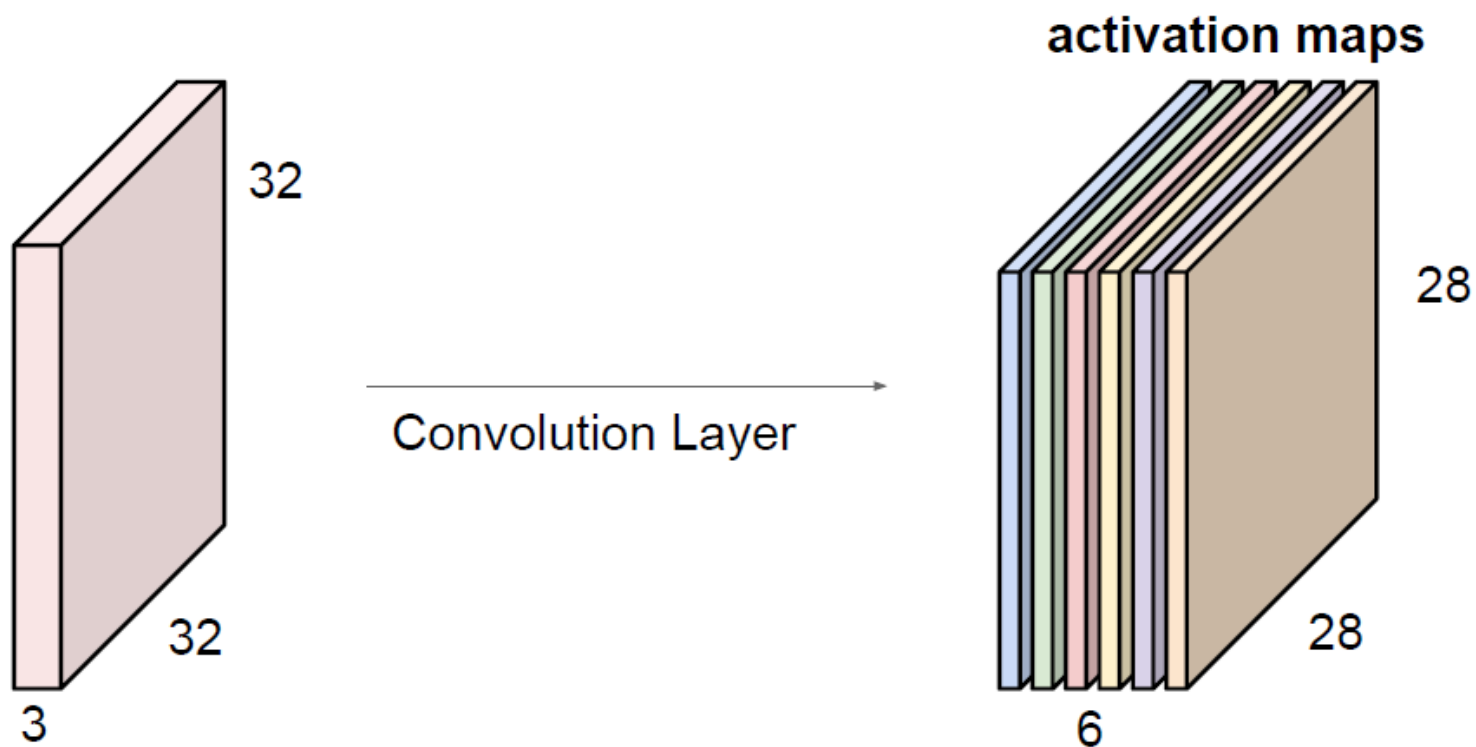


卷积层

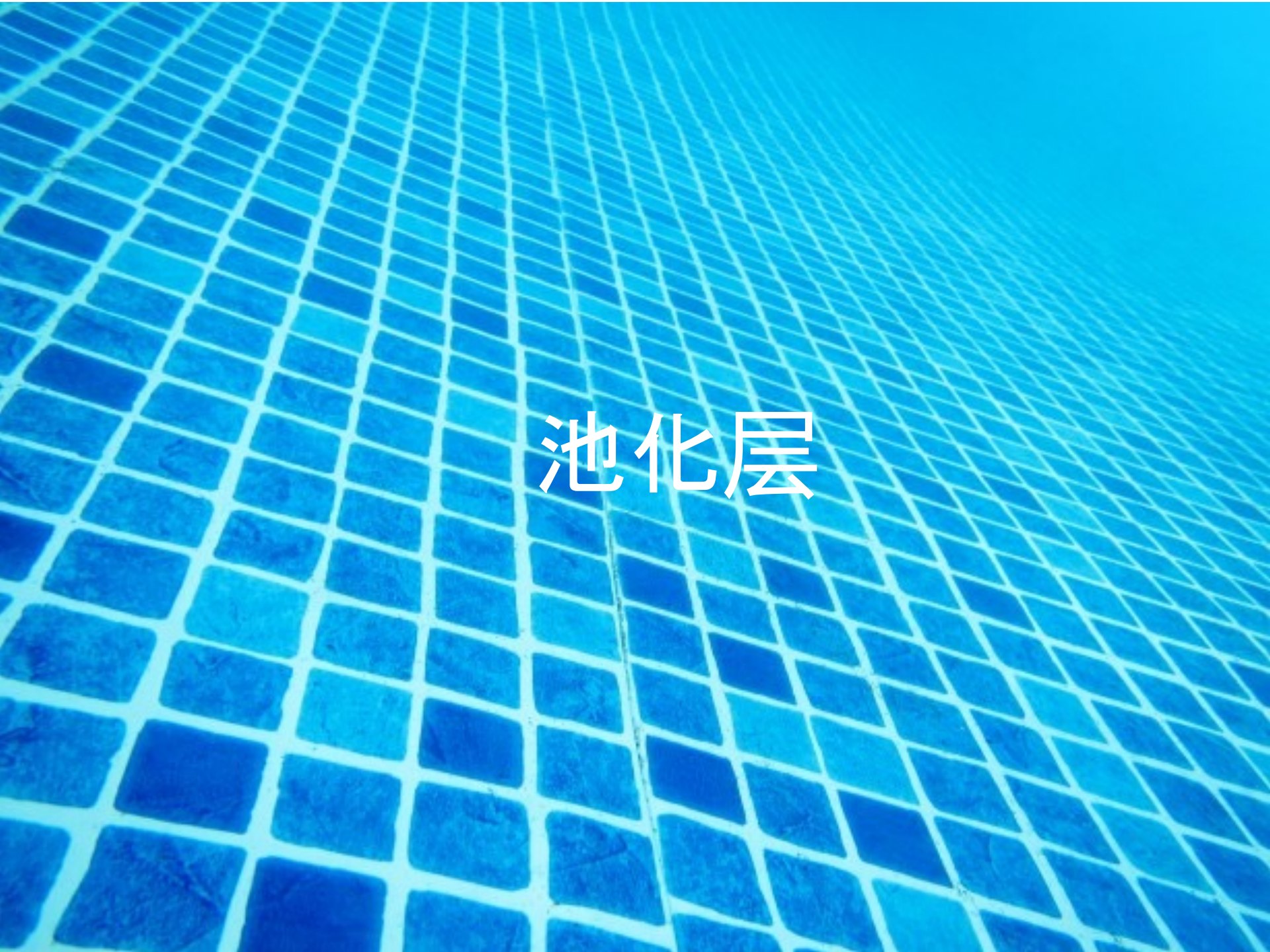
考虑到第二个,绿色的滤镜



可以使用多个卷积滤波器，例如，如果我们有6个5x5过滤器，我们将获得6个单独的激活映射：



我们堆叠这些，就可以获得一个28x28x6尺寸的“新形象”！



池化层

池化层

- 卷积对位置敏感
 - 检测垂直边缘

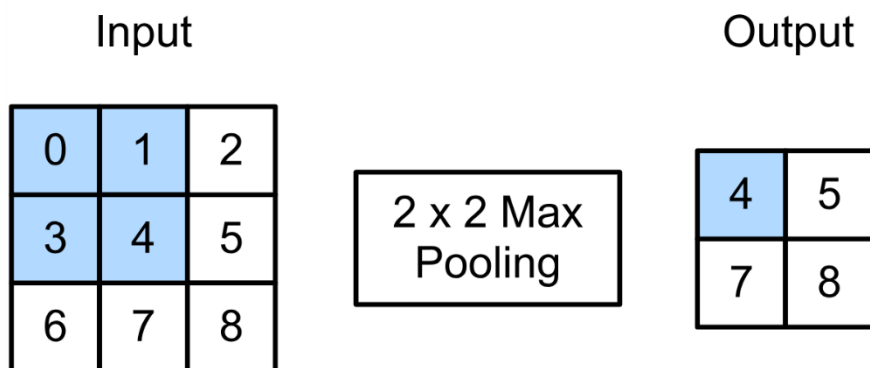
$$\begin{array}{c} X \\ \begin{bmatrix} 1. & 1. & 0. & 0. & 0. \\ 1. & 1. & 0. & 0. & 0. \\ 1. & 1. & 0. & 0. & 0. \\ 1. & 1. & 0. & 0. & 0. \end{bmatrix} \end{array} \quad \begin{array}{c} Y \\ \begin{bmatrix} 0. & 1. & 0. & 0. \\ 0. & 1. & 0. & 0. \\ 0. & 1. & 0. & 0. \\ 0. & 1. & 0. & 0. \end{bmatrix} \end{array}$$

输出，1像素移位

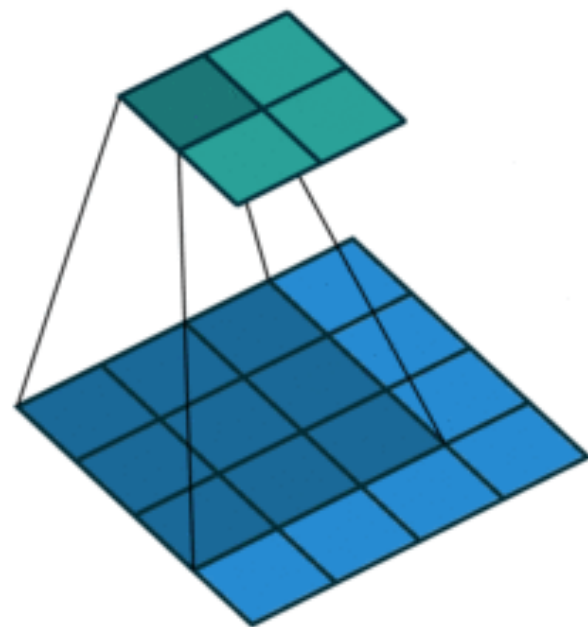
- 需要一定程度的平移不变性
 - 照明，物体位置，比例，外观等等因图像而异

2-D 最大池化

- 返回滑动窗口中的最大值



$$\max(0, 1, 3, 4) = 4$$



2-D 最大池化

- 返回滑动窗口中的最大值

垂直边缘检测

```
[[1. 1. 0. 0. 0.
 [1. 1. 0. 0. 0.
 [1. 1. 0. 0. 0.
 [1. 1. 0. 0. 0.
```

卷积输出

```
[[ 0.  1.  0.  0.
 [ 0.  1.  0.  0.
 [ 0.  1.  0.  0.
 [ 0.  1.  0.  0.
```

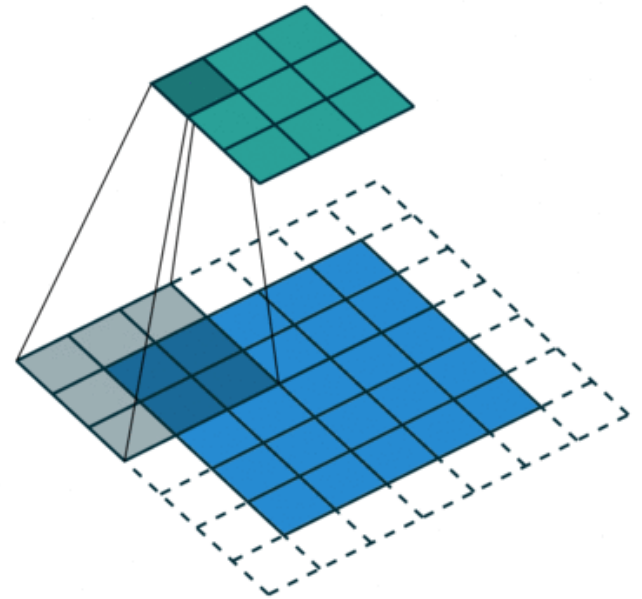
2-D 最大池化

```
[[ 1.  1.  1.  0.
 [ 1.  1.  1.  0.
 [ 1.  1.  1.  0.
 [ 1.  1.  1.  0.
```

可容1像素移位

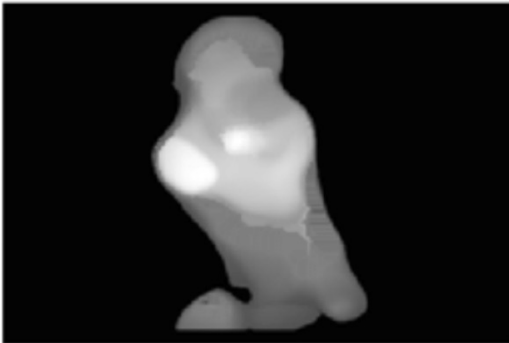
池化层 - 填充, 步幅和多个通道

- 池化层与卷积层类似, 都具有填充和步幅
- 没有可学习的参数
- 在每个输入通道应用池化层以获得相应的输出通道
- $\# \text{输出通道} = \# \text{输入通道}$

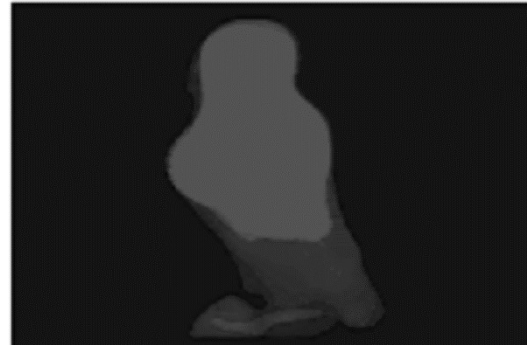


平均池化层

- 最大池化层: 每个窗口中最强的模式信号
- 平均池化层:
 - 将最大池化层中的“最大”操作替换为“平均”
 - 窗口中的平均信号强度



最大池化层



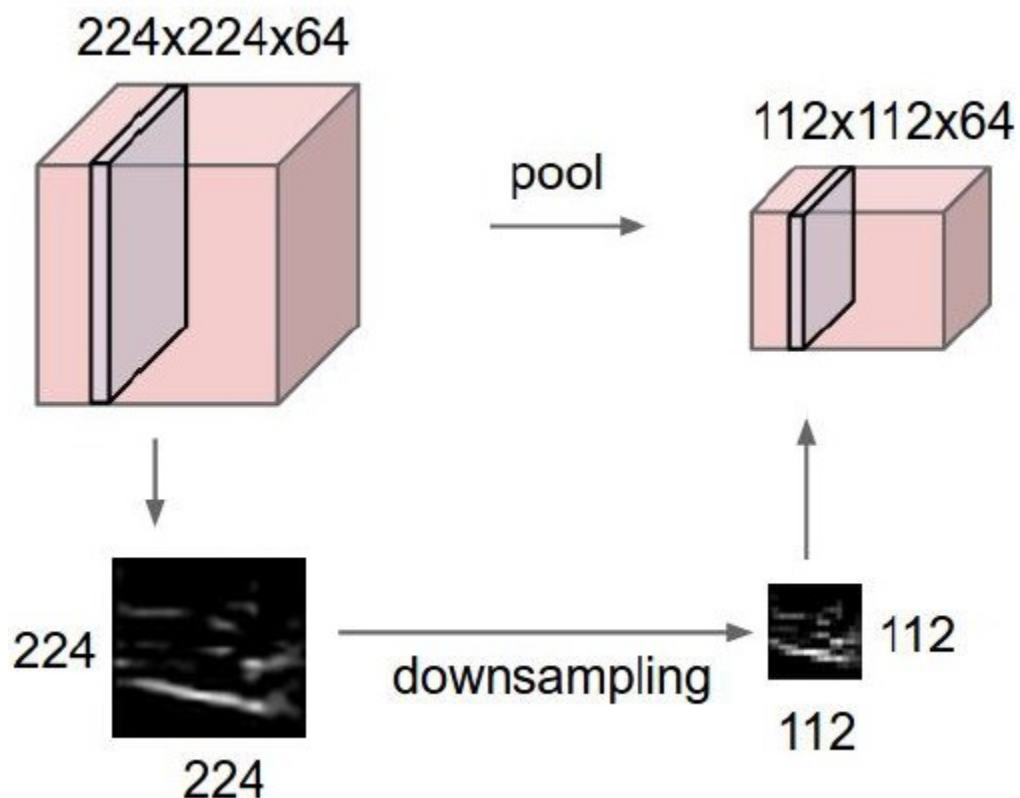
平均池化层

总结

- 卷积层
 - 与稠密层相比, 模型容量降低
 - 有效地检测空间模式
 - 计算复杂度高
 - 通过填充, 步幅和通道控制输出形状
- 最大 / 平均池化层
 - 提供一定程度的平移不变性

池化层

- 使表示更小更易于管理
- 独立操作每个激活图



最大池化

Single depth slice

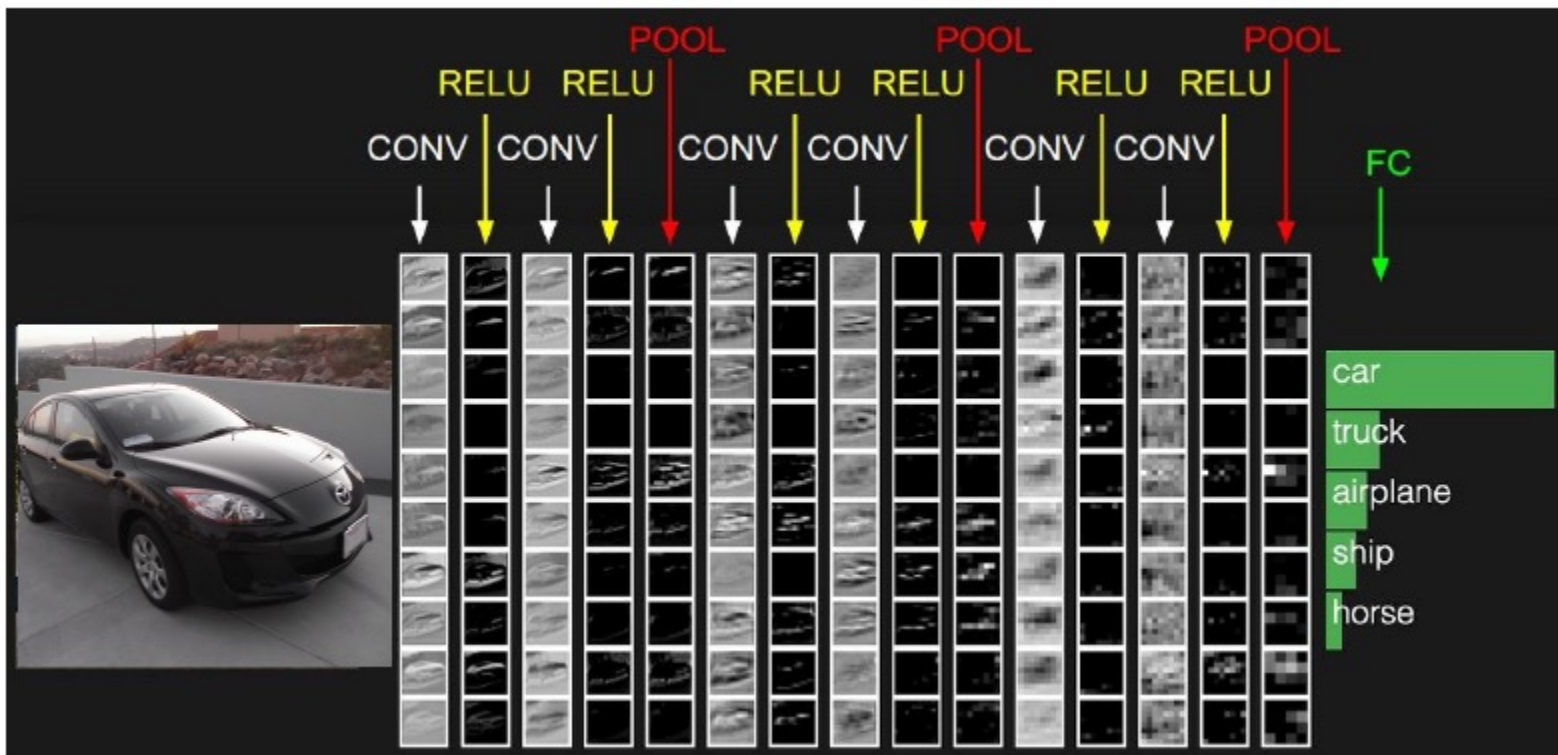
1	1	2	4
5	6	7	8
3	2	1	0
1	2	3	4

max pool with 2x2 filters
and stride 2

6	8
3	4

全连接层(FC layer)

-包含连接到整个输入卷的神经元，如在普通神经网络中



CNN模型详解

概要

□ LeNet （第一个卷积神经网络）

□ AlexNet

- ✓ 升级版 LeNet
- ✓ ReLu 激活，丢弃法，平移不变性

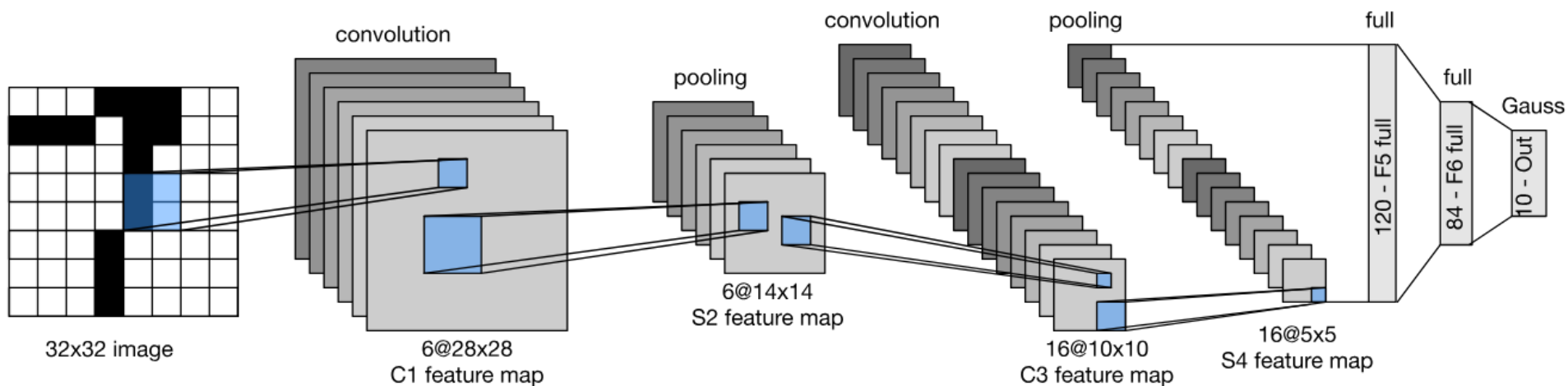
□ VGG

- ✓ 升华版的 AlexNet
- ✓ 重复的 VGG 块

□ NiN

- ✓ 1×1 卷积 + 全局池化

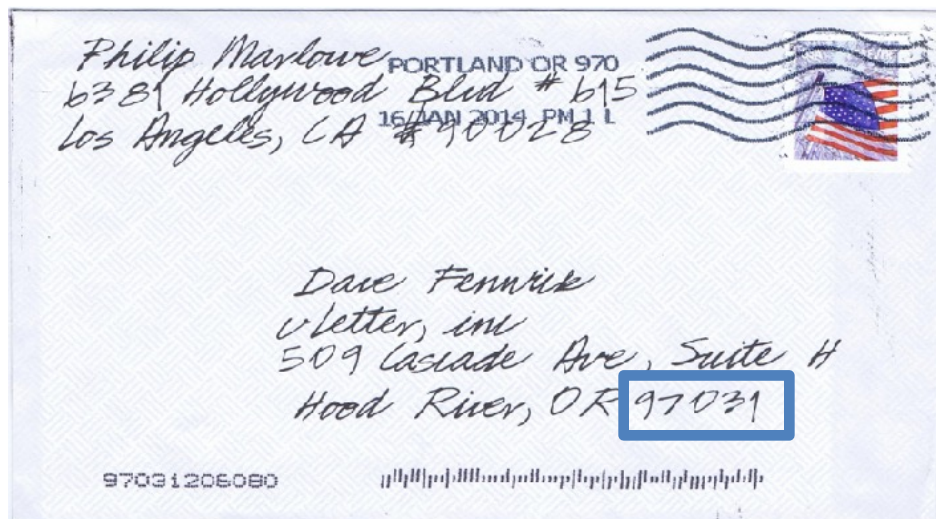
LeNet 架构



LeNet是深度CNN网络的基石，AlexNet、VGG、GoogLeNet、ResNet等都是在VGG基础上加入各类激活函数或加深网络演变而来的

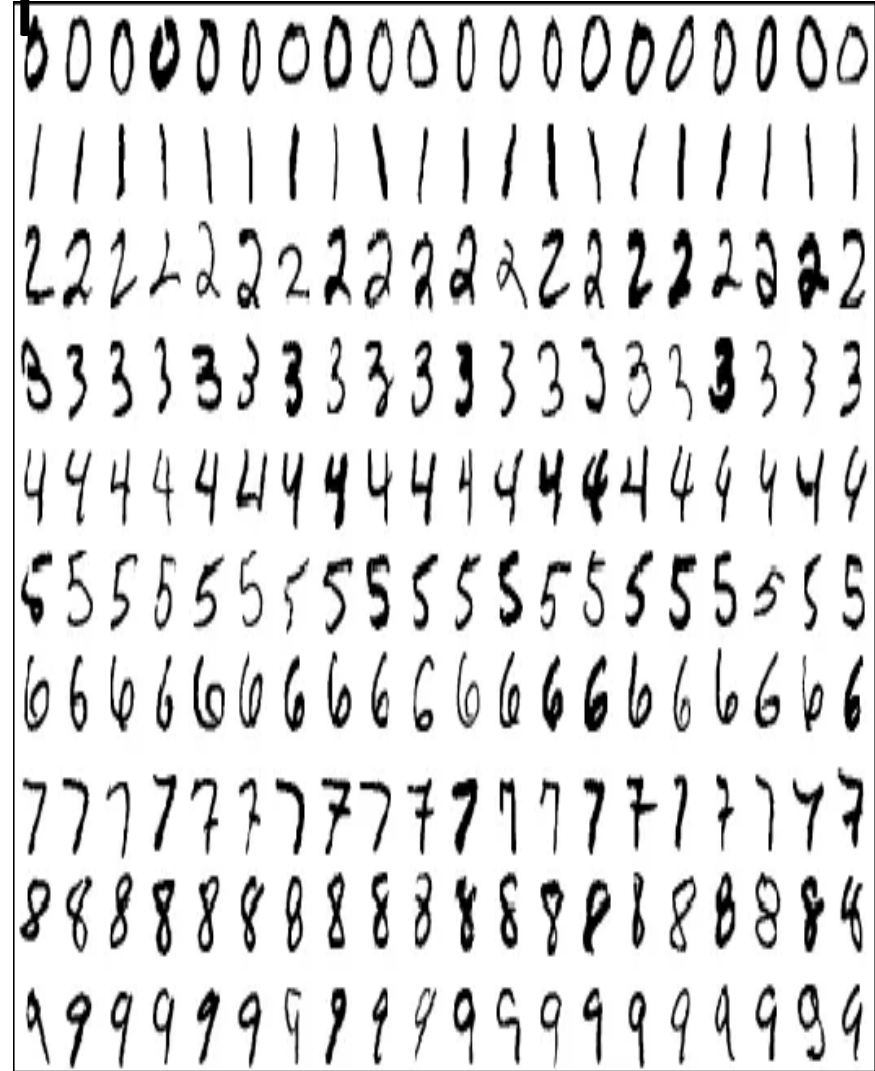
Lecun Y , Bottou L . Gradient-based learning applied to document recognition[J]. Proceedings of the IEEE, 1998, 86(11):2278-2324.

手写的数字 识别

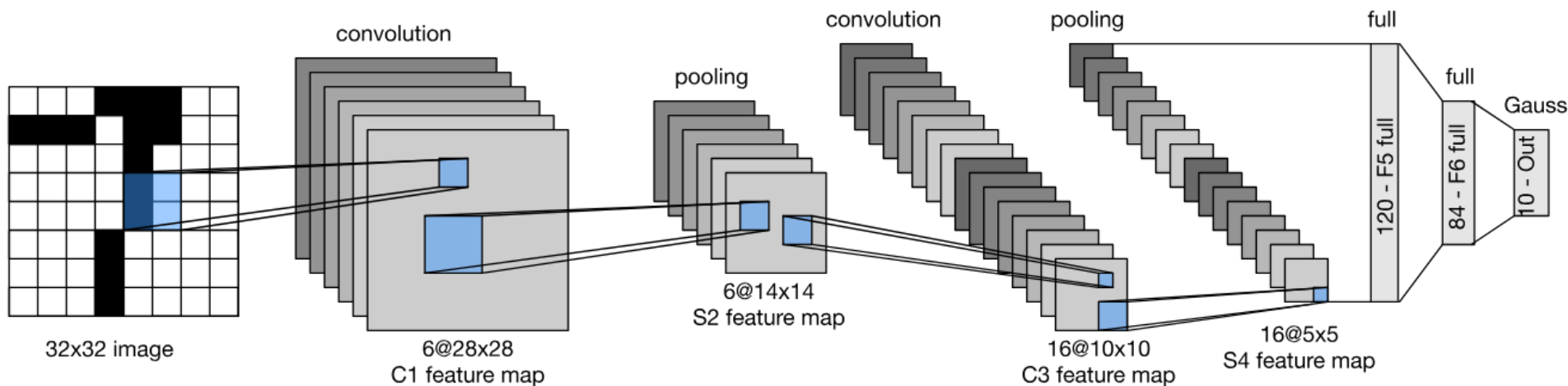


MNIST

- ✓ 居中和缩放
- ✓ 50,000 个训练数据
- ✓ 10,000 个测试数据
- ✓ 图像大小 28×28
- ✓ 10 类



LeNet 架构



- ✓ LeNet共有7层，其中有3个卷积层，2个子采样层，1个全连接层和1个高斯连接层。上图中用C代表卷积层，用S代表采样层，用F代表全连接层。图片的输入size固定在32*32，，其实为28*28图片填充后的结果。
- ✓ LeNet图片的输入仅是规范化后的二值图像。网络的输出为0~9十个数字的RBF度量。



0
103



Y. LeCun, L. Bottou, Y.
Bengio, P. Haffner,
1998

Gradient-based
learning applied to
document
recognition

Pytorch 中的 LeNet

```
1 import time
2 import torch
3 from torch import nn, optim
4
5 import sys
6 sys.path.append("../")
7 import d2lzh_pytorch as d2l
8 device = torch.device('cuda' if torch.cuda.is_available() else
9 'cpu')
10
11 class LeNet(nn.Module):
12     def __init__(self):
13         super(LeNet, self).__init__()
14         self.conv = nn.Sequential(
15             nn.Conv2d(1, 6, 5), # in_channels, out_channels,
16                                # kernel_size, stride
17             nn.Sigmoid(),
18             nn.MaxPool2d(2, 2), # kernel_size, stride
19             nn.Conv2d(6, 16, 5),
20             nn.Sigmoid(),
21             nn.MaxPool2d(2, 2)
22         )
```

Pytorch 中的 LeNet

```
21         self.fc = nn.Sequential(  
22             nn.Linear(16*4*4, 120),  
23             nn.Sigmoid(),  
24             nn.Linear(120, 84),  
25             nn.Sigmoid(),  
26             nn.Linear(84, 10)  
27         )  
28  
29     def forward(self, img):  
30         feature = self.conv(img)  
31         output = self.fc(feature.view(img.shape[0], -1))  
32         return output
```

```
1 net = LeNet()  
2 print(net)
```

Pytorch 中的 LeNet

```
1 LeNet(  
2     (conv): Sequential(  
3         (0): Conv2d(1, 6, kernel_size=(5, 5), stride=(1, 1))  
4         (1): Sigmoid()  
5         (2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,  
        ceil_mode=False)  
6         (3): Conv2d(6, 16, kernel_size=(5, 5), stride=(1, 1))  
7         (4): Sigmoid()  
8         (5): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,  
        ceil_mode=False)  
9     )  
10    (fc): Sequential(  
11        (0): Linear(in_features=256, out_features=120, bias=True)  
12        (1): Sigmoid()  
13        (2): Linear(in_features=120, out_features=84, bias=True)  
14        (3): Sigmoid()  
15        (4): Linear(in_features=84, out_features=10, bias=True)  
16    )  
17 )
```

Pytorch 中的 LeNet

```
1 batch_size = 256
2 train_iter, test_iter =
  d2l.load_data_fashion_mnist(batch_size=batch_size)

1 # 本函数已保存在d2lzh_pytorch包中方便以后使用
2 def train_ch5(net, train_iter, test_iter, batch_size, optimizer,
  device, num_epochs):
3     net = net.to(device)
4     print("training on ", device)
5     loss = torch.nn.CrossEntropyLoss()
6     batch_count = 0
7     for epoch in range(num_epochs):
8         train_l_sum, train_acc_sum, n, start = 0.0, 0.0, 0,
  time.time()
9         for X, y in train_iter:
10             X = X.to(device)
11             y = y.to(device)
12             y_hat = net(X)
13             l = loss(y_hat, y)
14             optimizer.zero_grad()
15             l.backward()
16             optimizer.step()
17             train_l_sum += l.cpu().item()
18             train_acc_sum += (y_hat.argmax(dim=1) ==
  y).sum().cpu().item()
19             n += y.shape[0]
20             batch_count += 1
21         test_acc = evaluate_accuracy(test_iter, net)
```

Pytorch 中的 LeNet

```
22         print('epoch %d, loss %.4f, train acc %.3f, test acc %.3f,  
23             % (epoch + 1, train_l_sum / batch_count,  
train_acc_sum / n, test_acc, time.time() - start))
```

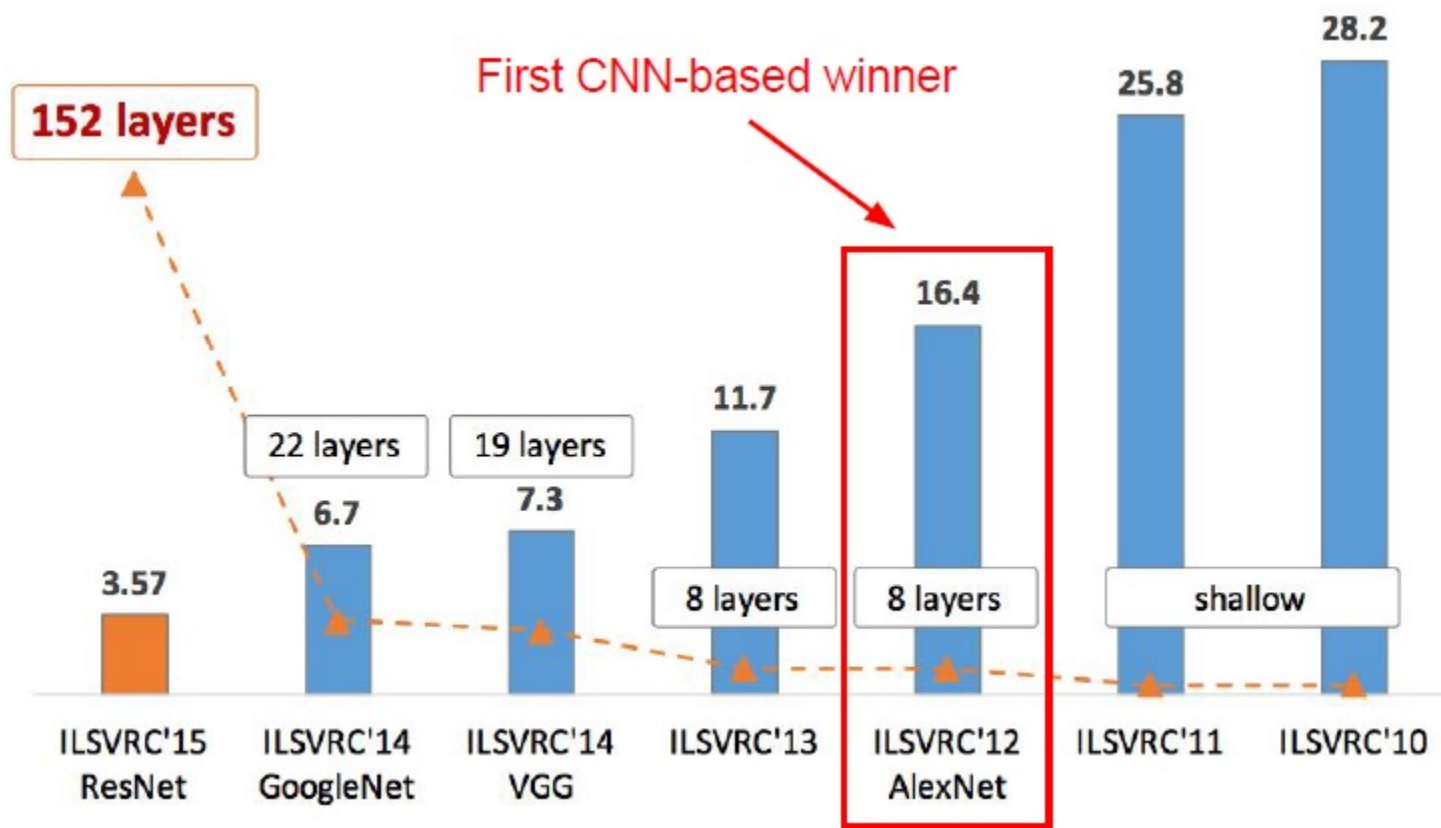
```
1 lr, num_epochs = 0.001, 5  
2 optimizer = torch.optim.Adam(net.parameters(), lr=lr)  
3 train_ch5(net, train_iter, test_iter, batch_size, optimizer, device,  
num_epochs)
```

ImageNet 数据集 (2010)

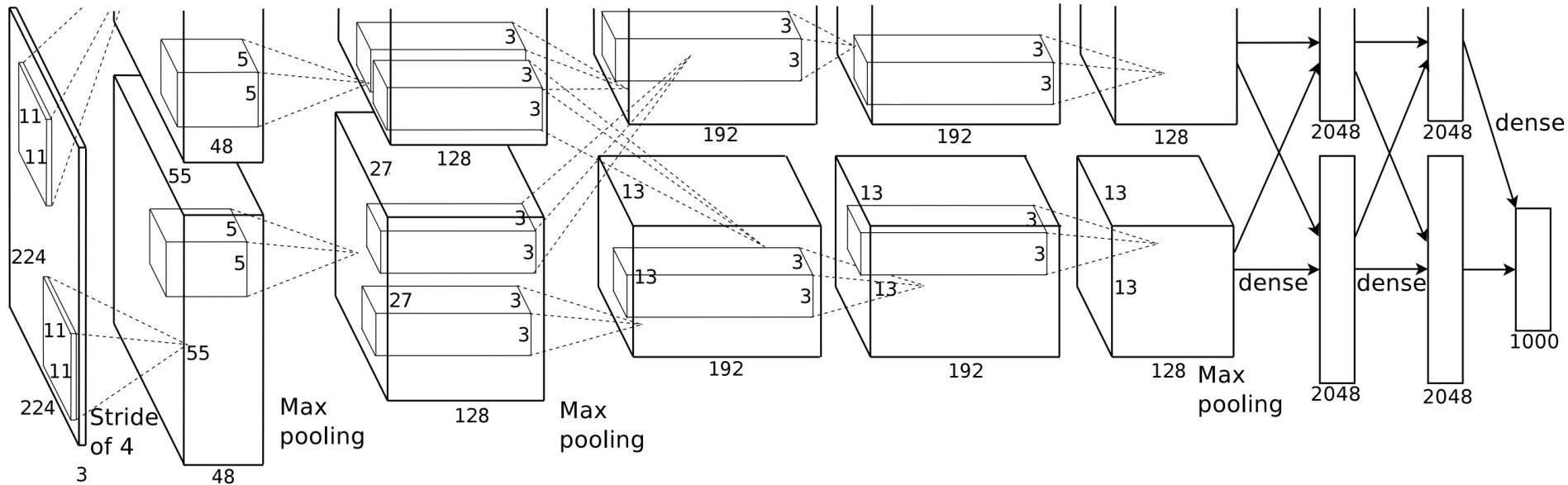


图像	自然物体的彩色图像	手写数字的灰色图像
尺寸	469 x 387	28 x 28
# 样本数	1200万	6万
# 类别数	1,000	10

大型视觉识别挑战赛（ILSVRC）获奖者



AlexNet



Krizhevsky A , Sutskever I , Hinton G . ImageNet Classification with Deep Convolutional Neural Networks[C], NIPS. Curran Associates Inc. 2012.

AlexNet

Architecture:

CONV1

MAX POOL1

NORM1

CONV2

MAX POOL2

NORM2

CONV3

CONV4

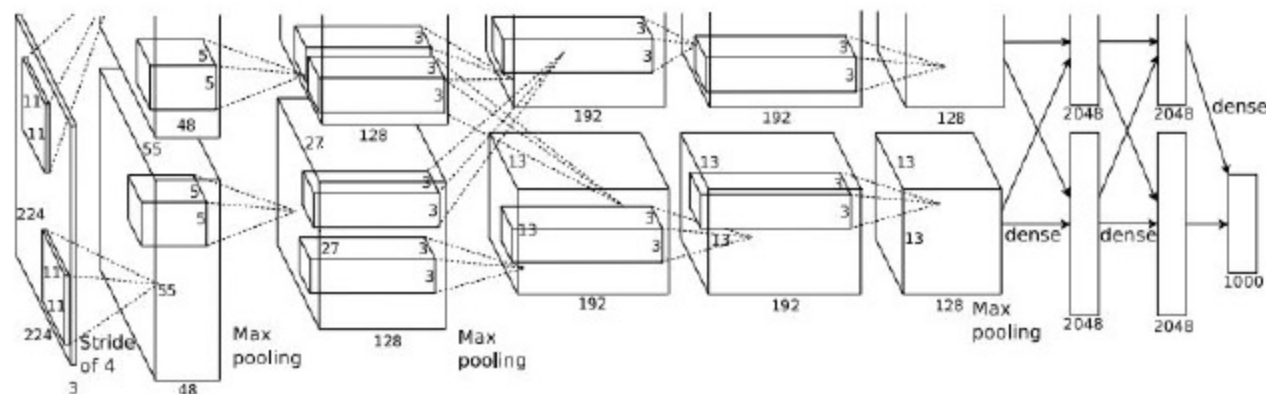
CONV5

Max POOL3

FC6

FC7

FC8

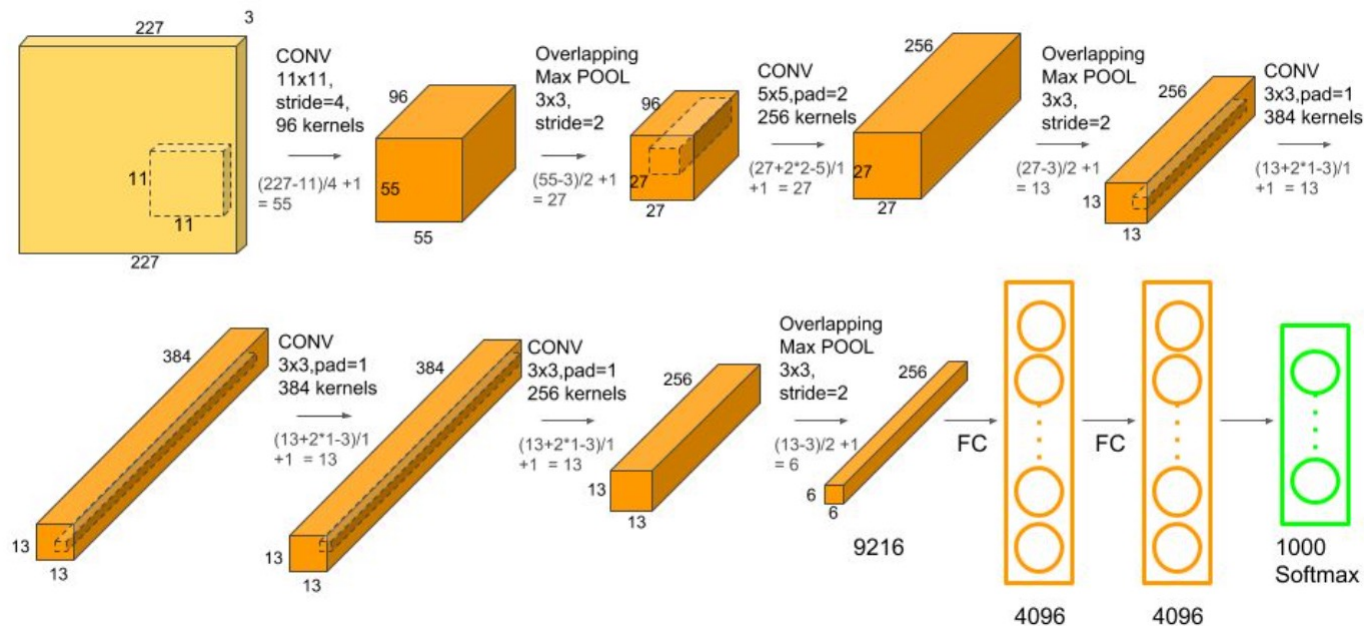


图分为上下两个部分的网络，这两部分网络是分别对应两个GPU，只有到了特定的网络层后才需要两块GPU进行交互，这种设置完全是利用两块GPU来提高运算的效率

AlexNet



AlexNet



输入： 227x227x3 图像

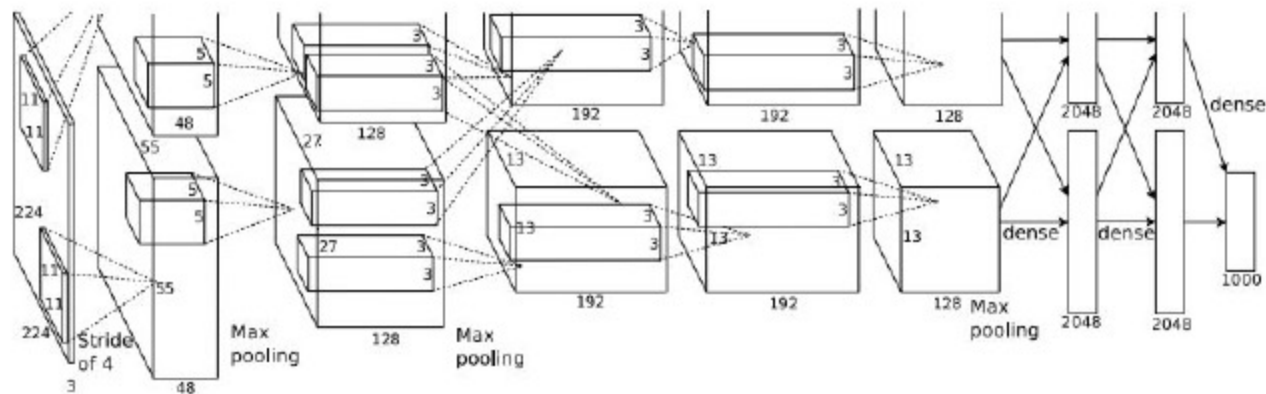
First layer (CONV1): 96 11x11 过滤器，应用在步幅4上

=>

输出体积： [55x55x96]

参数： $(11*11*3)*96 = 35K$

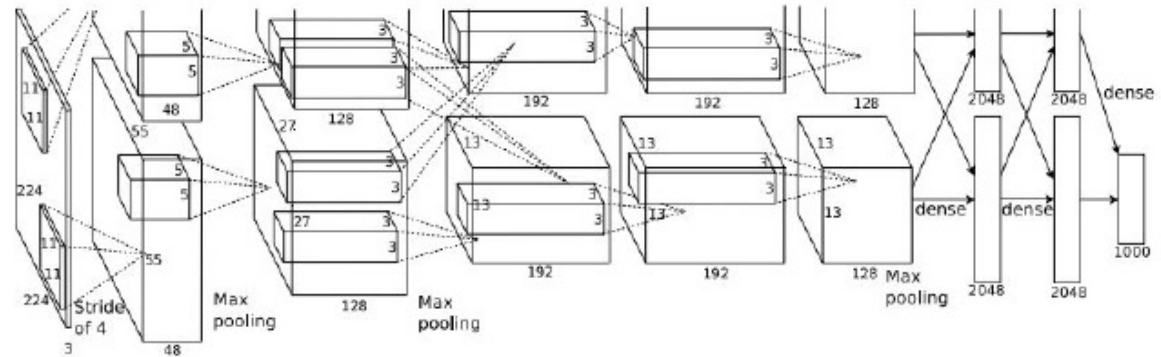
AlexNet



输入： 227x227x3 图像
通过 CONV1: 55x55x96

第二层 (POOL1): 3x3 过滤器，应用在步幅2上
输出体积： [27x27x96]
参数： 0

AlexNet



Full (simplified) AlexNet architecture:

[227x227x3] INPUT

[55x55x96] **CONV1**: 96 11x11 filters at stride 4, pad 0

[27x27x96] **MAX POOL1**: 3x3 filters at stride 2

[27x27x96] **NORM1**: Normalization layer

[27x27x256] **CONV2**: 256 5x5 filters at stride 1, pad 2

[13x13x256] **MAX POOL2**: 3x3 filters at stride 2

[13x13x256] **NORM2**: Normalization layer

[13x13x384] **CONV3**: 384 3x3 filters at stride 1, pad 1

[13x13x384] **CONV4**: 384 3x3 filters at stride 1, pad 1

[13x13x256] **CONV5**: 256 3x3 filters at stride 1, pad 1

[6x6x256] **MAX POOL3**: 3x3 filters at stride 2

[4096] **FC6**: 4096 neurons

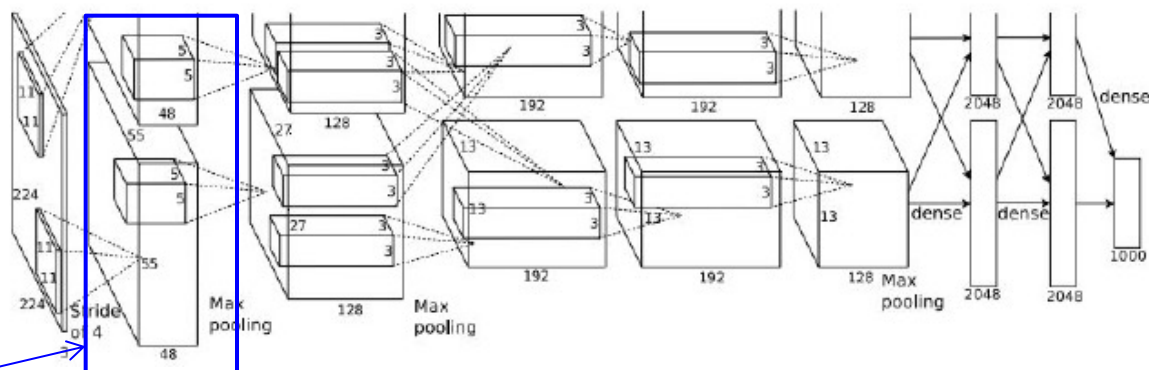
[4096] **FC7**: 4096 neurons

[1000] **FC8**: 1000 neurons (class scores)

Details/Retrospectives:

- ✓ 第一次使用ReLU
- ✓ 使用标准层（不常见）
- ✓ Data Augmentation（过拟合）
- ✓ dropout 0.5（过拟合）

AlexNet



Full (simplified) AlexNet architecture:

[227x227x3] INPUT

[55x55x96] CONV1: 96 11x11 filters at stride 4, pad 0

[27x27x96] MAX POOL1: 3x3 filters at stride 2

[27x27x96] NORM1: Normalization layer

[27x27x256] CONV2: 256 5x5 filters at stride 1, pad 2

[13x13x256] MAX POOL2: 3x3 filters at stride 2

[13x13x256] NORM2: Normalization layer

[13x13x384] CONV3: 384 3x3 filters at stride 1, pad 1

[13x13x384] CONV4: 384 3x3 filters at stride 1, pad 1

[13x13x256] CONV5: 256 3x3 filters at stride 1, pad 1

[6x6x256] MAX POOL3: 3x3 filters at stride 2

[4096] FC6: 4096 neurons

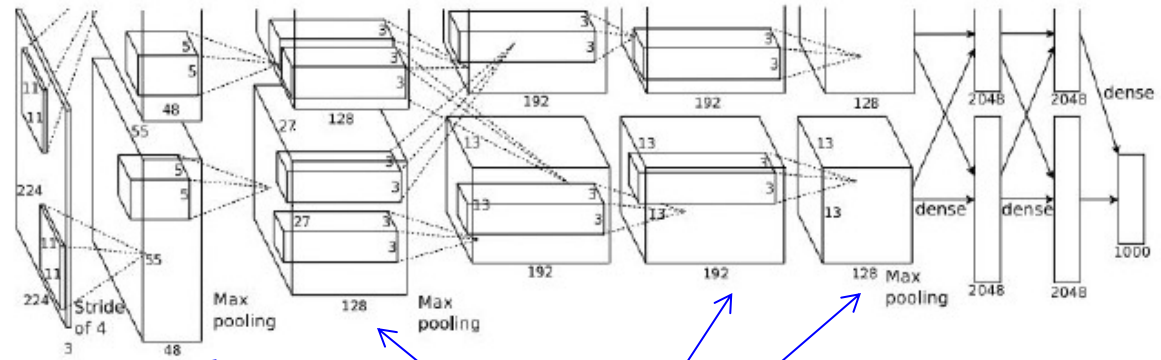
[4096] FC7: 4096 neurons

[1000] FC8: 1000 neurons (class scores)

历史记录：在GTX 580 GPU上训练，只有3GB的内存。

网络分布在两个GPU上，每个GPU上有一半的神经元（特征图）。

AlexNet



Full (simplified) AlexNet architecture:

[227x227x3] INPUT

[55x55x96] **CONV1**: 96 11x11 filters at stride 4, pad 0

[27x27x96] **MAX POOL1**: 3x3 filters at stride 2

[27x27x96] **NORM1**: Normalization layer

[27x27x256] **CONV2**: 256 5x5 filters at stride 1, pad 2

[13x13x256] **MAX POOL2**: 3x3 filters at stride 2

[13x13x256] **NORM2**: Normalization layer

[13x13x384] **CONV3**: 384 3x3 filters at stride 1, pad 1

[13x13x384] **CONV4**: 384 3x3 filters at stride 1, pad 1

[13x13x256] **CONV5**: 256 3x3 filters at stride 1, pad 1

[6x6x256] **MAX POOL3**: 3x3 filters at stride 2

[4096] **FC6**: 4096 neurons

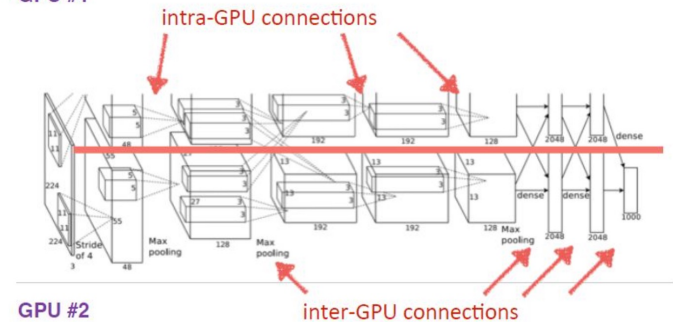
[4096] **FC7**: 4096 neurons

[1000] **FC8**: 1000 neurons (class scores)

CONV1, CONV2, CONV4, CONV5:

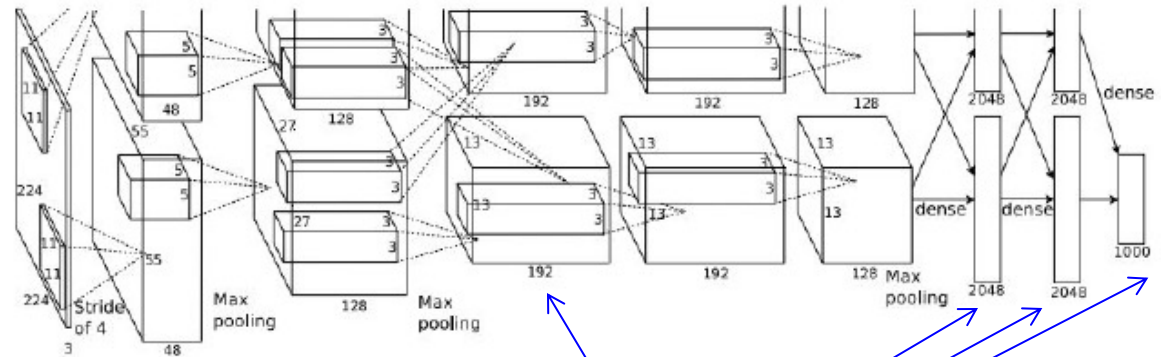
仅与特征图在同一GPU上连接

GPU #1



Top-1 and Top-5 error rates decreases by 1.7% & 1.2% respectively, comparing to the net trained with one GPU and half neurons!!

AlexNet



Full (simplified) AlexNet architecture:

[227x227x3] INPUT

[55x55x96] **CONV1**: 96 11x11 filters at stride 4, pad 0

[27x27x96] **MAX POOL1**: 3x3 filters at stride 2

[27x27x96] **NORM1**: Normalization layer

[27x27x256] **CONV2**: 256 5x5 filters at stride 1, pad 2

[13x13x256] **MAX POOL2**: 3x3 filters at stride 2

[13x13x256] **NORM2**: Normalization layer

[13x13x384] **CONV3**: 384 3x3 filters at stride 1, pad 1

[13x13x384] **CONV4**: 384 3x3 filters at stride 1, pad 1

[13x13x256] **CONV5**: 256 3x3 filters at stride 1, pad 1

[6x6x256] **MAX POOL3**: 3x3 filters at stride 2

[4096] **FC6**: 4096 neurons

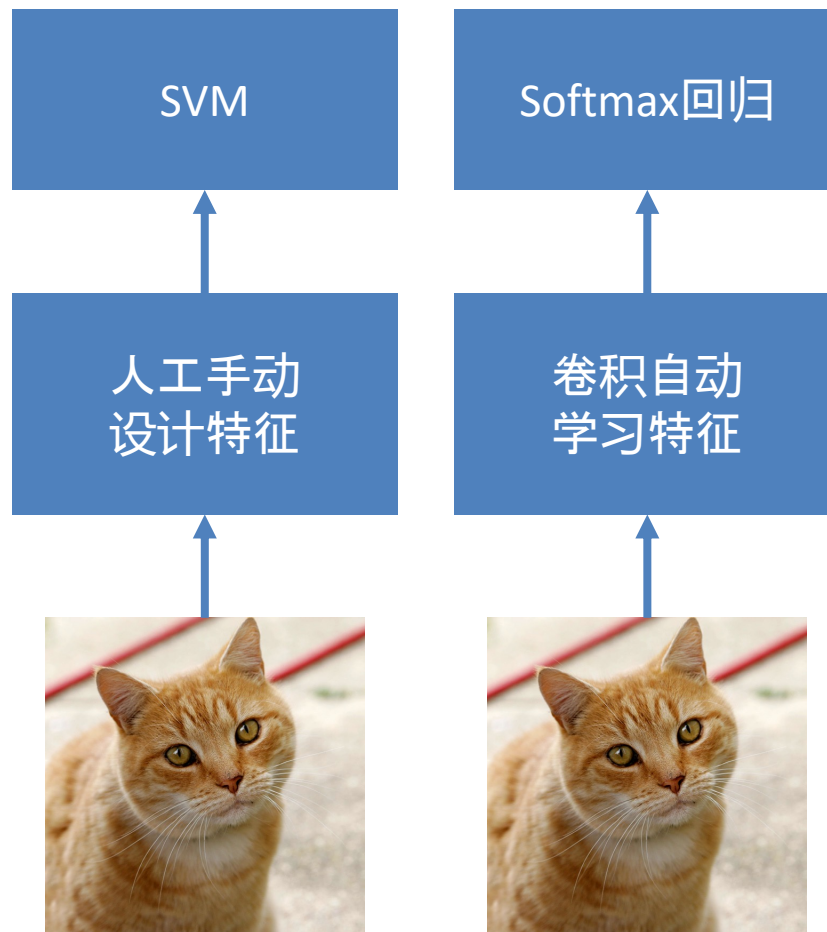
[4096] **FC7**: 4096 neurons

[1000] **FC8**: 1000 neurons (class scores)

CONV3, FC6, FC7,
通过GPU, 与上一层中进行通信的所有特征图的连接

AlexNet总结

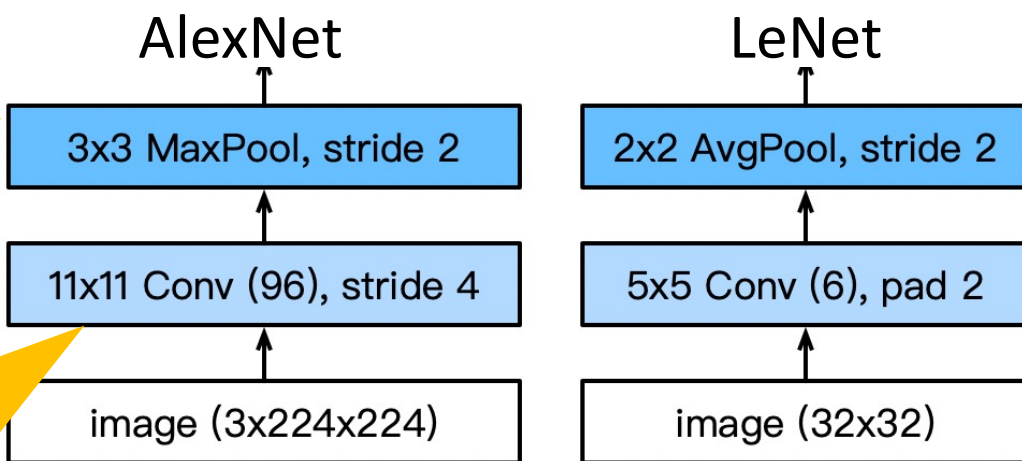
- AlexNet 在 2012 年赢得了 ImageNet 竞赛
- 更深更大的 LeNet
- 主要修改
 - ✓ 丢弃法(正则化)
 - ✓ ReLu 激活函数 (训练)
 - ✓ 最大池化法
- 计算机视觉的范式转变



AlexNet 架构

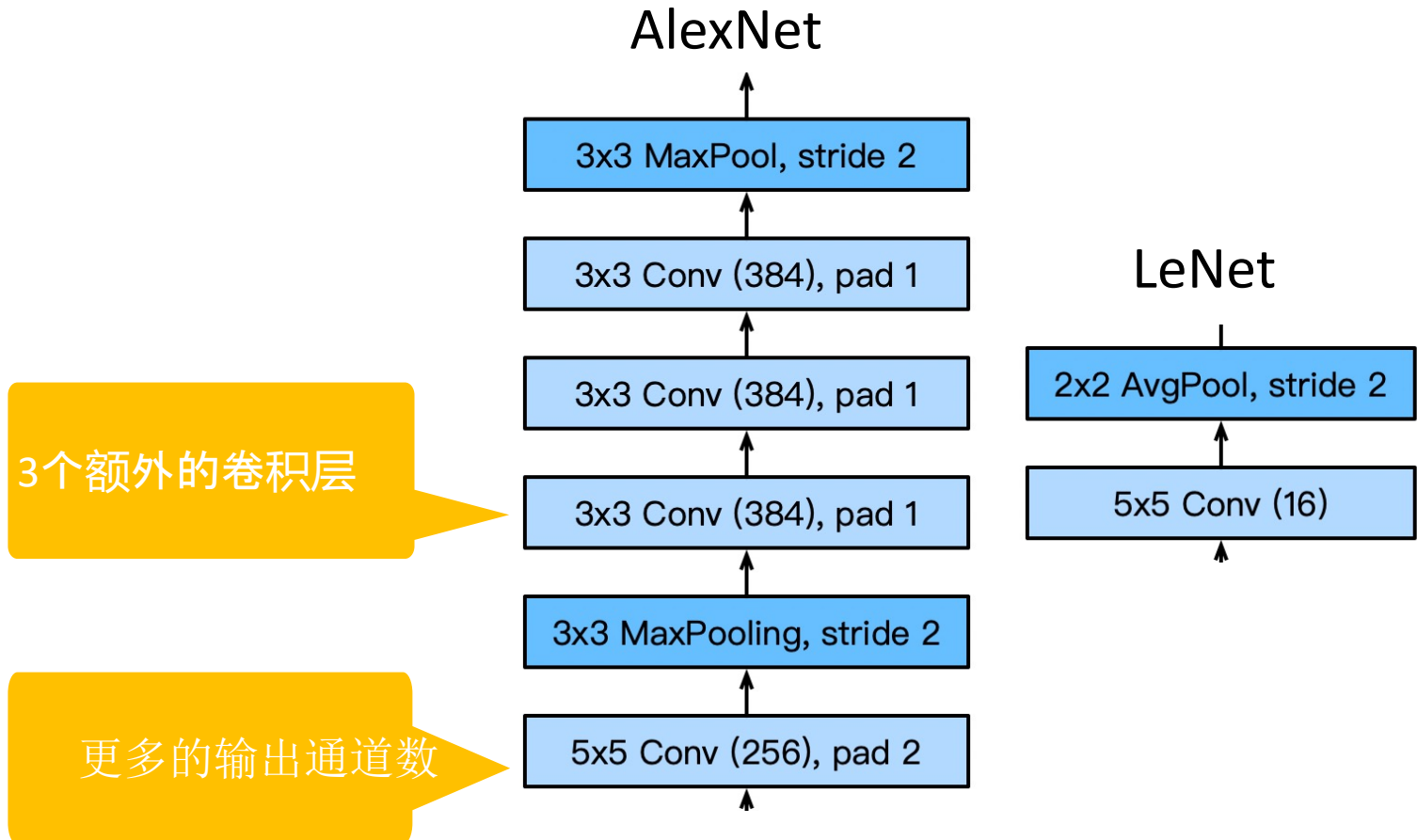
池化窗口更大, 更改为最大池化

更大的卷积核, 更大的步长, 由于图像尺寸增大, 输出通道更多。

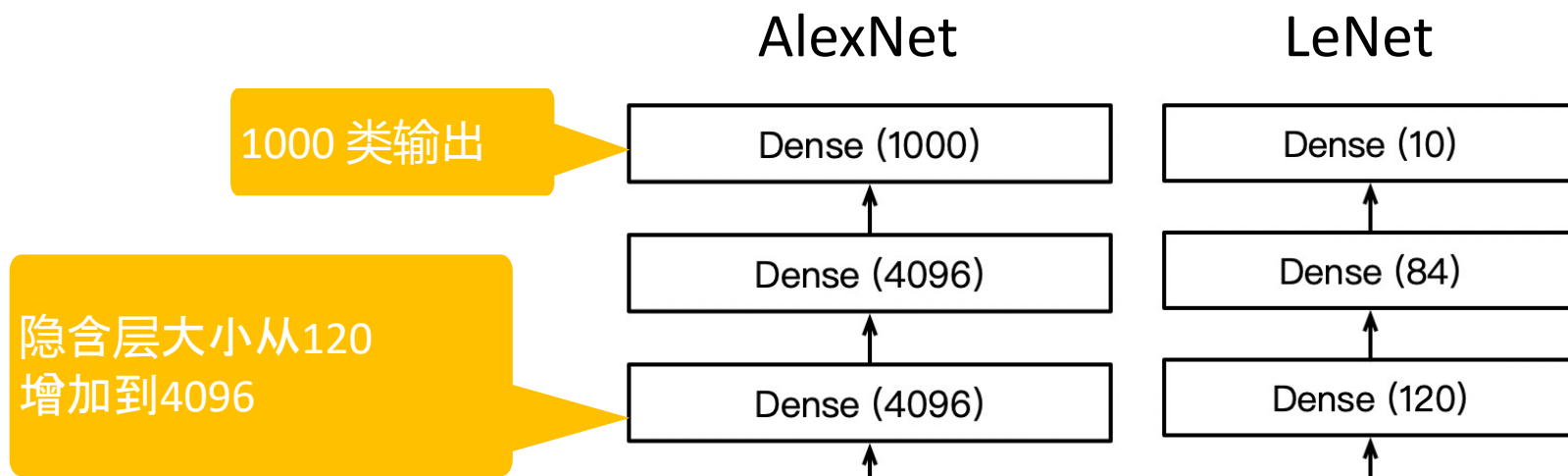


AlexNet第一层中的卷积窗口形状是11 x 11。因为ImageNet中绝大多数图像的高和宽均比MNIST图像的高和宽大10倍以上, ImageNet图像的物体占用更多的像素, 所以需要更大的卷积窗口来捕获物体。

AlexNet 架构



AlexNet 架构



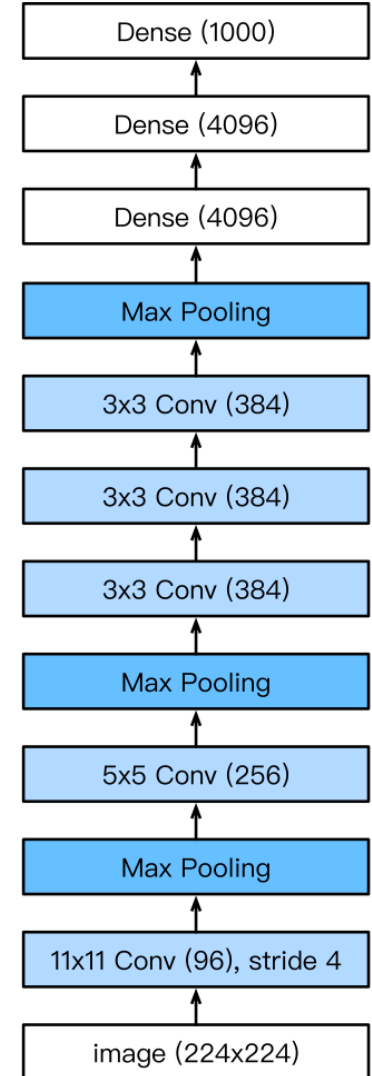
更多技巧

- ✓ 将激活函数从 sigmoid 更改为 ReLu (不再梯度消失)
- ✓ 在两个隐含层之后应用丢弃法 (更好的稳定性 / 正则化)
- ✓ 数据增广



复杂度

	# 参数量		浮点计算量 (FLOP)	
	AlexNet	LeNet	AlexNet	LeNet
卷积层1	35K	150	101M	1.2M
卷积层2	614K	2.4K	415M	2.4M
卷积层3-5	3M		445M	
稠密层1	26M	0.48M	26M	0.48M
稠密层2	16M	0.1M	16M	0.1M
总共	46M	0.6M	1G	4M
倍数增加	11x	1x	250x	1x



Pytorch 的 简化版AlexNet

```
1 import time
2 import torch
3 from torch import nn, optim
4 import torchvision
5
6 import sys
7 sys.path.append(".")
8 import d2lzh_pytorch as d2l
9 device = torch.device('cuda' if torch.cuda.is_available() else
    'cpu')
10
11 class AlexNet(nn.Module):
12     def __init__(self):
13         super(AlexNet, self).__init__()
14         self.conv = nn.Sequential(
15             nn.Conv2d(1, 96, 11, 4), # in_channels, out_channels,
16             kernel_size, stride, padding
17             nn.ReLU(),
18             nn.MaxPool2d(3, 2), # kernel_size, stride
19             # 减小卷积窗口，使用填充为2来使得输入与输出的高和宽一致，且增大输出通道数
20             nn.Conv2d(96, 256, 5, 1, 2),
21             nn.ReLU(),
22             nn.MaxPool2d(3, 2),
```

Pytorch 的 简化版AlexNet

```
22      # 连续3个卷积层，且使用更小的卷积窗口。除了最后的卷积层外，进一步  
    增大了输出通道数。
```

```
23      # 前两个卷积层后不使用池化层来减小输入的高和宽
```

```
24      nn.Conv2d(256, 384, 3, 1, 1),
```

```
25      nn.ReLU(),
```

```
26      nn.Conv2d(384, 384, 3, 1, 1),
```

```
27      nn.ReLU(),
```

```
28      nn.Conv2d(384, 256, 3, 1, 1),
```

```
29      nn.ReLU(),
```

```
30      nn.MaxPool2d(3, 2)
```

```
31  )
```

```
32      # 这里全连接层的输出个数比LeNet中的大数倍。使用丢弃层来缓解过拟合
```

```
33      self.fc = nn.Sequential(  
34          nn.Linear(256*5*5, 4096),
```

```
35          nn.ReLU(),
```

```
36          nn.Dropout(0.5),
```

```
37          nn.Linear(4096, 4096),
```

```
38          nn.ReLU(),
```

```
39          nn.Dropout(0.5),
```

```
40      # 输出层。由于这里使用Fashion-MNIST，所以用类别数为10，而非论文  
    中的1000
```

```
41          nn.Linear(4096, 10),
```

```
42      )
```

```
43  )
```

```
44      def forward(self, img):
```

```
45          feature = self.conv(img)
```

```
46          output = self.fc(feature.view(img.shape[0], -1))
```

```
47          return output
```

Pytorch 的 简化版AlexNet

```
1 net = AlexNet()  
2 print(net)
```

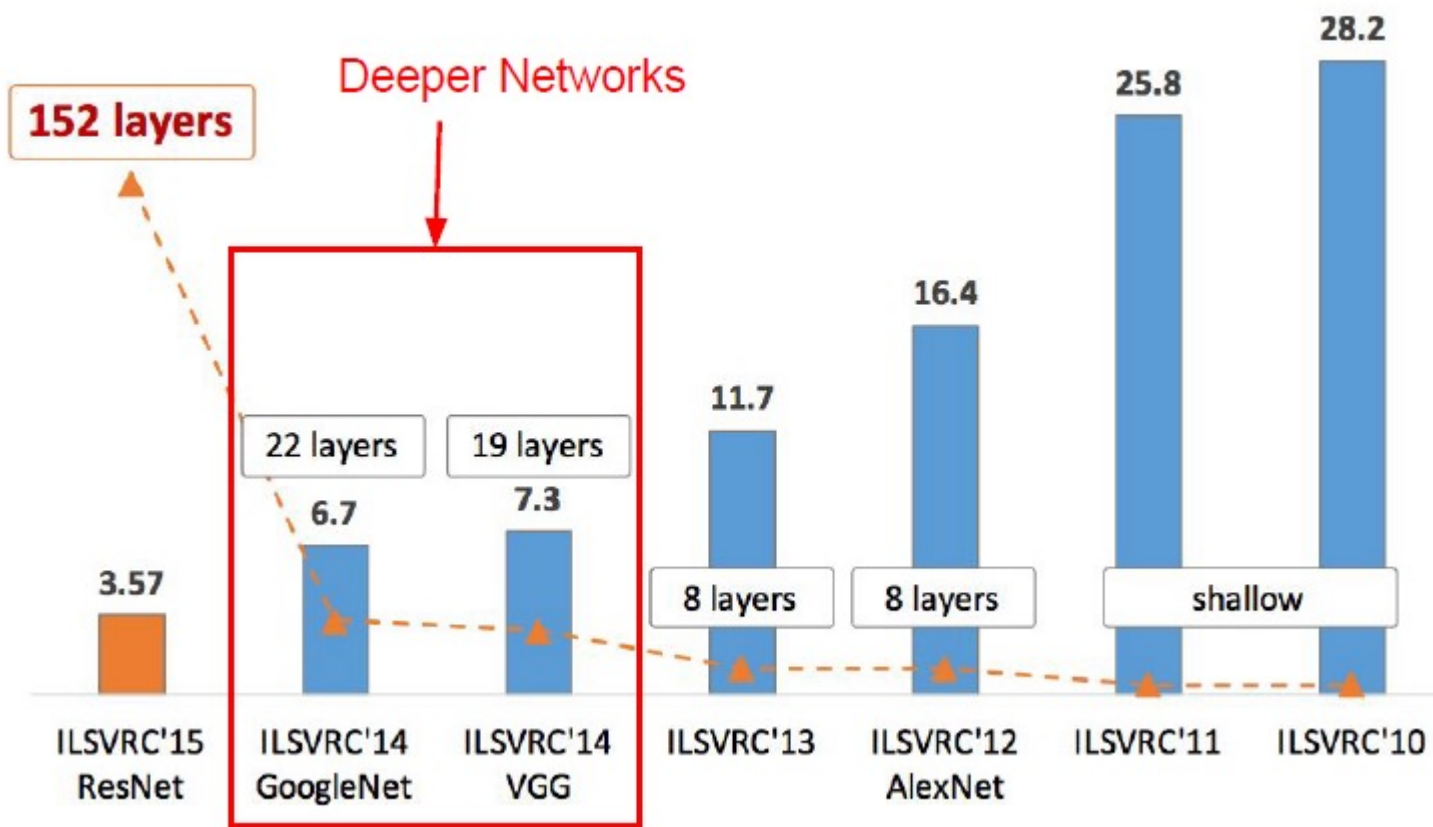
```
1 AlexNet(  
2   (conv): Sequential(  
3     (0): Conv2d(1, 96, kernel_size=(11, 11), stride=(4, 4))  
4     (1): ReLU()  
5     (2): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1,  
ceil_mode=False)  
6     (3): Conv2d(96, 256, kernel_size=(5, 5), stride=(1, 1),  
padding=(2, 2))  
7     (4): ReLU()  
8     (5): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1,  
ceil_mode=False)  
9     (6): Conv2d(256, 384, kernel_size=(3, 3), stride=(1, 1),  
padding=(1, 1))  
10    (7): ReLU()
```

```
11    (8): Conv2d(384, 384, kernel_size=(3, 3), stride=(1, 1),  
padding=(1, 1))  
12    (9): ReLU()  
13    (10): Conv2d(384, 256, kernel_size=(3, 3), stride=(1, 1),  
padding=(1, 1))  
14    (11): ReLU()  
15    (12): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1,  
ceil_mode=False)  
16  )  
17  (fc): Sequential(  
18    (0): Linear(in_features=6400, out_features=4096, bias=True)  
19    (1): ReLU()  
20    (2): Dropout(p=0.5)  
21    (3): Linear(in_features=4096, out_features=4096, bias=True)  
22    (4): ReLU()  
23    (5): Dropout(p=0.5)  
24    (6): Linear(in_features=4096, out_features=10, bias=True)  
25  )  
26 )
```

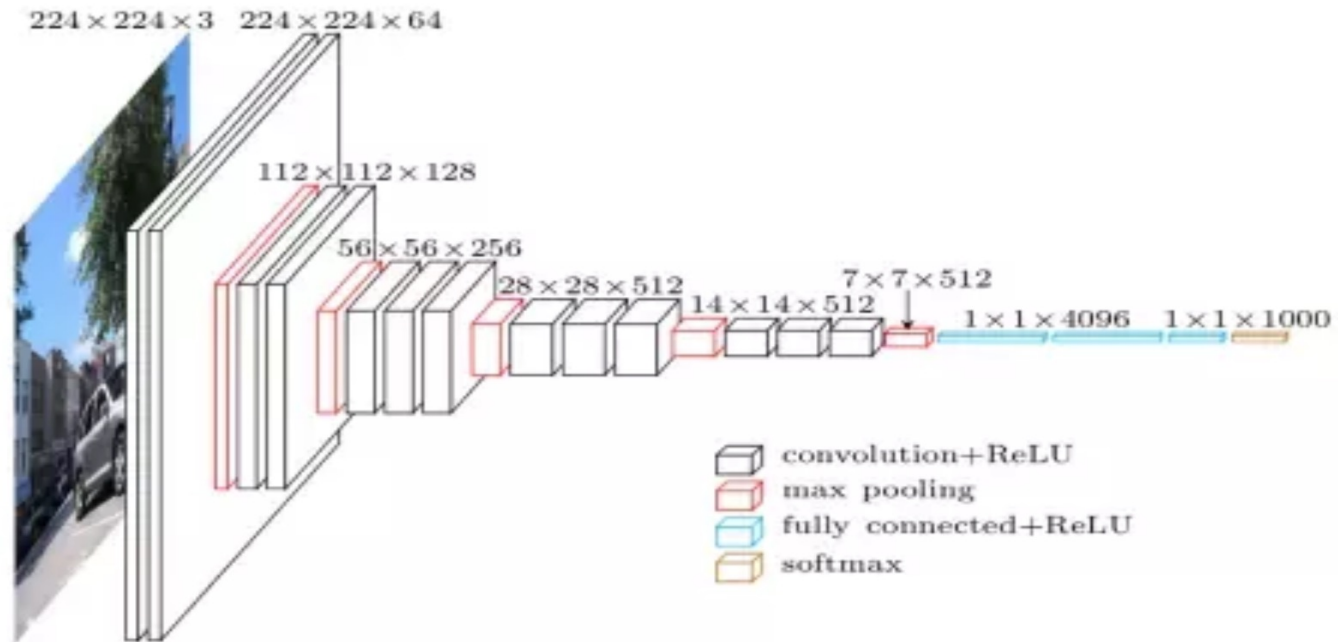
AlexNet跟LeNet结构类似，但使用了更多的卷积层和更大的参数空间来拟合大规模数据集ImageNet。它是浅层神经网络和深度神经网络的分界线。

AlexNet在LeNet的基础上增加了3个卷积层。AlexNet对它们的卷积窗口、输出通道数和构造顺序均做了大量的调整。虽然AlexNet指明了深度卷积神经网络可以取得出色的结果，但并没有提供简单的规则以指导后来的研究者如何设计新的网络。

大型视觉识别挑战赛（ILSVRC）获奖者



VGG



VGGNet由牛津大学的视觉几何组（Visual Geometry Group）和Google DeepMind公司的研究员共同提出，是ILSVRC-2014中定位任务第一名和分类任务第二名。其突出贡献在于证明使用很小的卷积（3*3），增加网络深度可以有效提升模型的效果。

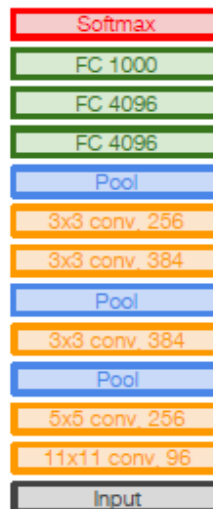
Simonyan K , Zisserman A . Very Deep Convolutional Networks for Large-Scale Image Recognition, 2014.

VGGNet

小型滤波器，更深的网络

8层 (AlexNet)

-> 16 - 19层 (VGG16Net)



AlexNet



VGG16

VGG19

VGGNet

INPUT: [224x224x3] memory: $224*224*3=150K$ params: 0 (not counting biases)
CONV3-64: [224x224x64] memory: $224*224*64=3.2M$ params: $(3*3*3)*64 = 1,728$
CONV3-64: [224x224x64] memory: $224*224*64=3.2M$ params: $(3*3*64)*64 = 36,864$
POOL2: [112x112x64] memory: $112*112*64=800K$ params: 0
CONV3-128: [112x112x128] memory: $112*112*128=1.6M$ params: $(3*3*64)*128 = 73,728$
CONV3-128: [112x112x128] memory: $112*112*128=1.6M$ params: $(3*3*128)*128 = 147,456$
POOL2: [56x56x128] memory: $56*56*128=400K$ params: 0
CONV3-256: [56x56x256] memory: $56*56*256=800K$ params: $(3*3*128)*256 = 294,912$
CONV3-256: [56x56x256] memory: $56*56*256=800K$ params: $(3*3*256)*256 = 589,824$
CONV3-256: [56x56x256] memory: $56*56*256=800K$ params: $(3*3*256)*256 = 589,824$
POOL2: [28x28x256] memory: $28*28*256=200K$ params: 0
CONV3-512: [28x28x512] memory: $28*28*512=400K$ params: $(3*3*256)*512 = 1,179,648$
CONV3-512: [28x28x512] memory: $28*28*512=400K$ params: $(3*3*512)*512 = 2,359,296$
CONV3-512: [28x28x512] memory: $28*28*512=400K$ params: $(3*3*512)*512 = 2,359,296$
POOL2: [14x14x512] memory: $14*14*512=100K$ params: 0
CONV3-512: [14x14x512] memory: $14*14*512=100K$ params: $(3*3*512)*512 = 2,359,296$
CONV3-512: [14x14x512] memory: $14*14*512=100K$ params: $(3*3*512)*512 = 2,359,296$
CONV3-512: [14x14x512] memory: $14*14*512=100K$ params: $(3*3*512)*512 = 2,359,296$
POOL2: [7x7x512] memory: $7*7*512=25K$ params: 0
FC: [1x1x4096] memory: 4096 params: $7*7*512*4096 = 102,760,448$
FC: [1x1x4096] memory: 4096 params: $4096*4096 = 16,777,216$
FC: [1x1x1000] memory: 1000 params: $4096*1000 = 4,096,000$

Note:

大多数内存都是早期的CONV

大多数参数都在FC后期

参数量主要消耗在最后3个全连接层，而前面的卷积层虽然层数多，但消耗的参数量不大。

TOTAL memory: $24M * 4 \text{ bytes} \sim 96MB / \text{image}$ (only forward! ~ 2 for bwd)

TOTAL params: 138M parameters

进程

□ LeNet (1995)

- ✓ 2 卷积层 + 池化层
- ✓ 2 隐含层

□ AlexNet

- ✓ 更大更深的 LeNet
- ✓ ReLu 激活, 丢弃法, 预处理

□ VGG

- ✓ 更大更深的 AlexNet (重复的 VGG 块)

Pytorch 的 VGG11

```
1 import time
2 import torch
3 from torch import nn, optim
4
5 import sys
6 sys.path.append(".")
7 import d2lzh_pytorch as d2l
8 device = torch.device('cuda' if torch.cuda.is_available() else
    'cpu')
9
10 def vgg_block(num_convs, in_channels, out_channels):
11     blk = []
12     for i in range(num_convs):
13         if i == 0:
14             blk.append(nn.Conv2d(in_channels, out_channels,
15 kernel_size=3, padding=1))
16         else:
17             blk.append(nn.Conv2d(out_channels, out_channels,
18 kernel_size=3, padding=1))
19         blk.append(nn.ReLU())
20     blk.append(nn.MaxPool2d(kernel_size=2, stride=2)) # 这里会使宽高减
    半
21     return nn.Sequential(*blk)
```

VGG11: 它有5个卷积块，前2块使用单卷积层，而后3块使用双卷积层。第一块的输入输出通道分别是1(数据的通道数为1)和64，之后每次对输出通道数翻倍，直到变为512。

Pytorch 的 VGG11

```
1 conv_arch = ((1, 1, 64), (1, 64, 128), (2, 128, 256), (2, 256, 512),  
2 (2, 512, 512))  
3 # 经过5个vgg_block, 宽高会减半5次, 变成 224/32 = 7  
4 fc_features = 512 * 7 * 7 # c * w * h  
5 fc_hidden_units = 4096 # 任意
```

```
1 def vgg(conv_arch, fc_features, fc_hidden_units=4096):  
2     net = nn.Sequential()  
3     # 卷积层部分  
4     for i, (num_convs, in_channels, out_channels) in  
5         enumerate(conv_arch):  
6             # 每经过一个vgg_block都会使宽高减半  
7             net.add_module("vgg_block_" + str(i+1),  
8                             vgg_block(num_convs, in_channels, out_channels))  
9     # 全连接层部分  
10    net.add_module("fc", nn.Sequential(d2l.FlattenLayer(),  
11                                       nn.Linear(fc_features,  
12                                               fc_hidden_units),  
13                                       nn.ReLU(),  
14                                       nn.Dropout(0.5),
```

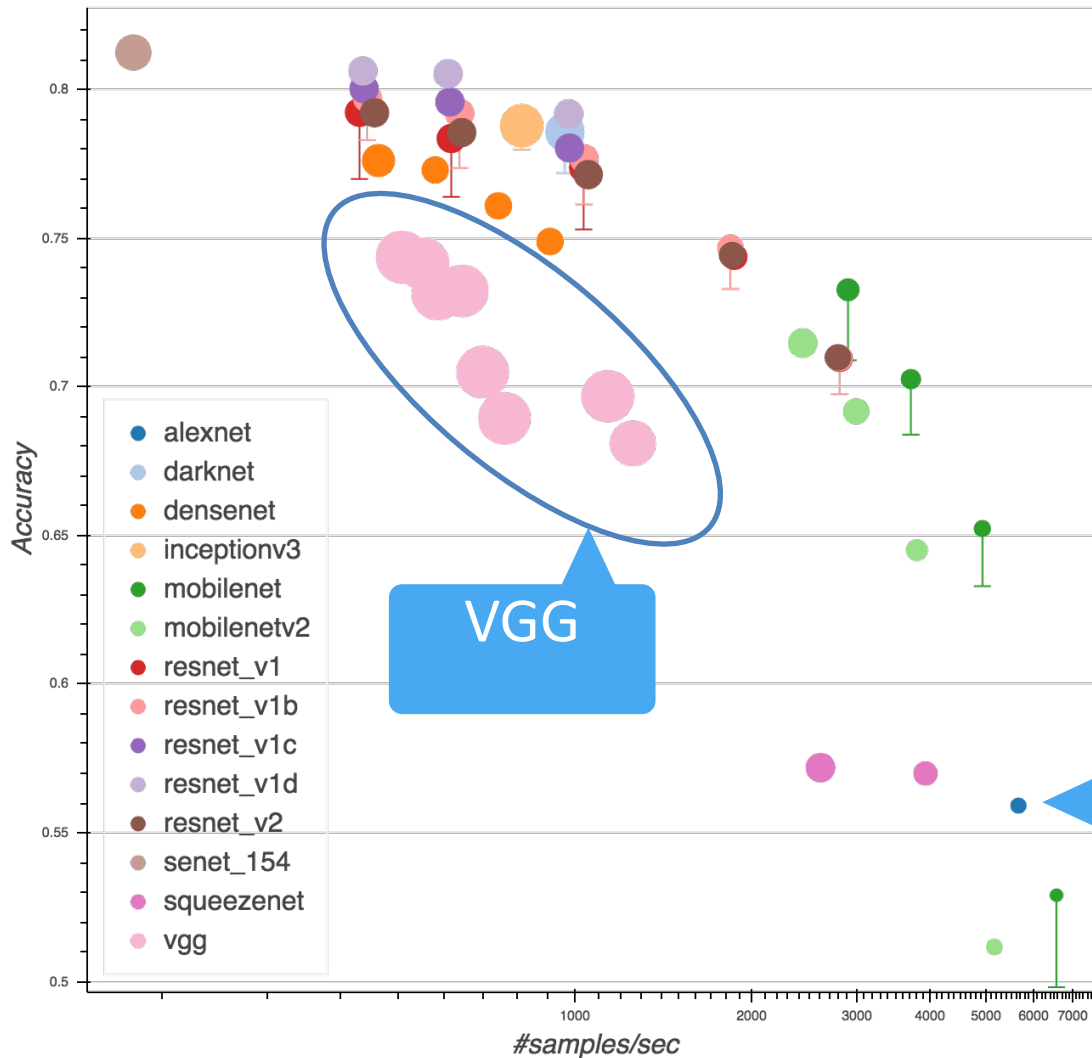
```
15                                       nn.Linear(fc_hidden_units,  
16                                               fc_hidden_units),  
17                                       nn.ReLU(),  
18                                       nn.Dropout(0.5),  
19                                       nn.Linear(fc_hidden_units, 10)  
20                                       ))  
21    return net
```

VGG11: 它有5个卷积块, 前2块使用单卷积层, 而后3块使用双卷积层。第一块的输入输出通道分别是1(数据的通道数为1)和64, 之后每次对输出通道数翻倍, 直到变为512。

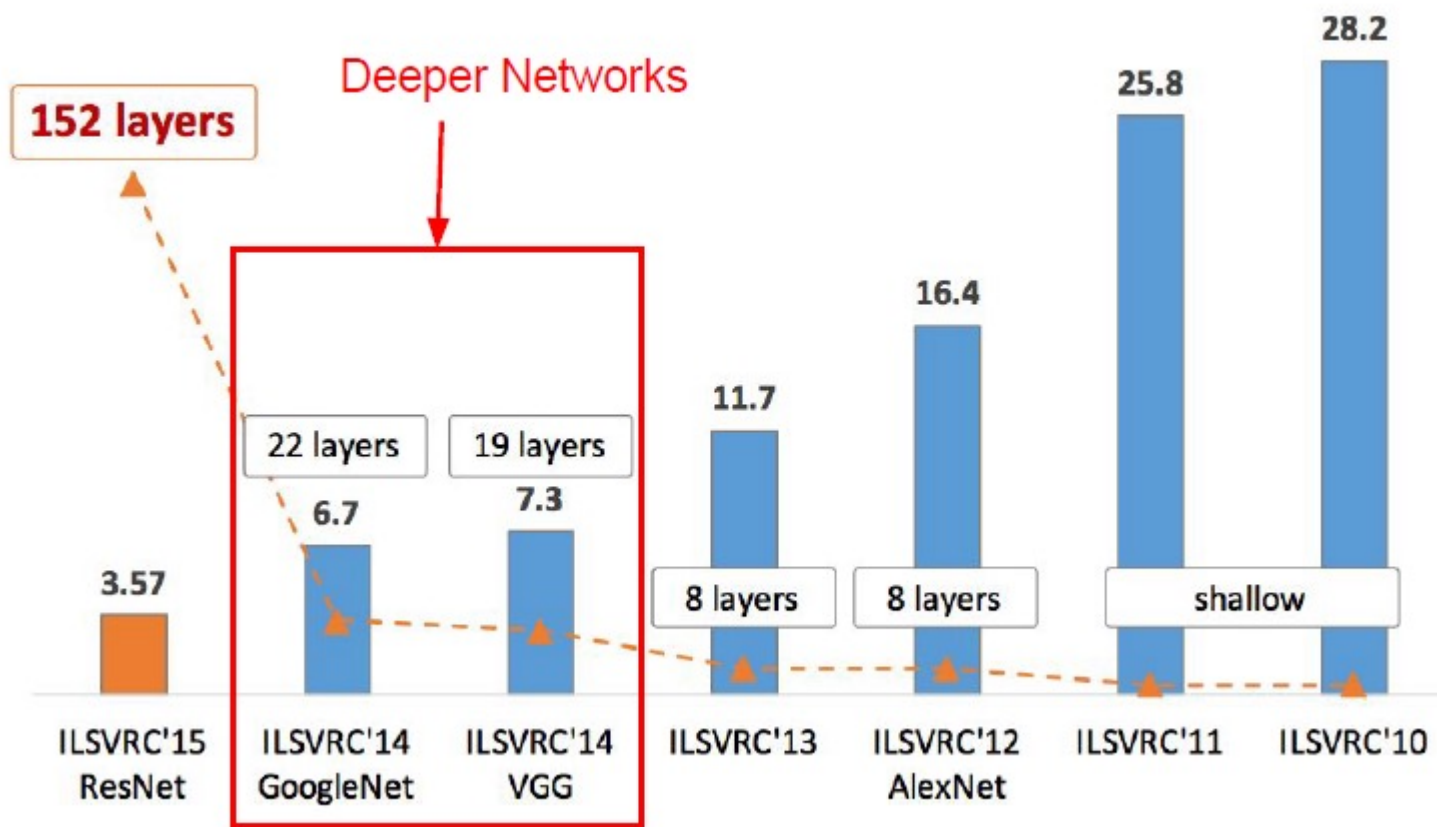
```
1 net = vgg(conv_arch, fc_features, fc_hidden_units)  
2 X = torch.rand(1, 1, 224, 224)  
3  
4 # named_children获取一级子模块及其名字(named_modules会返回所有子模块, 包括子  
5   模块的子模块)  
6 for name, blk in net.named_children():  
7     X = blk(X)  
8     print(name, 'output shape: ', X.shape)
```

GluonCV Model Zoo

gluon-cv.mxnet.io/model_zoo/classification.html



大型视觉识别挑战赛（ILSVRC）获奖者



概要

- **Inception**

- 卷积的不均匀混合 (不同深度)
- 批量归一正则化

- **ResNet**

- 泰勒展开式
- 残差网络 (**ResNet**) 分解卷积

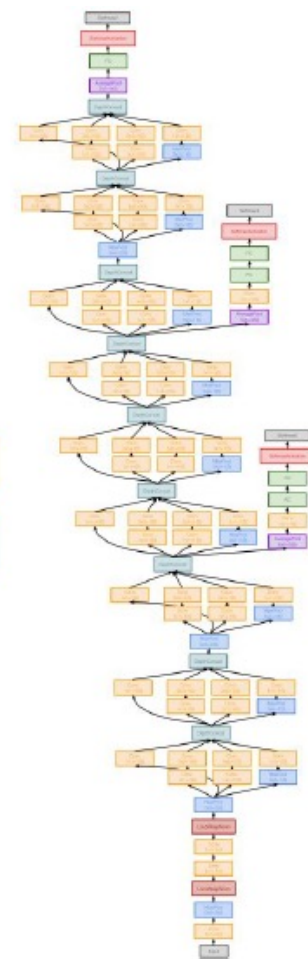
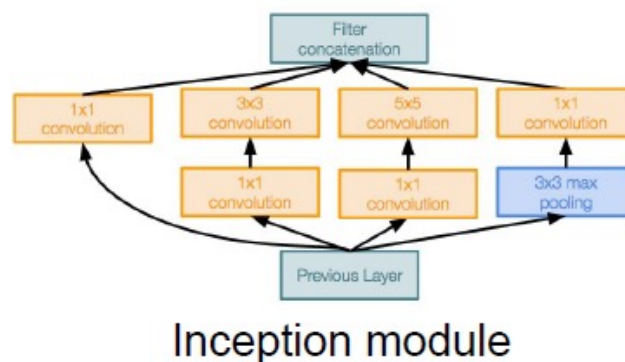
- **Zoo**

稠密连接网络 (DenseNet), ShuffleNet, 可分解卷积层, ...

GoogLeNet

更深的网络，具有更高计算效率

- 22层
- 高效的“Inception”模块
- 没有FC层
- 只有500万个参数！比AlexNet少12倍
- ILSVRC'14分类冠军
(6.7%前5位错误)



GoogLeNet

- ✓ 以前的卷积神经网络基本都是由多个卷积层与池化层堆积而成，然后接入一个或者多个全连接层来预测输出。
- ✓ 网络模型参数主要集中在全连接层。
- ✓ 提高深度网络模型性能的常用方法就是提高网络模型大小，网络模型大小包括网络深度与网络宽度。
- ✓ 网络模型参数增加，加大模型过拟合的风险。
- ✓ 增加计算开销。

方法：将全连接甚至一般的卷积都转化为稀疏连接。

理论来源：

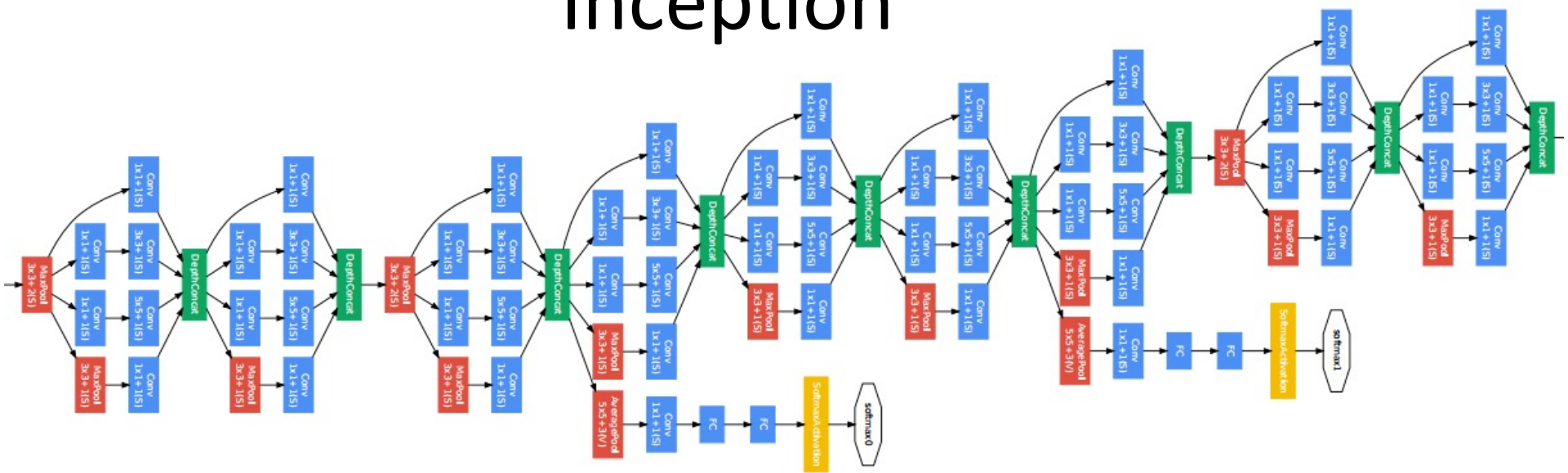
- 1)现实生物神经系统的连接也是稀疏的；
- 2)文献[1]表明：对于大规模稀疏的神经网络，可以通过分析激活值的统计特性和对高度相关的输出进行聚类来逐层构建出一个最优网络

GoogLeNet

- ✓ 原来的网络AlexNet、VGG都是通过增大层数来取得更好的效果，但带来副作用：过拟合、梯度消失、梯度爆炸。
- ✓ GoogLeNet引入Inception模块。
- ✓ Inception模块的两个优点：
 - 使用 1×1 卷积来升降维，
 - 多个尺寸上同时进行卷积再聚合：不同大小的卷积核可以获取到不同大小的感受野

GoogLeNet的成功主要得益于Inception模块，整个GoogLeNet的主体架构可以看成多个Inception模块堆叠而成。

Inception



选最合适的卷积 ...

1x1

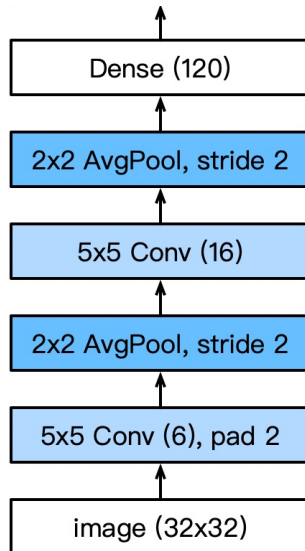
3x3

5x5

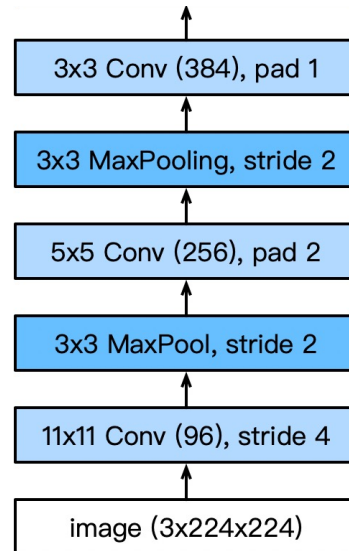
最大池化

许多 1x1

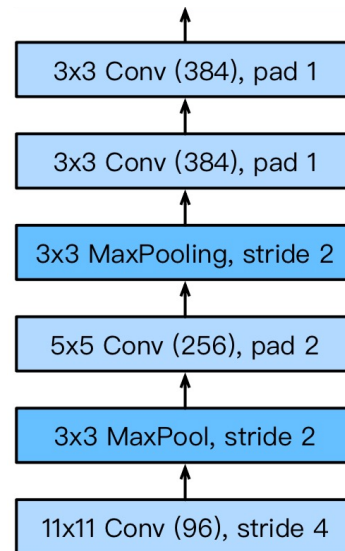
LeNet



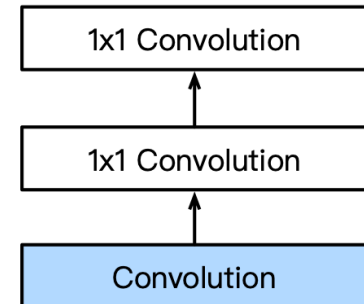
AlexNet



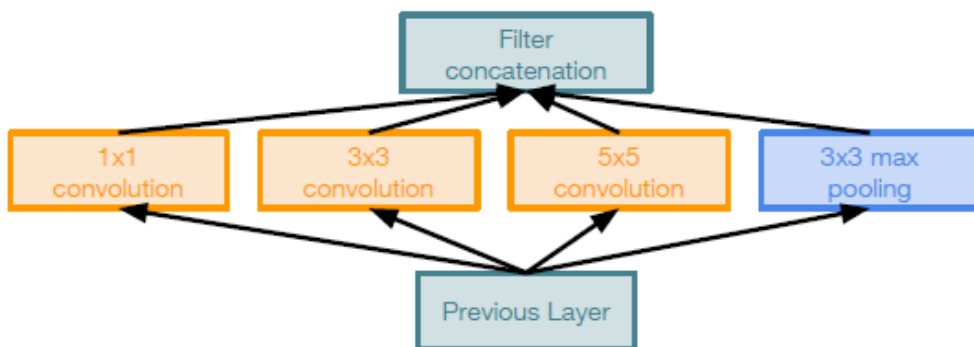
VGG



NiN



GoogLeNet



Naive Inception module

近似最优的局部稀疏结构

对上一层的输入应用并行滤波操作：

- 卷积的多个感受野大小

($1 \times 1, 3 \times 3, 5 \times 5$)

- 池化操作 (3×3)

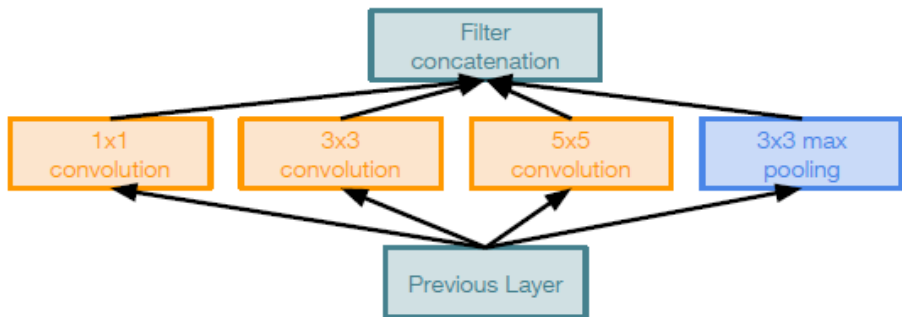
将所有过滤器输出按通道连接在一起

Q: 这有什么问题?

[提示: 计算复杂性]

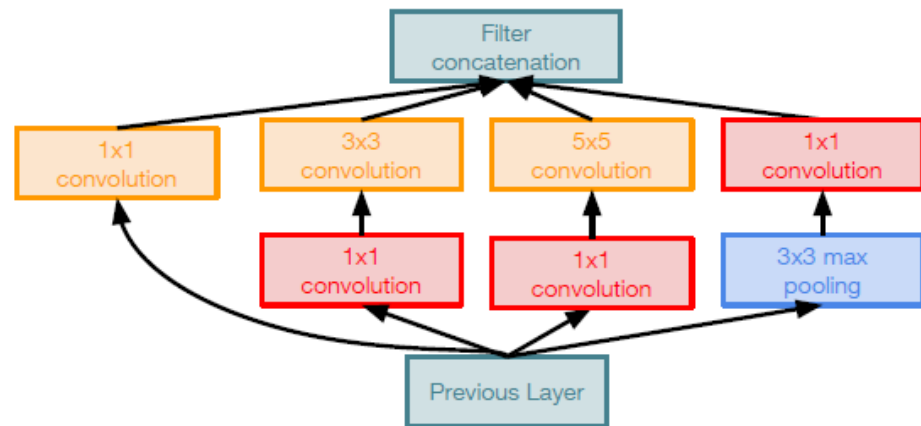
- ✓ 不同大小的卷积核意味着不同大小的感受野，最后拼接意味着不同尺度特征的融合；
- ✓ 卷积核大小采用1、3和5，方便对齐。stride=1，分别设定pad=0、1、2，卷积之后特征的维度的相同，可以拼接；
- ✓ Pool有效。

GoogLeNet



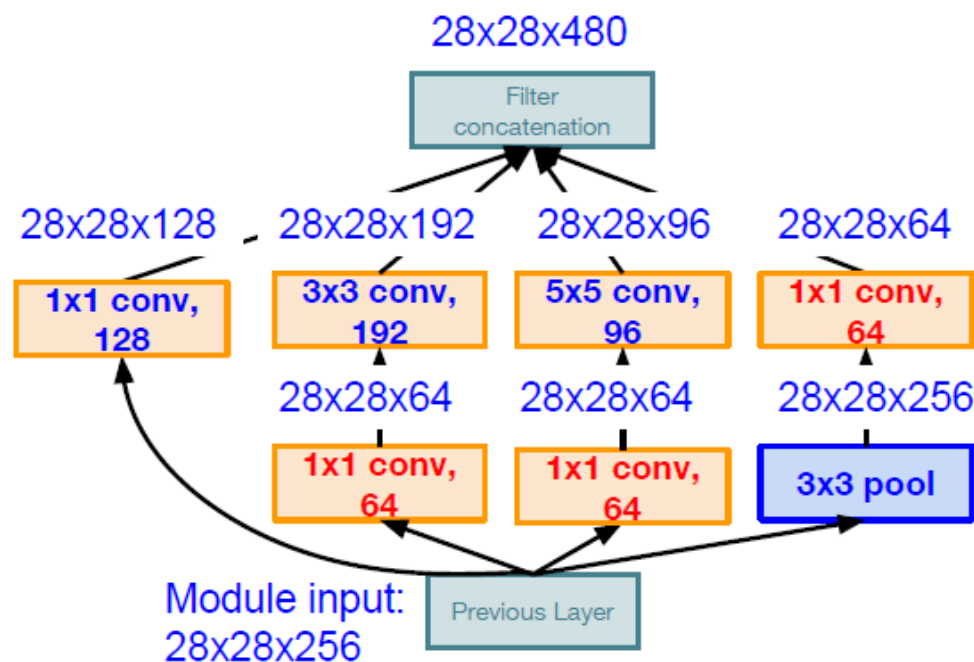
Naive Inception module

1x1 conv “瓶颈”层



Inception module with dimension reduction

案例分析: GoogLeNet



Conv动作:

[1x1 conv, 64] $28 \times 28 \times 64 \times 1 \times 1 \times 256$
[1x1 conv, 64] $28 \times 28 \times 64 \times 1 \times 1 \times 256$
[1x1 conv, 128] $28 \times 28 \times 128 \times 1 \times 1 \times 256$
[3x3 conv, 192] $28 \times 28 \times 192 \times 3 \times 3 \times 64$
[5x5 conv, 96] $28 \times 28 \times 96 \times 5 \times 5 \times 64$
[1x1 conv, 64] $28 \times 28 \times 64 \times 1 \times 1 \times 256$

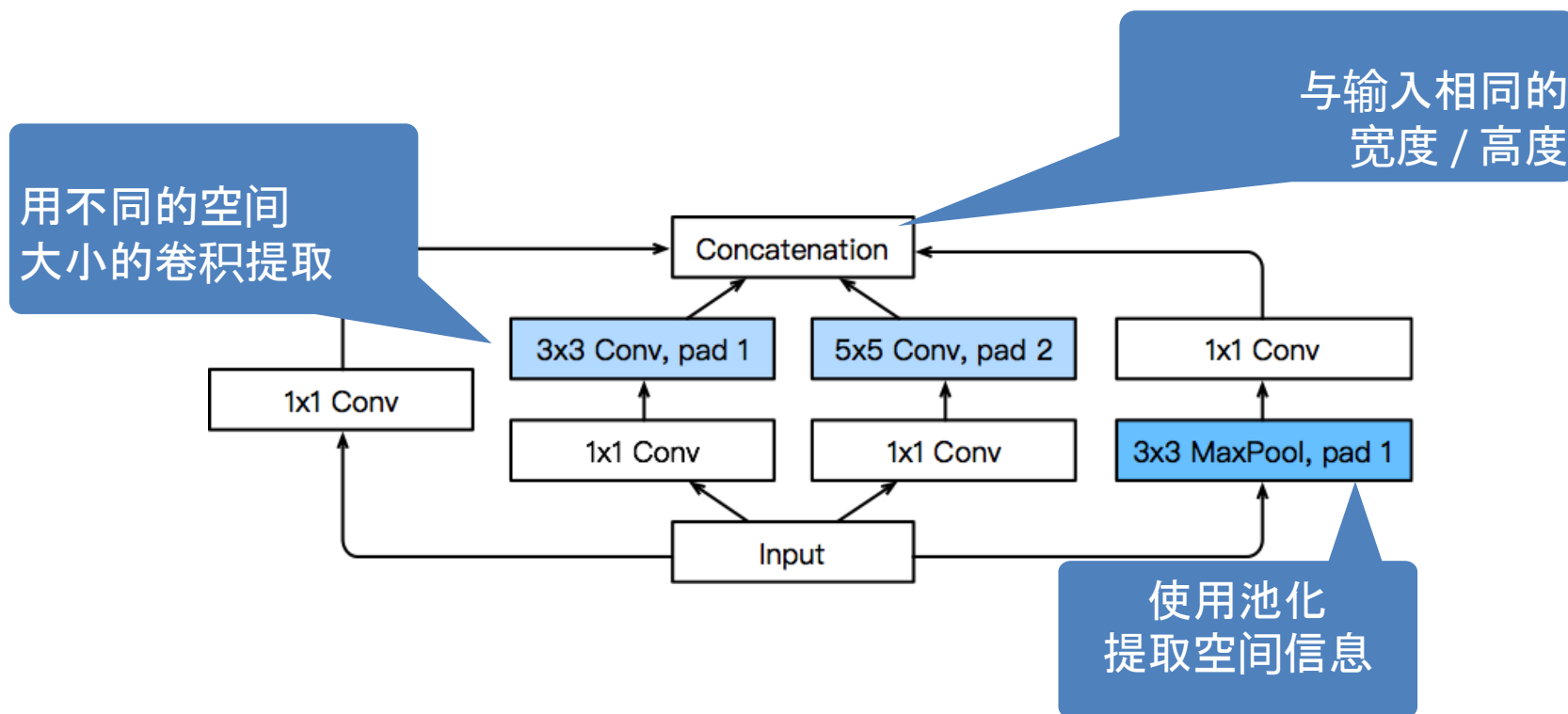
总计: 358M操作

与854M操作相比, 大大减少

Inception module with dimension reduction

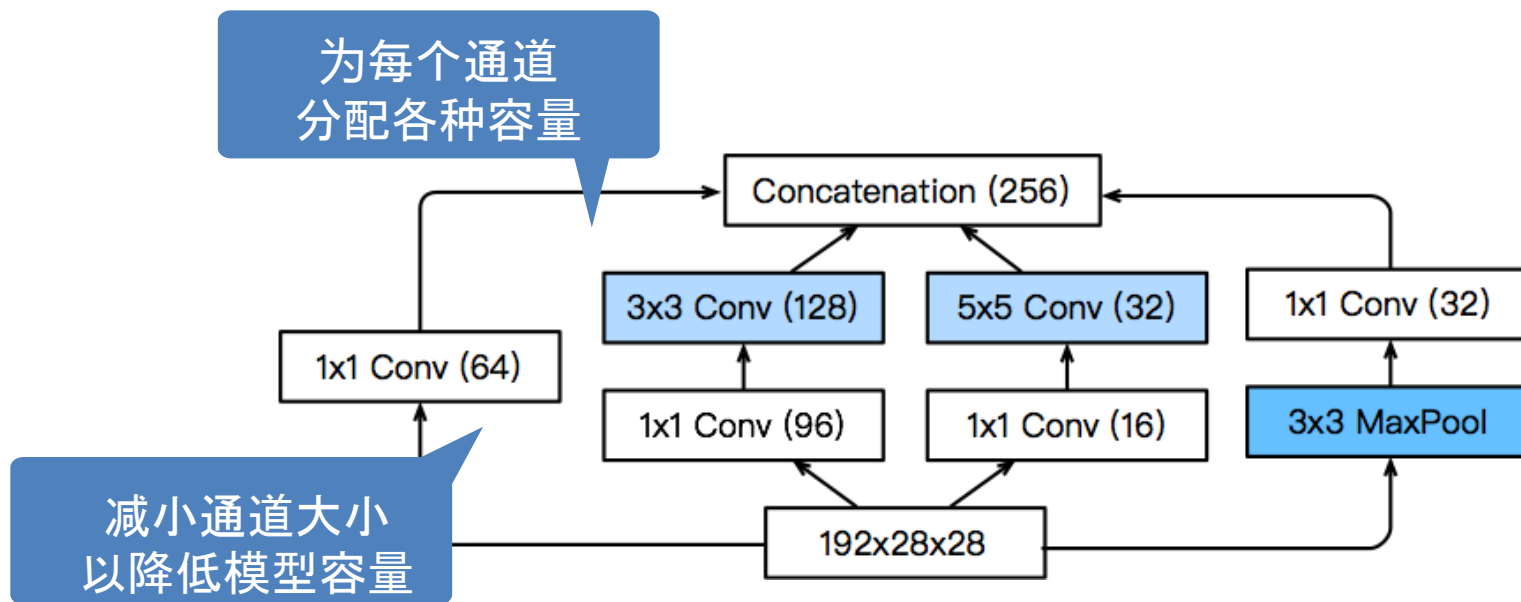
Inception 块

4个路径从不同方面提取信息，然后拼接作为输出通道



Inception 块

(第一个初始块) 指定的通道大小

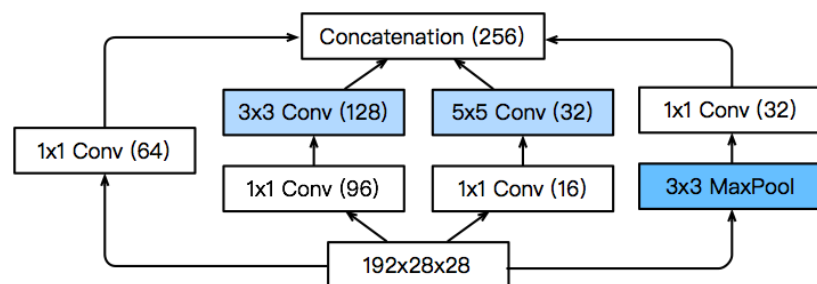


Inception 块

与单个3x3或5x5卷积层相比，初始块具有更少的参数和更低的计算复杂度

- ✓不同功能混合(多样的功能类)
- ✓卷积核计算高效(良好的泛化)

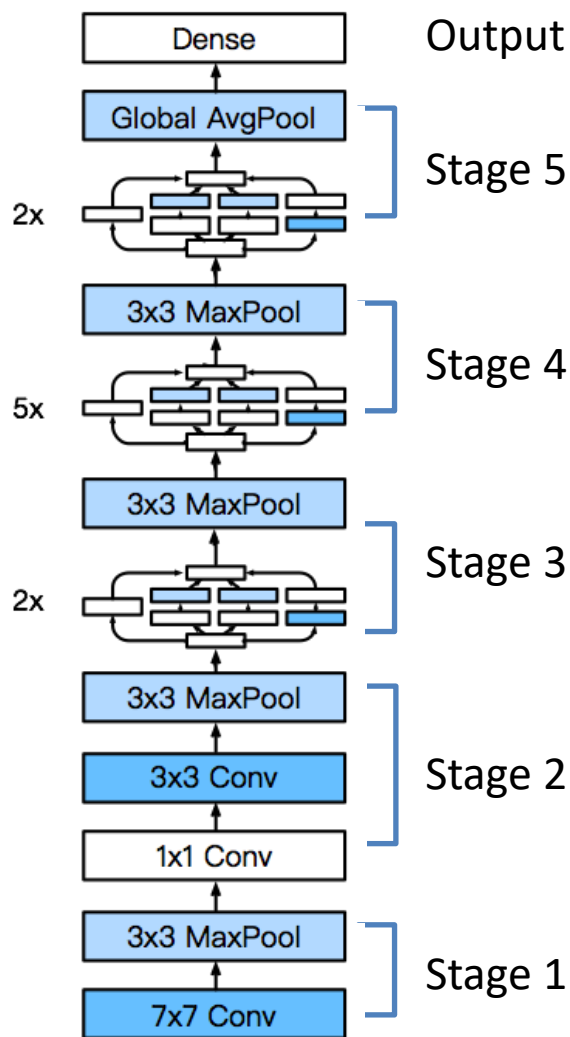
	# 参数	浮点运算 FLOPS
Inception	0.16 M	128 M
3x3 卷积	0.44 M	346 M
5x5 卷积	1.22 M	963 M



GoogLeNet

✓ 5 个阶段

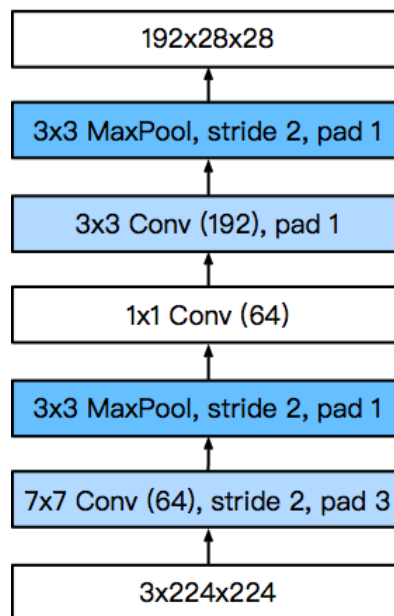
✓ 9 个 Inception 块



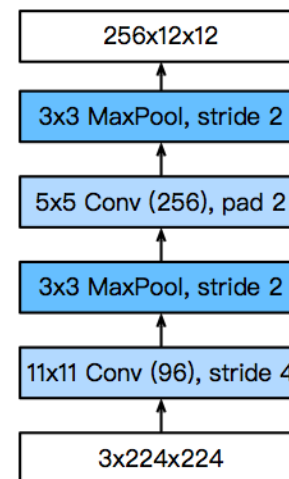
阶段 1 & 2

- 由于更多层:
 - 更小的内核
 - 更小的输出通道

GoogLeNet



AlexNet



许多种类的 Inception 网络

- ✓ Inception-BN (v2) - 添加批量归一化
- ✓ Inception-V3 - 修改了初始块
 - 用多个 3×3 卷积替换 5×5
 - 用 1×7 和 7×1 卷积替换 5×5
 - 用 1×3 和 3×1 卷积替换 3×3
 - 通常用更深的堆
- ✓ Inception-V4 - 添加残差块连接

批量归一化



批量归一化

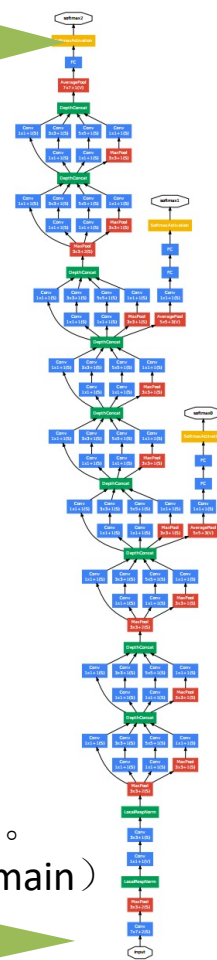
- 损失发生在最后一层
 - 最后一层可快速学习
- 数据插入在第一层（底层）
 - 底层变化 - 一切都变化
 - 最后一层需要多次重新学习
 - 收敛缓慢

我们可以避免在学习第一层时

- ✓ 独立同分布的数据可以简化常规机器学习模型的训练、提升机器学习模型的预测能力。
- ✓ 统计机器学习中的一个经典假设是“源空间（source domain）和目标空间（target domain）的数据分布（distribution）是一致的。
- ✓ 如果不一致，出现新问题：如 transfer learning / domain adaptation 等。而 covariate shift 就是分布不一致假设之下的一个分支问题，它是指源空间和目标空间的条件概率是一致的，但是其边缘概率不同。
- ✓ 对于神经网络的各层输出，由于它们经过了层内操作作用，其分布显然与各层对应的输入信号分布不同，而且差异会随着网络深度增大而增大，可是它们所能“指示”的样本标记（label）仍然是不变的，符合 covariate shift 的定义。

损失

数据



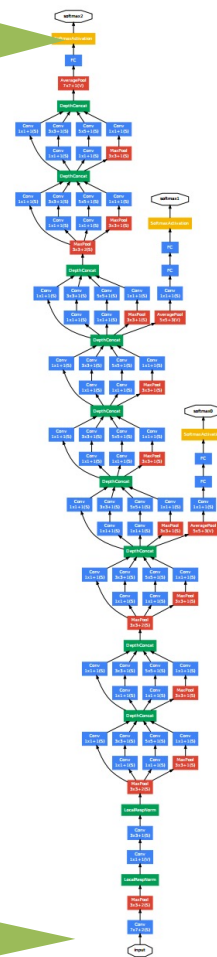
批量归一化

Internal Covariate Shift（内部协变量偏移）：当底层网络中参数发生微弱变化时，由于每一层中的线性变换与非线性激活映射，这些微弱变化随着网络层数的加深而被放大；另一方面，参数的变化导致每一层的输入分布会发生改变，进而上层的网络需要不停地去适应这些分布变化，使得我们的模型训练变得困难。

协变量偏移（**covariance shift**），就是指训练/历史数据和测试/待评估数据的特征分布不一致的情况。此时，用训练数据学出来的策略就不一定是测试数据上最优的了。

损失

数据



批量归一化

Internal Covariate Shift会带来什么问题：每个神经元的输入数据不再是“独立同分布”。

（1）上层网络需要不停调整来适应输入数据分布的变化，导致网络学习速度的降低；

（2）网络的训练过程容易陷入梯度饱和区，减缓网络收敛速度。

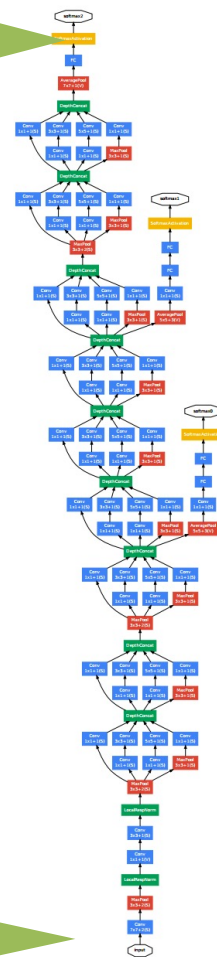
解决激活函数梯度饱和问题的方法：

（1）非饱和性激活函数，ReLU。

（2）让激活函数的输入分布保持在一个稳定状态来尽可能避免它们陷入梯度饱和区（Normalization）。

损失

数据



批量归一化

如何减缓Internal Covariate Shift

ICS产生的原因是由于参数更新带来的网络中每一层输入值分布的改变，并且随着网络层数的加深而变得更加严重，因此可以通过固定每一层网络输入值的分布来对减缓ICS问题。

（1）白化（Whitening）

白化是机器学习里面常用的一种规范化数据分布的方法，（PCA白化与ZCA白化）。白化是对输入数据分布进行变换，进而达到以下两个目的：

- ✓ 使得输入特征分布具有相同的均值与方差。其中PCA白化保证了所有特征分布均值为0，方差为1；而ZCA白化则保证了所有特征分布均值为0，方差相同；
- ✓ 去除特征之间的相关性。

白化主要有以下两个问题：

- ✓ 白化过程计算成本太高，并且在每一轮训练中的每一层都需要做如此高成本计算的白化操作；
- ✓ 白化过程由于改变了网络每一层的分布，因而改变了网络层中本身数据的表达能力。底层网络学习到的参数信息会被白化操作丢失掉。

批量归一化

白化计算过程比较复杂，可采取如下方法：

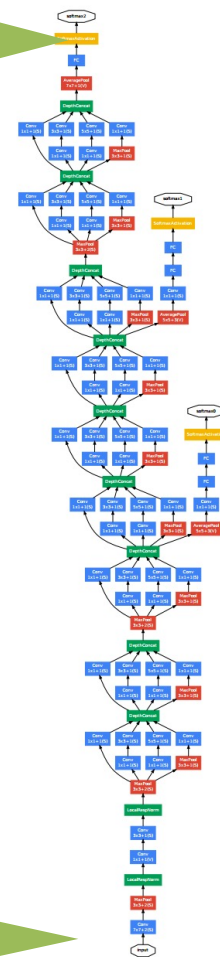
- ✓ 可以尝试单独对每个特征进行normalization，让每个特征都有均值为0，方差为1的分布。
- ✓ 白化操作减弱了网络中每一层输入数据表达能力，可通过增加线性变换操作，让这些数据再能够尽可能恢复本身的表达能力。

Normalization 的通用公式：

$$h = f \left(\mathbf{g} \cdot \frac{\mathbf{x} - \mu}{\sigma} + \mathbf{b} \right)$$

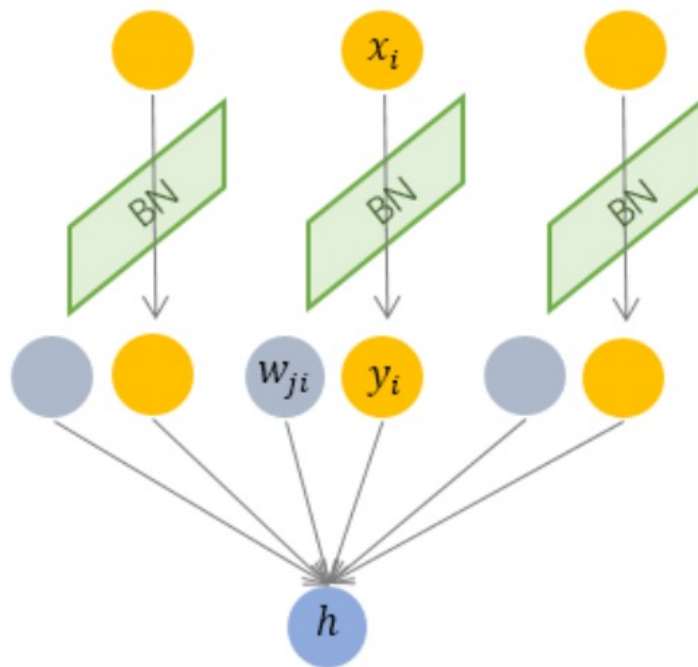
损失

数据



Batch Normalization : 纵向规范化

Batch Normalization 于2015年提出，其规范化针对单个神经元进行，利用网络训练时一个 mini-batch 的数据来计算该神经元的均值和方差,因而称为 Batch Normalization。



PyTorch 版本GoogLeNet

```
11 class Inception(nn.Module):
12     # c1 - c4为每条线路里的层的输出通道数
13     def __init__(self, in_c, c1, c2, c3, c4):
14         super(Inception, self).__init__()
15         # 线路1, 单1 x 1卷积层
16         self.p1_1 = nn.Conv2d(in_c, c1, kernel_size=1)
17         # 线路2, 1 x 1卷积层后接3 x 3卷积层
18         self.p2_1 = nn.Conv2d(in_c, c2[0], kernel_size=1)
19         self.p2_2 = nn.Conv2d(c2[0], c2[1], kernel_size=3,
padding=1)
20         # 线路3, 1 x 1卷积层后接5 x 5卷积层
21         self.p3_1 = nn.Conv2d(in_c, c3[0], kernel_size=1)
22         self.p3_2 = nn.Conv2d(c3[0], c3[1], kernel_size=5,
padding=2)
23         # 线路4, 3 x 3最大池化层后接1 x 1卷积层
24         self.p4_1 = nn.MaxPool2d(kernel_size=3, stride=1,
padding=1)
25         self.p4_2 = nn.Conv2d(in_c, c4, kernel_size=1)
26
27     def forward(self, x):
28         p1 = F.relu(self.p1_1(x))
29         p2 = F.relu(self.p2_2(F.relu(self.p2_1(x))))
30         p3 = F.relu(self.p3_2(F.relu(self.p3_1(x))))
31         p4 = F.relu(self.p4_2(self.p4_1(x)))
32         return torch.cat((p1, p2, p3, p4), dim=1) # 在通道维上连结输
```

出

PyTorch 版本GoogLeNet

GoogLeNet跟VGG一样，在主体卷积部分中使用5个模块（block），每个模块之间使用步幅为2的 3×3 最大池化层来减小输出高宽。第一模块使用一个64通道的 7×7 卷积层。

```
1 b1 = nn.Sequential(nn.Conv2d(1, 64, kernel_size=7, stride=2,
2 padding=3),
3                       nn.ReLU(),
4                       nn.MaxPool2d(kernel_size=3, stride=2, padding=1))
```

第二模块使用2个卷积层：首先是64通道的 1×1 卷积层，然后将通道增大3倍的 3×3 卷积层。它对应Inception块中的第二条线路。

```
1 b2 = nn.Sequential(nn.Conv2d(64, 64, kernel_size=1),
2                     nn.Conv2d(64, 192, kernel_size=3, padding=1),
3                     nn.MaxPool2d(kernel_size=3, stride=2, padding=1))
```

第三模块串联2个完整的Inception块。第一个Inception块的输出通道数为 $64 + 128 + 32 + 32 = 256$ ，其中4条线路的输出通道数比例为 $64 : 128 : 32 : 32 = 2 : 4 : 1 : 1$ 。其中第二、第三条线路先分别将输入通道数减小至 $96/192 = 1/2$ 和 $16/192 = 1/12$ 后，再接上第二层卷积层。第二个Inception块输出通道数增至 $128 + 192 + 96 + 64 = 480$ ，每条线路的输出通道数之比为 $128 : 192 : 96 : 64 = 4 : 6 : 3 : 2$ 。其中第二、第三条线路先分别将输入通道数减小至 $128/256 = 1/2$ 和 $32/256 = 1/8$ 。

```
1 b3 = nn.Sequential(Inception(192, 64, (96, 128), (16, 32), 32),
2                     Inception(256, 128, (128, 192), (32, 96), 64),
3                     nn.MaxPool2d(kernel_size=3, stride=2, padding=1))
```

PyTorch 版本GoogLeNet

第四模块更加复杂。它串联了5个Inception块，其输出通道数分别是 $192 + 208 + 48 + 64 = 512$ 、 $160 + 224 + 64 + 64 = 512$ 、 $128 + 256 + 64 + 64 = 512$ 、 $112 + 288 + 64 + 64 = 528$ 和 $256 + 320 + 128 + 128 = 832$ 。这些线路的通道数分配和第三模块中的类似，首先含 3×3 卷积层的第二条线路输出最多通道，其次是仅含 1×1 卷积层的第一条线路，之后是含 5×5 卷积层的第三条线路和含 3×3 最大池化层的第四条线路。其中第二、第三条线路都会先按比例减小通道数。这些比例在各个Inception块中都略有不同。

```
1 b4 = nn.Sequential(Inception(480, 192, (96, 208), (16, 48), 64),
2                     Inception(512, 160, (112, 224), (24, 64), 64),
3                     Inception(512, 128, (128, 256), (24, 64), 64),
4                     Inception(512, 112, (144, 288), (32, 64), 64),
5                     Inception(528, 256, (160, 320), (32, 128), 128),
6                     nn.MaxPool2d(kernel_size=3, stride=2, padding=1))
```

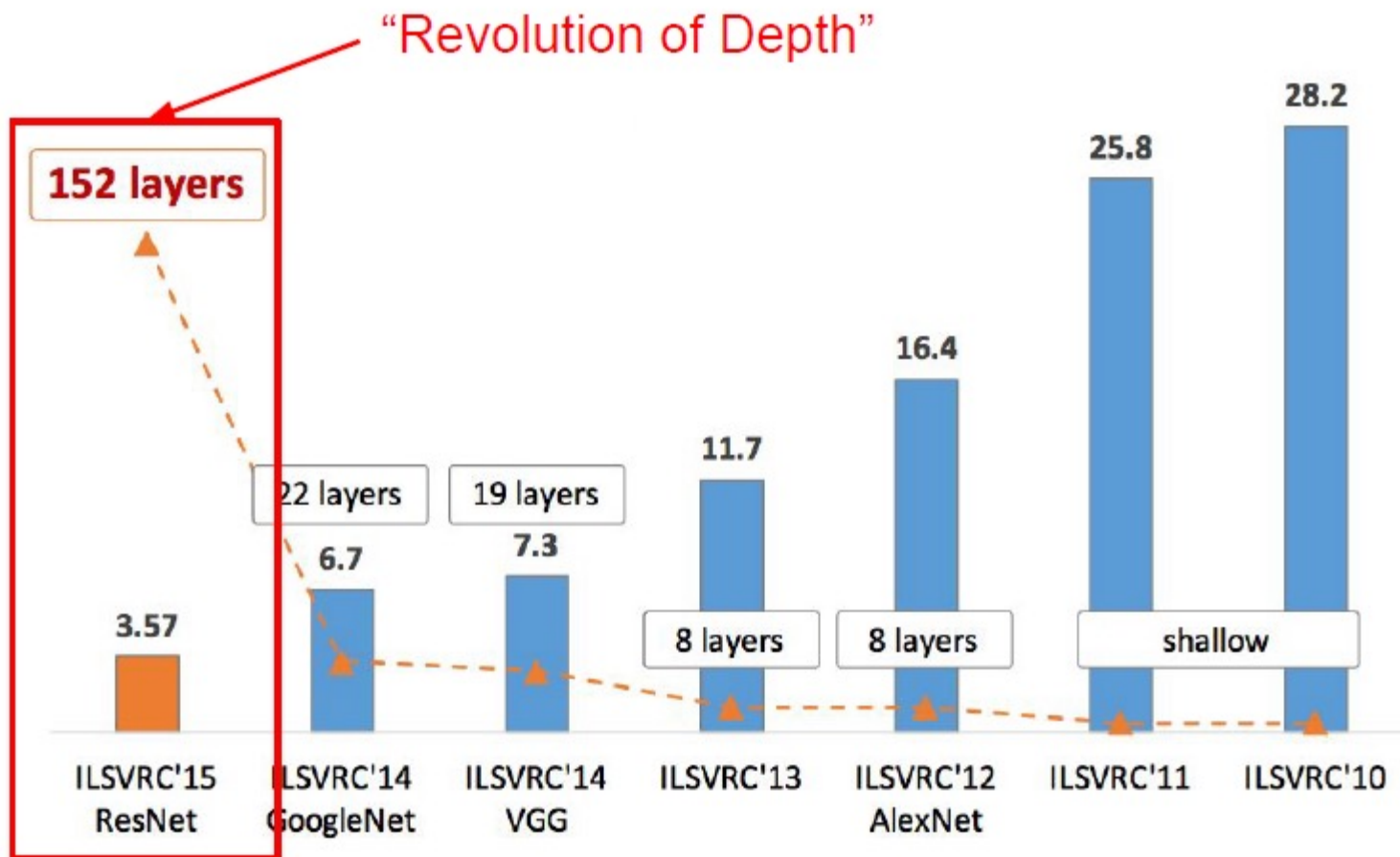
第五模块有输出通道数为 $256 + 320 + 128 + 128 = 832$ 和 $384 + 384 + 128 + 128 = 1024$ 的两个Inception块。其中每条线路的通道数的分配思路和第三、第四模块中的一致，只是在具体数值上有所不同。需要注意的是，第五模块的后面紧跟输出层，该模块同NIN一样使用全局平均池化层来将每个通道的高和宽变成1。最后我们将输出变成二维数组后接上一个输出个数为标签类别数的全连接层。

```
1 b5 = nn.Sequential(Inception(832, 256, (160, 320), (32, 128), 128),
2                     Inception(832, 384, (192, 384), (48, 128), 128),
3                     d2l.GlobalAvgPool2d())
4
5 net = nn.Sequential(b1, b2, b3, b4, b5,
6                     d2l.FlattenLayer(), nn.Linear(1024, 10))
```

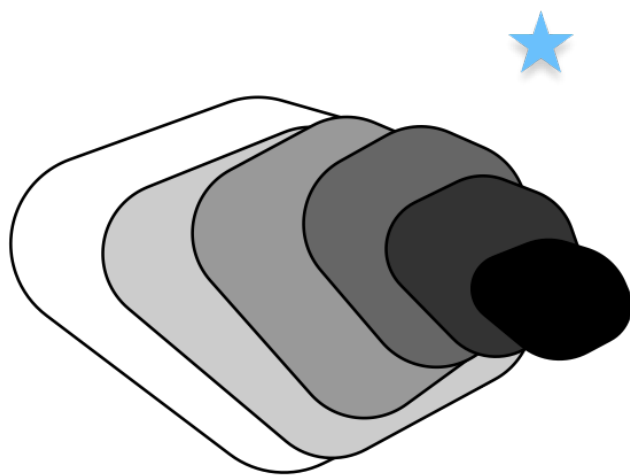
PyTorch 版本GoogLeNet

```
1 net = nn.Sequential(b1, b2, b3, b4, b5, d21.FlattenLayer(),  
  nn.Linear(1024, 10))  
2 X = torch.rand(1, 1, 96, 96)  
3 for blk in net.children():  
4     X = blk(X)  
5     print('output shape: ', X.shape)
```

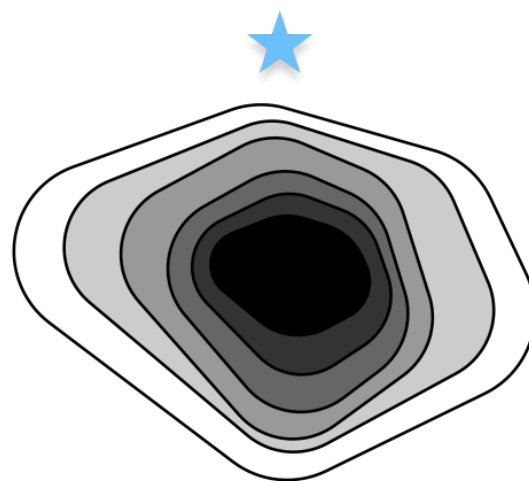
ImageNet大型视觉识别挑战赛（ILSVRC）获奖者



残差网络 (ResNet)



generic function classes

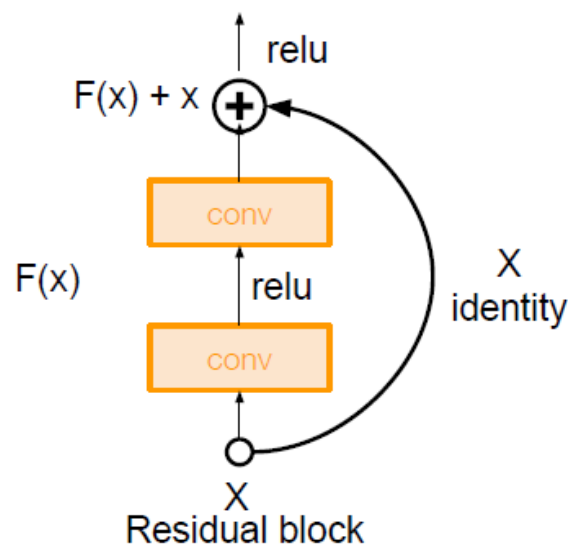


nested function classes

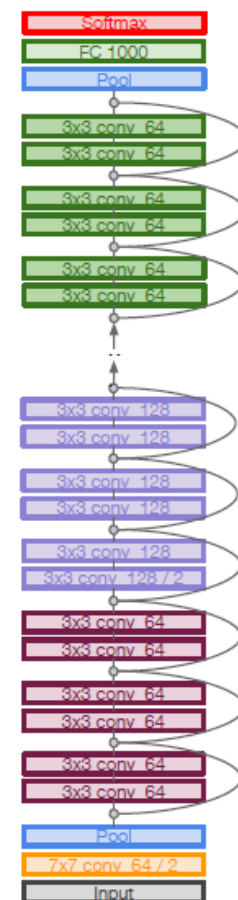
ResNet

使用残差连接的非常深的网络

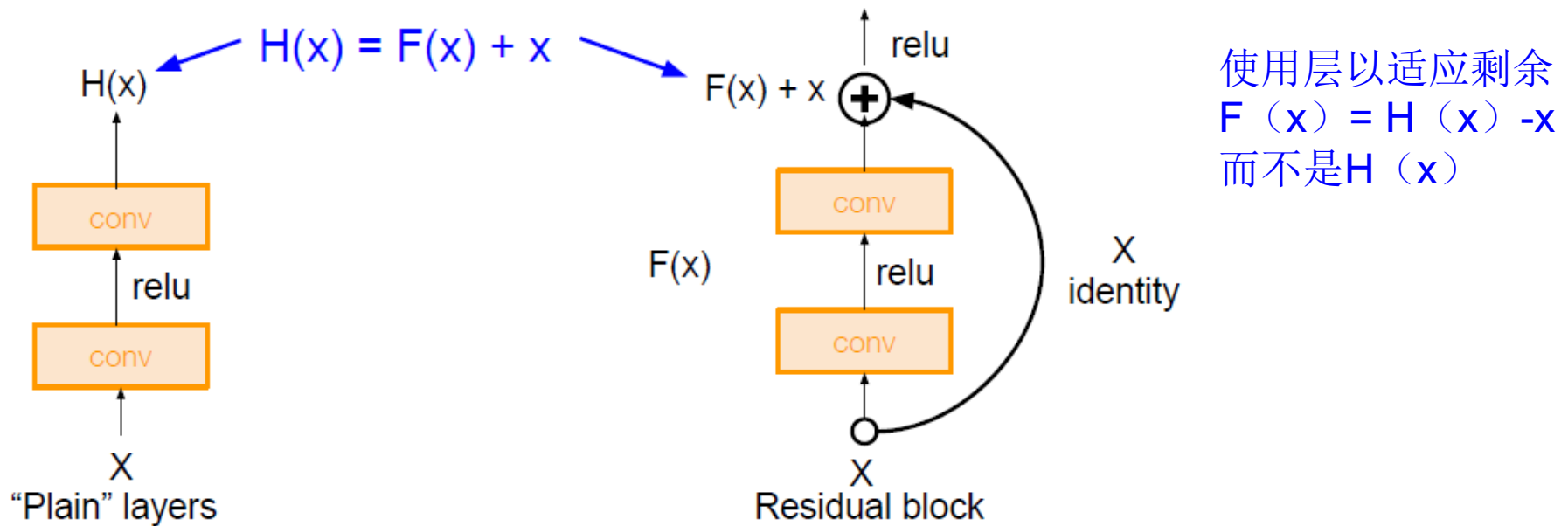
- ImageNet的152层模型
- ILSVRC'15分类冠军
(3.57%前5名错误)
- 赢得了ILSVRC'15和COCO'15中的所有分类和检测比赛！



网络的层数越多也就意味这能够提取到的不同级别的抽象特征更加丰富，并且越深的网络提取的特征越抽象，就越具有语义信息



ResNet

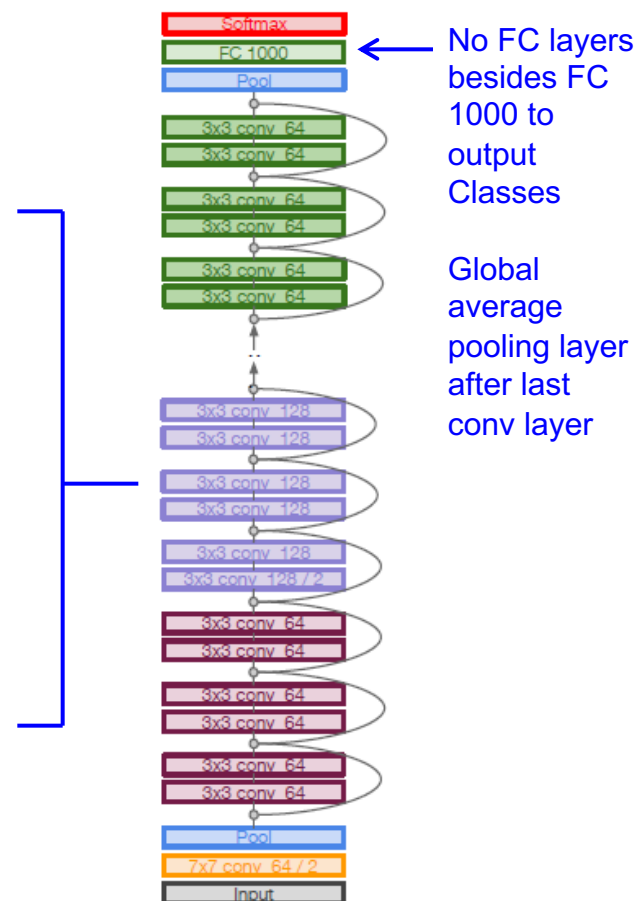
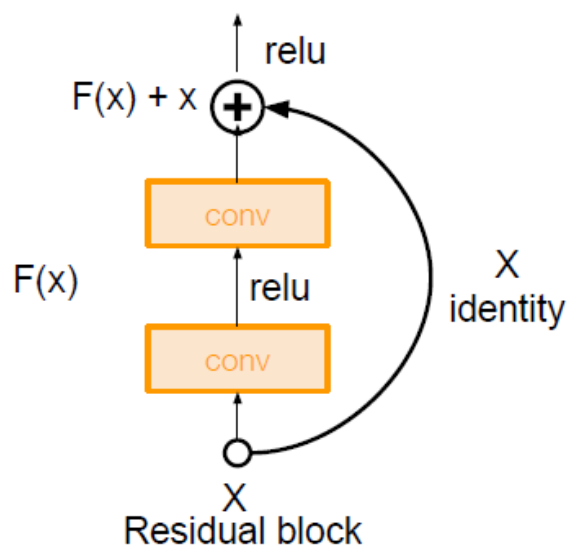


- ✓ 左半部分是常规的神经网络形式，每层有权值和激活函数，RELU函数，目的是为了避免梯度消失的问题，右半部分是残差网络的基本单元，和左边最大的不同在于多了一个身份即多了一个直接到达输出前的连线。
- ✓ 右图：刚开始输入的 X ，按照常规的神经网络进行权值叠加然后通过激活函数，

ResNet

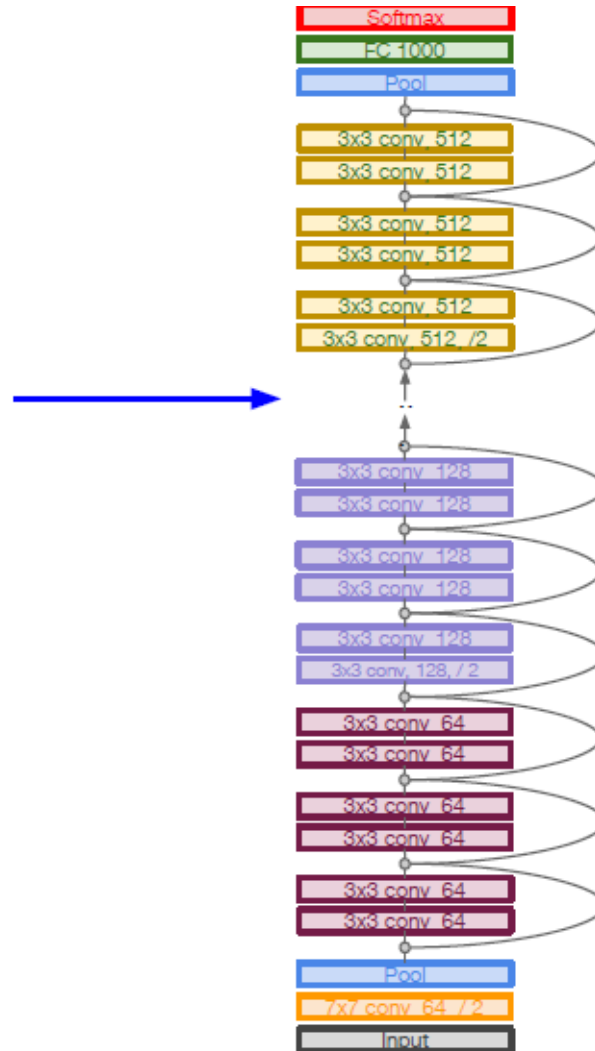
完整的ResNet架构:

- 堆积残差块
- 每个残差块有两个3x3卷积层
- 周期性地, 使用步长2 (/ 2) 在每个维度上的双倍滤波器和空间下采样
- 开始时的额外卷积层
- 没有FC层结束 (只有 FC 1000输出类)



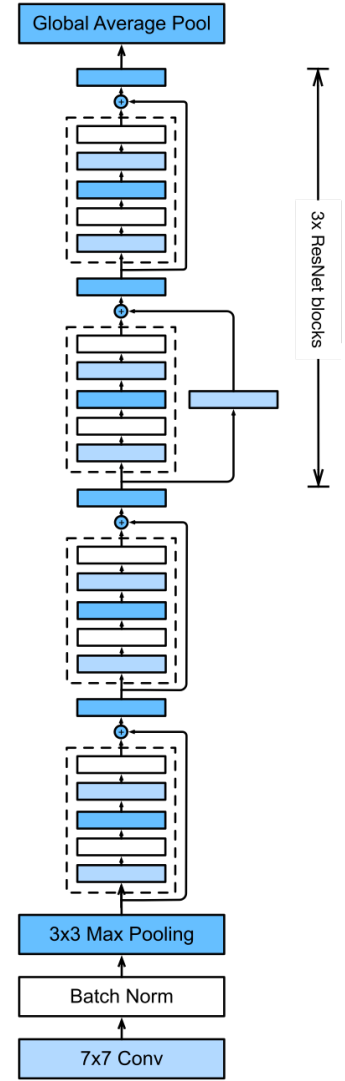
ResNet

Total depths of 34, 50, 101, or 152 layers for ImageNet



残差网络 (ResNet)

- 相同块结构，例如与VGG 或 GoogleNet使用块结构
- 残差块连接可增加表现力
- 池化 / 步幅 - 减少维度
- 批量归一化 - 容量控制
-大规模训练.....



PyTorch 版本ResNet

```
1 import time
2 import torch
3 from torch import nn, optim
4 import torch.nn.functional as F
5
6 import sys
7 sys.path.append("..")
8 import d2lzh_pytorch as d2l
9 device = torch.device('cuda' if torch.cuda.is_available() else
10 'cpu')
11
12 class Residual(nn.Module): # 本类已保存在d2lzh_pytorch包中方便以后使用
13     def __init__(self, in_channels, out_channels,
14         use_lxlconv=False, stride=1):
15         super(Residual, self).__init__()
16         self.conv1 = nn.Conv2d(in_channels, out_channels,
17             kernel_size=3, padding=1, stride=stride)
18         self.conv2 = nn.Conv2d(out_channels, out_channels,
19             kernel_size=3, padding=1)
20         if use_lxlconv:
21             self.conv3 = nn.Conv2d(in_channels, out_channels,
22                 kernel_size=1, stride=stride)
```

PyTorch 版本ResNet

```
18         else:
19             self.conv3 = None
20             self.bn1 = nn.BatchNorm2d(out_channels)
21             self.bn2 = nn.BatchNorm2d(out_channels)
22
23     def forward(self, X):
24         Y = F.relu(self.bn1(self.conv1(X)))
25         Y = self.bn2(self.conv2(Y))
26         if self.conv3:
27             X = self.conv3(X)
28         return F.relu(Y + X)
```

ResNet的前两层跟之前介绍的GoogLeNet中的一样：在输出通道数为64、步幅为2的 7×7 卷积层后接步幅为2的 3×3 的最大池化层。不同之处在于ResNet每个卷积层后增加的批量归一化层。

```
1 net = nn.Sequential(
2     nn.Conv2d(1, 64, kernel_size=7, stride=2, padding=3),
3     nn.BatchNorm2d(64),
4     nn.ReLU(),
5     nn.MaxPool2d(kernel_size=3, stride=2, padding=1))
```

GoogLeNet在后面接了4个由Inception块组成的模块。ResNet则使用4个由残差块组成的模块，每个模块使用若干个同样输出通道数的残差块。第一个模块的通道数同输入通道数一致。由于之前已经使用了步幅为2的最大池化层，所以无须减小高和宽。之后的每个模块在第一个残差块里将上一个模块的通道数翻倍，并将高和宽减半。

PyTorch 版本ResNet

```
1 def resnet_block(in_channels, out_channels, num_residuals,
2   first_block=False):
3     if first_block:
4         assert in_channels == out_channels # 第一个模块的通道数同输入通
5         道数一致
6         blk = []
7         for i in range(num_residuals):
8             if i == 0 and not first_block:
9                 blk.append(Residual(in_channels, out_channels,
10 use_1x1conv=True, stride=2))
11             else:
12                 blk.append(Residual(out_channels, out_channels))
13         return nn.Sequential(*blk)
```


PyTorch 版本ResNet

接着我们为ResNet加入所有残差块。这里每个模块使用两个残差块。

```
1 net.add_module("resnet_block1", resnet_block(64, 64, 2,
  first_block=True))
2 net.add_module("resnet_block2", resnet_block(64, 128, 2))
3 net.add_module("resnet_block3", resnet_block(128, 256, 2))
4 net.add_module("resnet_block4", resnet_block(256, 512, 2))
```

最后，与GoogLeNet一样，加入全局平均池化层后接上全连接层输出。

```
1 net.add_module("global_avg_pool", d2l.GlobalAvgPool2d()) #
  GlobalAvgPool2d的输出: (Batch, 512, 1, 1)
2 net.add_module("fc", nn.Sequential(d2l.FlattenLayer(),
  nn.Linear(512, 10)))
```

这里每个模块里有4个卷积层（不计算 1×1 卷积层），加上最开始的卷积层和最后的全连接层，共计18层。这个模型通常也被称为ResNet-18。通过配置不同的通道数和模块里的残差块数可以得到不同的ResNet模型，例如更深的含152层的ResNet-152。虽然ResNet的主体架构跟GoogLeNet的类似，但ResNet结构更简单，修改更方便。这些因素都导致了ResNet迅速被广泛使用。

稠密连接网络（DenseNet）

- ✓ ResNet, 通过shortcut连接可以有效使得网络更加深，且不会退化。
- ✓ DenseNet脱离了加深网络层数(ResNet)和加宽网络结构(Inception)来提升网络性能的定式思维,从特征的角度考虑,通过特征重用和旁路(Bypass)设置,既大幅度减少了网络的参数量,又在一定程度上缓解了gradient vanishing问题的产生。
- ✓ **特征复用**。论文是从**feature**入手，通过对**feature**的特征复用达到更好的效果和更少的参数

稠密连接网络（DenseNet）

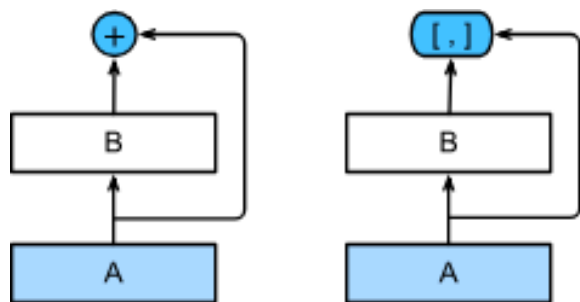
- ✓ CNN网络层数的不断增加,出现gradient vanishing和model degradation问题。
- ✓ BatchNormalization的广泛使用在一定程度上缓解了gradient vanishing的问题
- ✓ ResNet和Highway Networks通过构造恒等映射设置旁路,进一步减少了gradient vanishing和model degradation的产生

稠密连接网络（DenseNet）

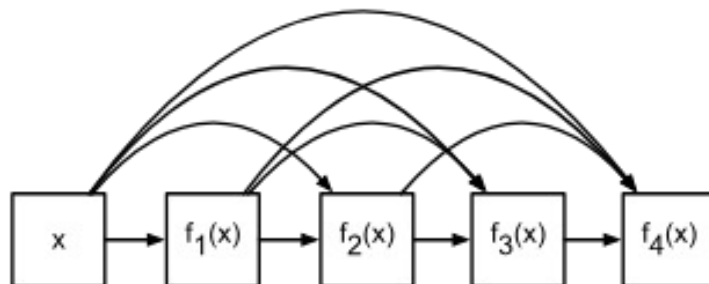
- ✓ 相比ResNet拥有更少的参数数量.
- ✓ 旁路加强了特征的重用.
- ✓ 网络更易于训练,并具有一定的正则效果.
- ✓ 缓解了gradient vanishing和model degradation的问题.

稠密连接网络（DenseNet）

- ✓ 在传统的卷积神经网络中，如果你有L层，那么就会有L个连接，但是在DenseNet中，会有 $L(L+1)/2$ 个连接。。简单讲，每就是一层的输入侧来自前面所有层的输出如下图产品：X0是输入，H1的输入是X0（输入），H2的输入是X0和X1（X1是H1的输出）



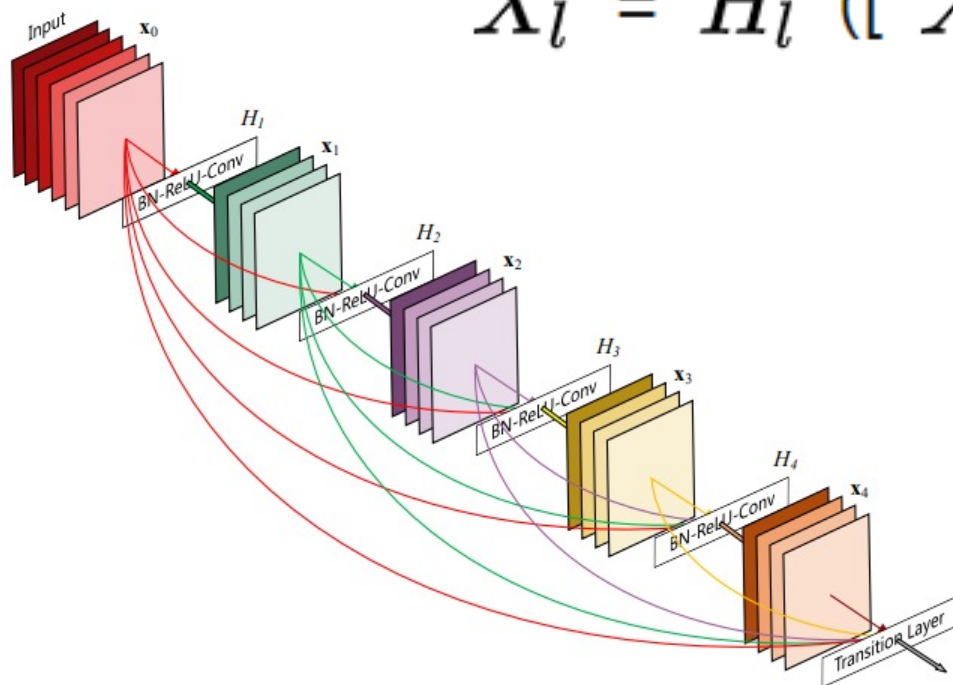
$$\mathbf{x}_\ell = H_\ell(\mathbf{x}_{\ell-1}) + \mathbf{x}_{\ell-1}.$$



稠密连接网络 (DenseNet)

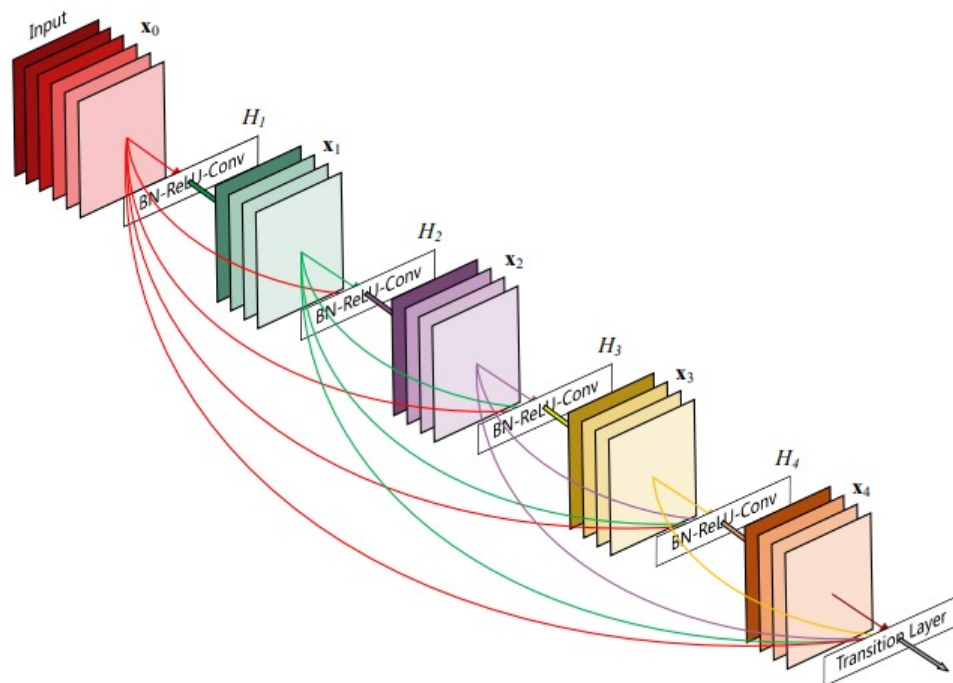
- ✓ 第*i*层的输入不仅与*i*-1层的输出相关,还有所有之前层的输出有关。

$$X_l = H_l ([X_0, X_1, \dots, X_{l-1}]),$$



稠密连接网络 (DenseNet)

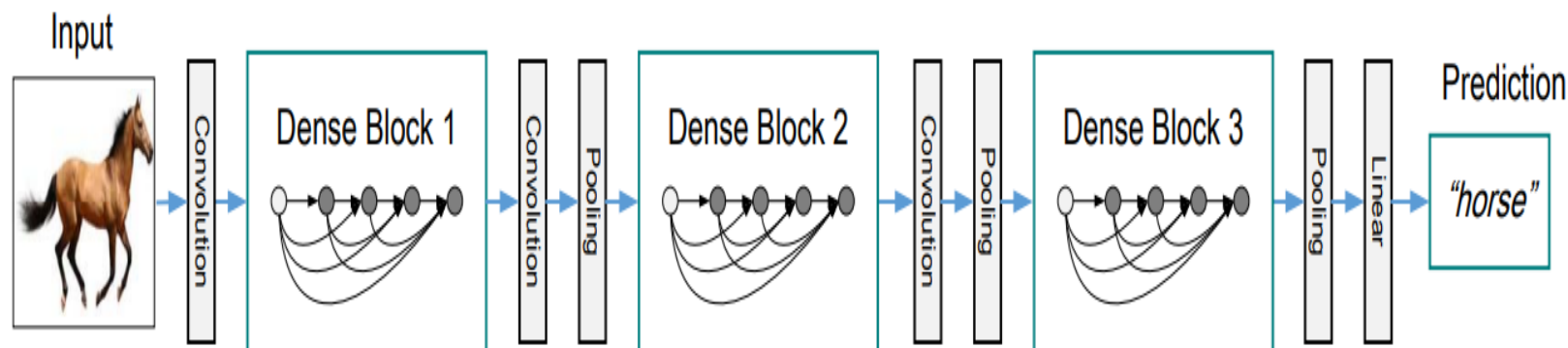
- ✓ []代表concatenation(拼接),既将 x_0 到 x_{l-1} 层的所有输出feature map按Channel组合在一起。



稠密连接网络（DenseNet）

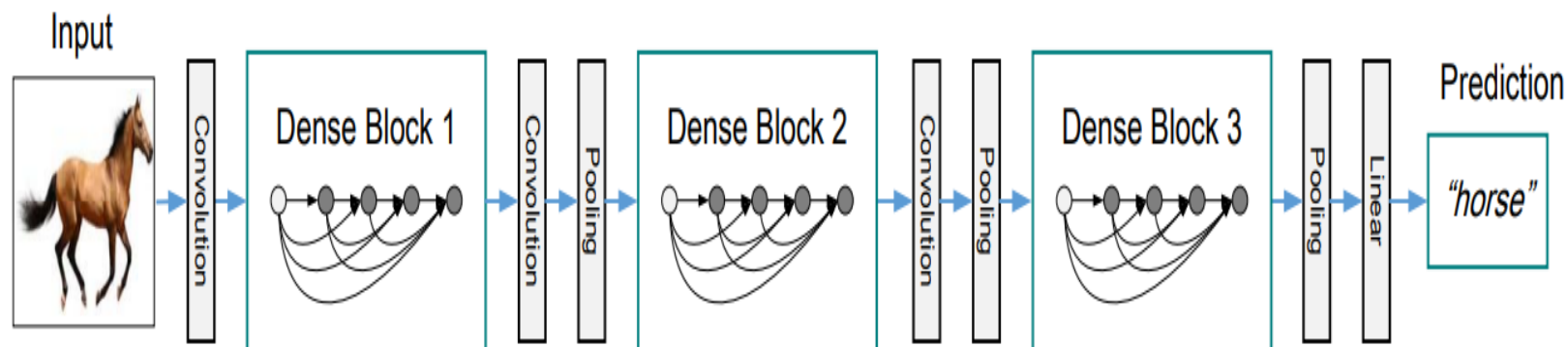
- ✓ 在DenseNet中需要对不同层的feature map进行cat操作,所以需要不同层的feature map保持相同的feature size,这就限制了网络中Down sampling的实现。为了使用Down sampling,作者将DenseNet分为多个Denseblock。

✓



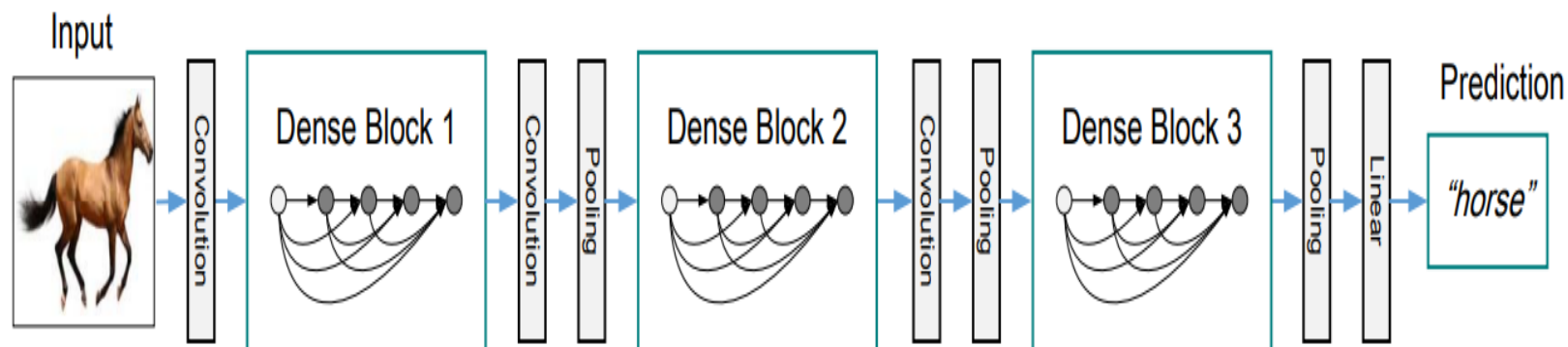
稠密连接网络（DenseNet）

- ✓ DenseNet的主要模块是稠密块（dense block）和过渡层（transition layer）。前者定义输入和输出如何连接。后者用来控制通道数，使之不过大。



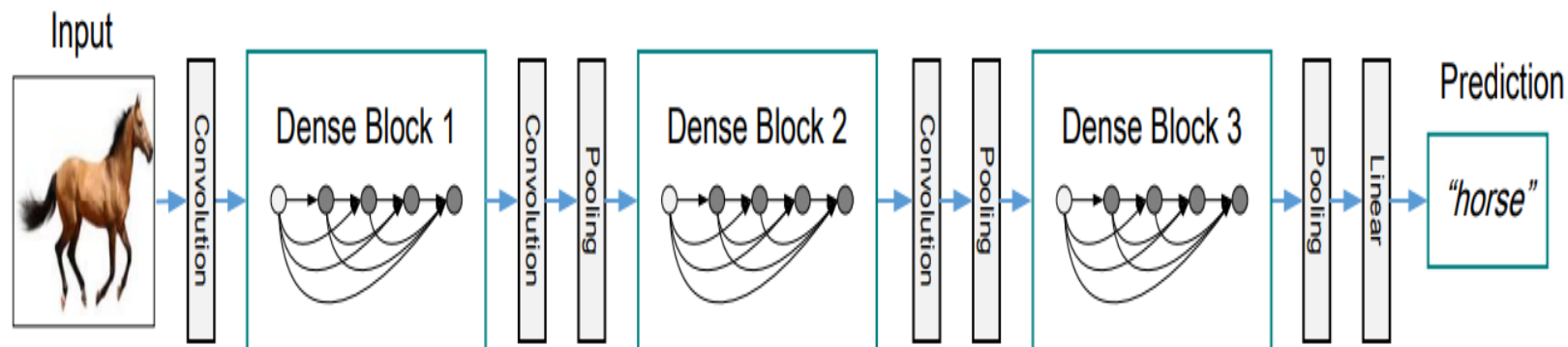
稠密连接网络（DenseNet）

- ✓ DenseNet的主要模块是稠密块（dense block）和过渡层（transition layer）。前者定义输入和输出如何连接。后者用来控制通道数，使之不过大。



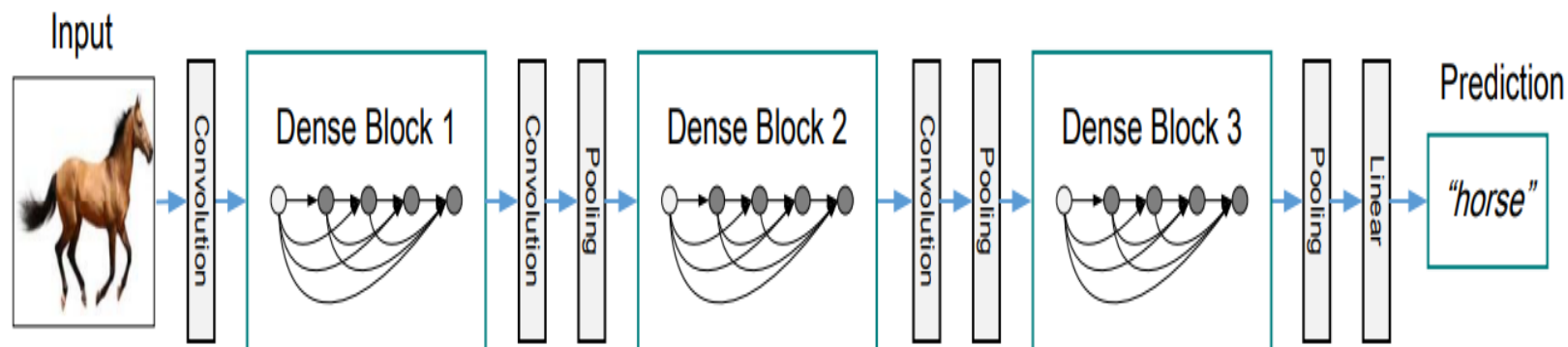
稠密连接网络 (DenseNet)

✓ Dense block



稠密连接网络 (DenseNet)

- ✓ 在同一个Denseblock中要求feature size保持相同大小,在不同Denseblock之间设置transition layers实现Down sampling, 实验中, transition layer由BN + Conv(1×1) + 2×2 average-pooling组成.



PyTorch 版本DenseNet(稠密层)

```
1 import time
2 import torch
3 from torch import nn, optim
4 import torch.nn.functional as F
5
6 import sys
7 sys.path.append("..")
8 import d2lzh_pytorch as d2l
9 device = torch.device('cuda' if torch.cuda.is_available() else
10                        'cpu')
11
12 def conv_block(in_channels, out_channels):
13     blk = nn.Sequential(nn.BatchNorm2d(in_channels),
14                         nn.ReLU(),
15                         nn.Conv2d(in_channels, out_channels,
16                                 kernel_size=3, padding=1))
17     return blk
```

PyTorch 版本DenseNet(稠密层)

稠密块由多个 `conv_block` 组成，每块使用相同的输出通道数。但在前向计算时，我们将每块的输入和输出在通道维上连结。

```
1 class DenseBlock(nn.Module):
2     def __init__(self, num_convs, in_channels, out_channels):
3         super(DenseBlock, self).__init__()
4         net = []
5         for i in range(num_convs):
6             in_c = in_channels + i * out_channels
7             net.append(conv_block(in_c, out_channels))
8         self.net = nn.ModuleList(net)
9         self.out_channels = in_channels + num_convs * out_channels
10        # 计算输出通道数
11
12    def forward(self, X):
13        for blk in self.net:
14            Y = blk(X)
15            X = torch.cat((X, Y), dim=1) # 在通道维上将输入和输出连结
16        return X
```

PyTorch 版本DenseNet(稠密层)

在下面的例子中，我们定义一个有2个输出通道数为10的卷积块。使用通道数为3的输入时，我们会得到通道数为 $3 + 2 \times 10 = 23$ 的输出。卷积块的通道数控制了输出通道数相对于输入通道数的增长，因此也被称为增长率（growth rate）。

```
1 blk = DenseBlock(2, 3, 10)
2 X = torch.rand(4, 3, 8, 8)
3 Y = blk(X)
4 Y.shape # torch.Size([4, 23, 8, 8])
```

PyTorch 版本DenseNet(过渡层)

```
1 def transition_block(in_channels, out_channels):
2     blk = nn.Sequential(
3         nn.BatchNorm2d(in_channels),
4         nn.ReLU(),
5         nn.Conv2d(in_channels, out_channels, kernel_size=1),
6         nn.AvgPool2d(kernel_size=2, stride=2))
7     return blk
```

对上一个例子中稠密块的输出使用通道数为10的过渡层。此时输出的通道数减为10，高和宽均减半。

```
1 blk = transition_block(23, 10)
2 blk(Y).shape # torch.Size([4, 10, 4, 4])
```

BN， 1*1卷积层， 平均池化

PyTorch 版本DenseNet

我们来构造DenseNet模型。DenseNet首先使用同ResNet一样的单卷积层和最大池化层。

```
1 net = nn.Sequential(  
2     nn.Conv2d(1, 64, kernel_size=7, stride=2, padding=3),  
3     nn.BatchNorm2d(64),  
4     nn.ReLU(),  
5     nn.MaxPool2d(kernel_size=3, stride=2, padding=1))
```

类似于ResNet接下来使用的4个残差块，DenseNet使用的是4个稠密块。同ResNet一样，我们可以设置每个稠密块使用多少个卷积层。这里我们设成4，从而与上一节的ResNet-18保持一致。稠密块里的卷积层通道数（即增长率）设为32，所以每个稠密块将增加128个通道。

ResNet里通过步幅为2的残差块在每个模块之间减小高和宽。这里我们则使用过渡层来减半高和宽，并减半通道数。

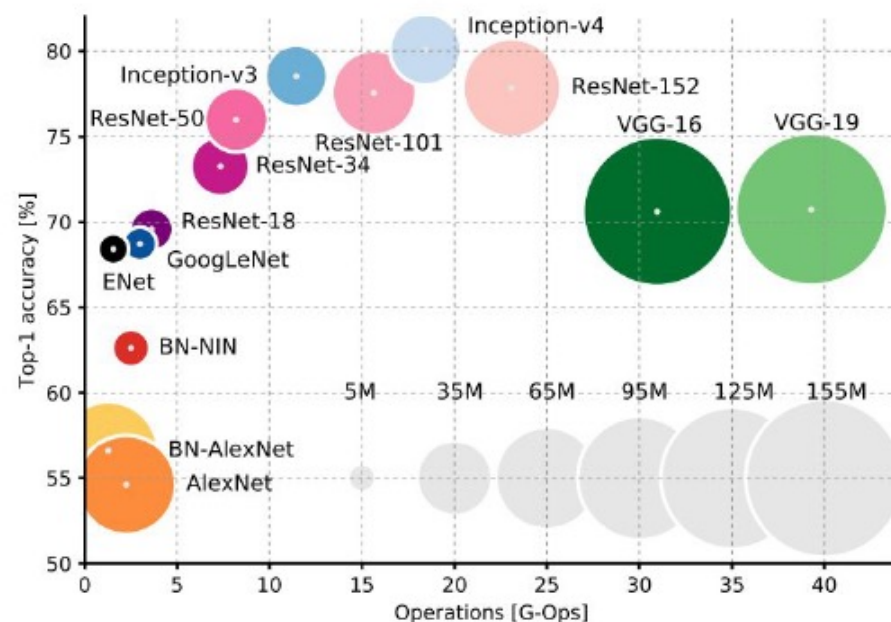
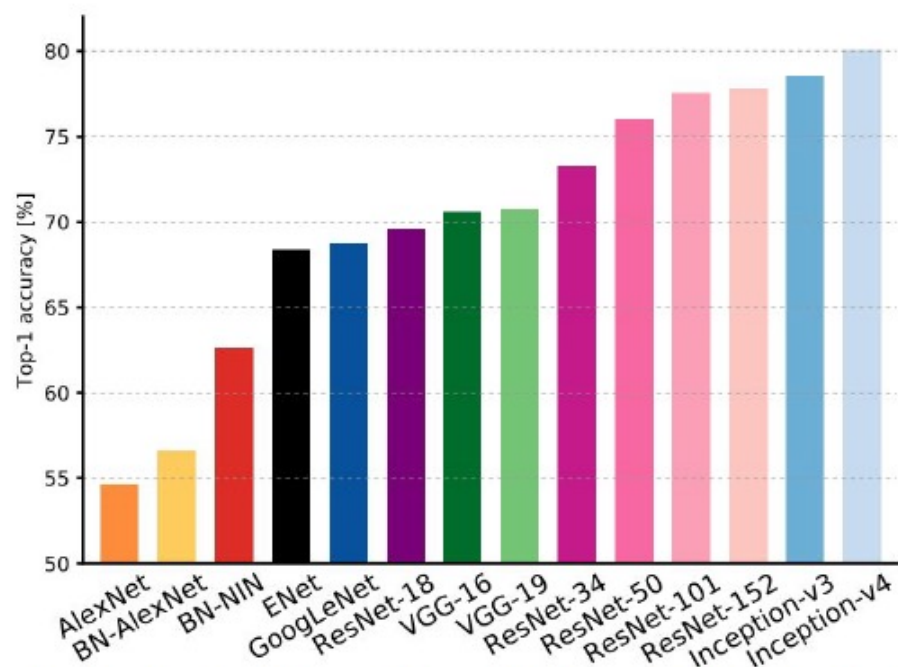
PyTorch 版本DenseNet

```
1 num_channels, growth_rate = 64, 32 # num_channels为当前的通道数
2 num_convs_in_dense_blocks = [4, 4, 4, 4]
3
4 for i, num_convs in enumerate(num_convs_in_dense_blocks):
5     DB = DenseBlock(num_convs, num_channels, growth_rate)
6     net.add_module("DenseBlock_%d" % i, DB)
7     # 上一个稠密块的输出通道数
8     num_channels = DB.out_channels
9     # 在稠密块之间加入通道数减半的过渡层
10    if i != len(num_convs_in_dense_blocks) - 1:
11        net.add_module("transition_block_%d" % i,
12            transition_block(num_channels, num_channels // 2))
13        num_channels = num_channels // 2
```

同ResNet一样，最后接上全局池化层和全连接层来输出。

```
1 net.add_module("BN", nn.BatchNorm2d(num_channels))
2 net.add_module("relu", nn.ReLU())
3 net.add_module("global_avg_pool", d2l.GlobalAvgPool2d()) #
   GlobalAvgPool2d的输出: (Batch, num_channels, 1, 1)
4 net.add_module("fc", nn.Sequential(d2l.FlattenLayer(),
   nn.Linear(num_channels, 10)))
```

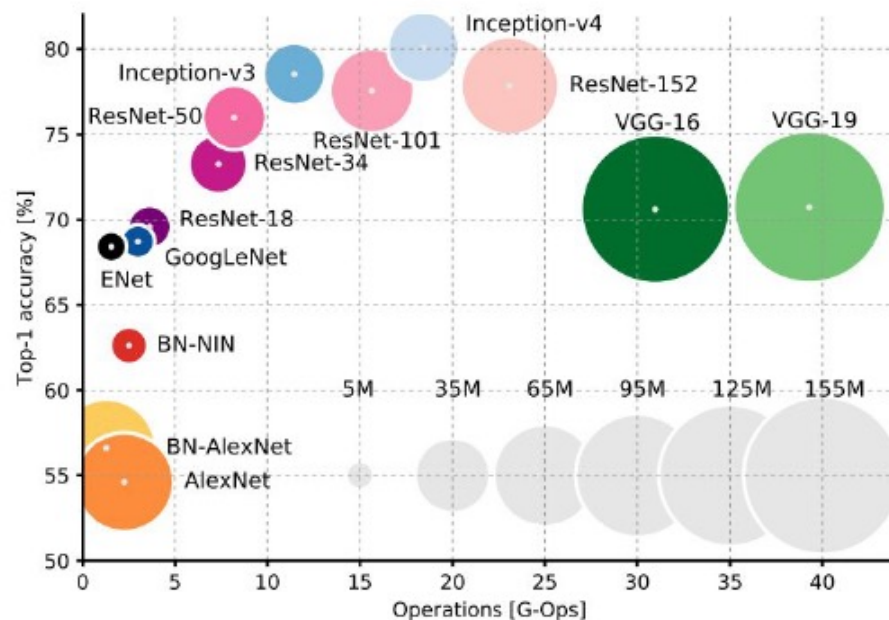
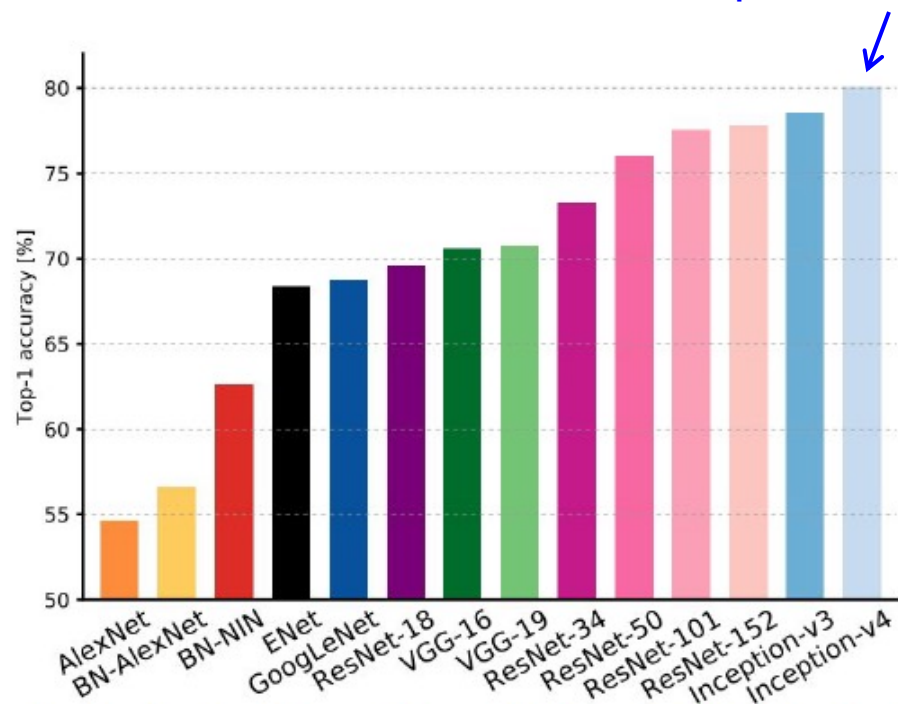
复杂度比较



An Analysis of Deep Neural Network Models for Practical Applications, 2017.

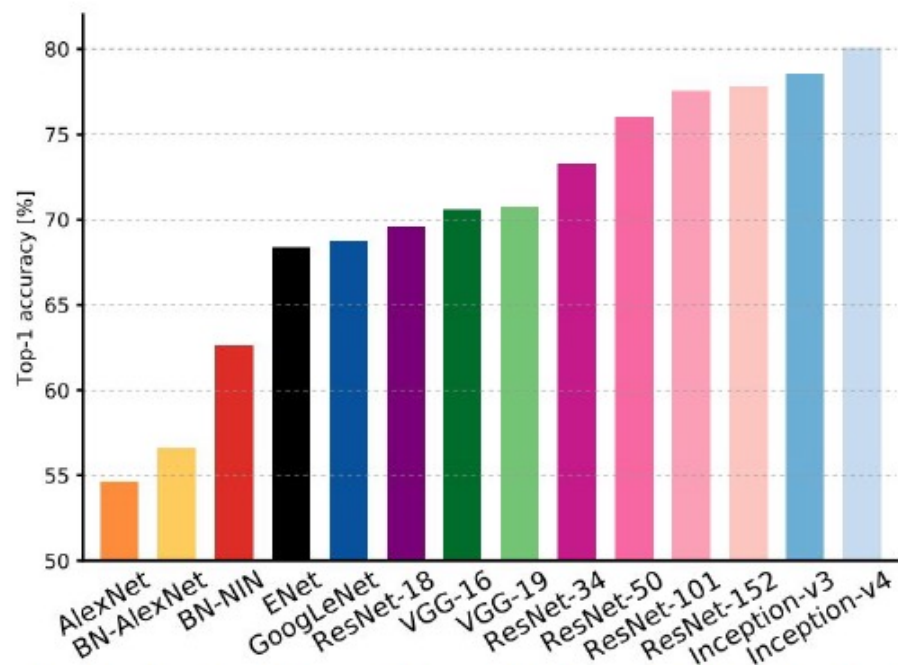
复杂度比较

Inception-v4: Resnet + Inception!

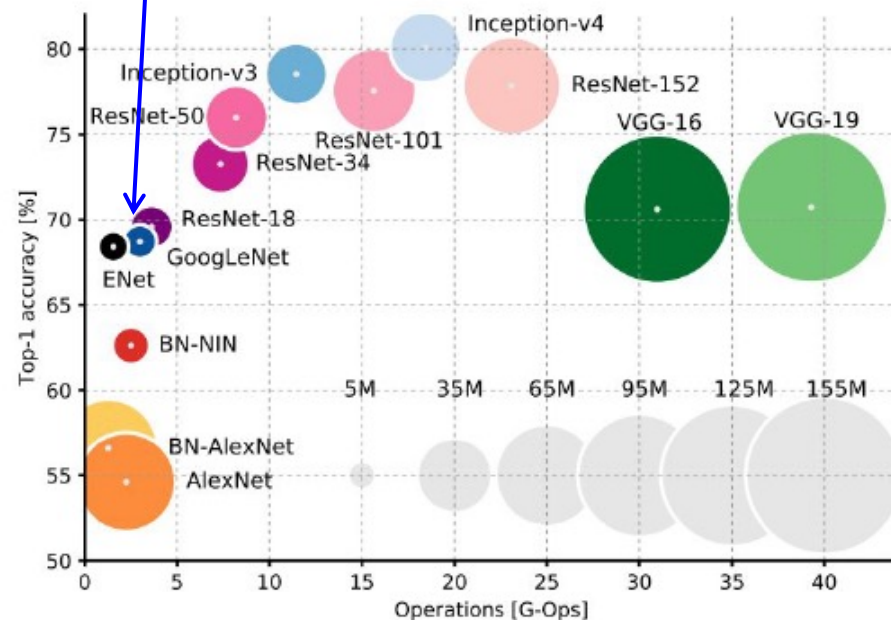


An Analysis of Deep Neural Network Models for Practical Applications, 2017.

复杂度比较

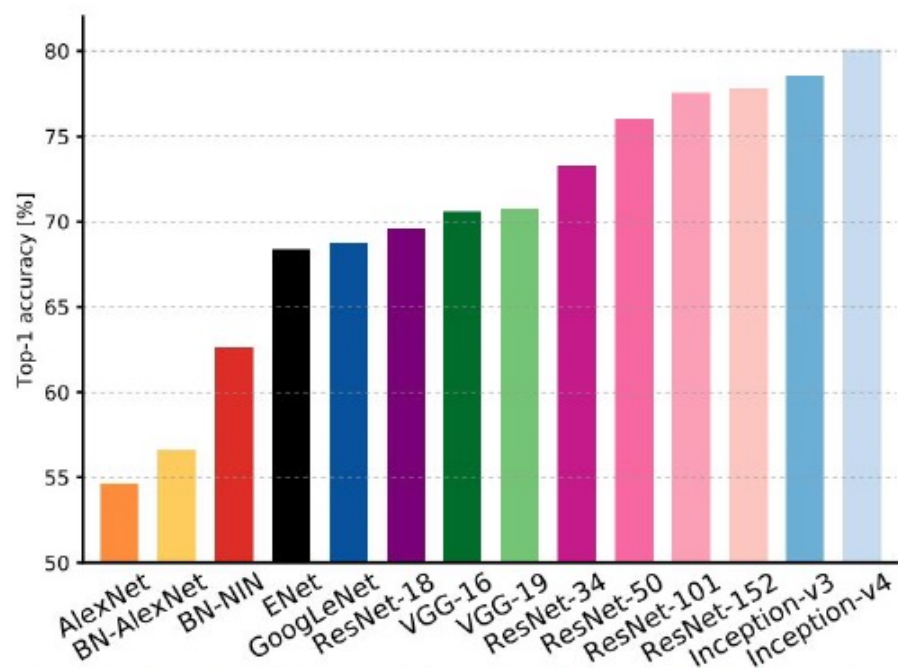


GoogLeNet:
Most efficient

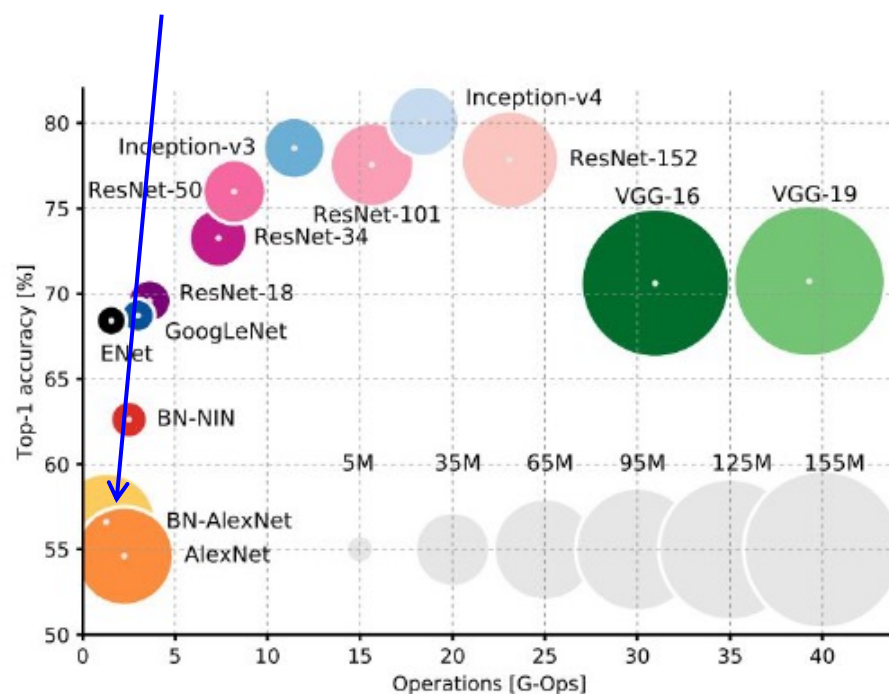


An Analysis of Deep Neural Network Models for Practical Applications, 2017.

复杂度比较

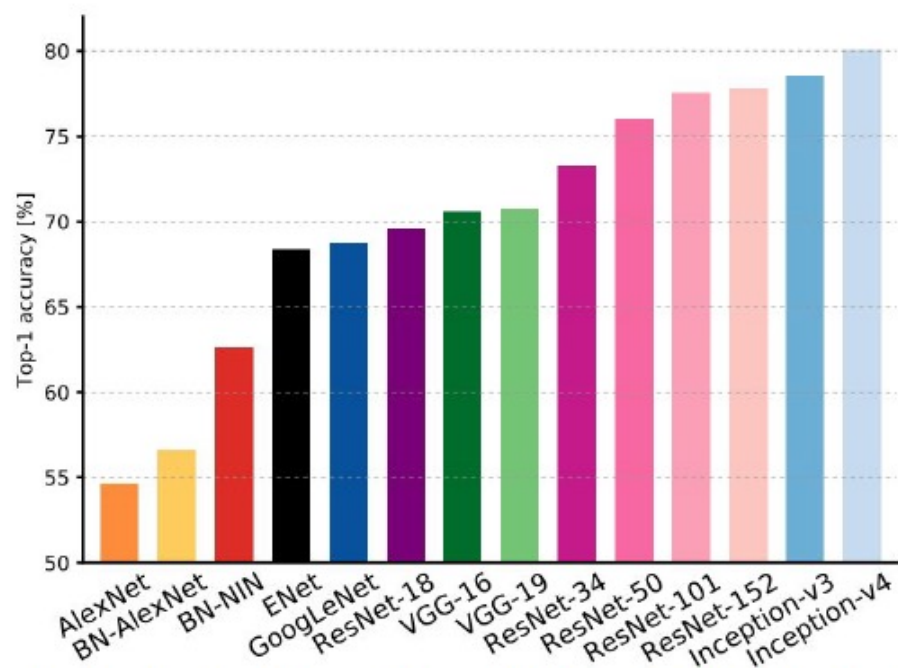


AlexNet:
Smaller compute, still memory
heavy, lower accuracy

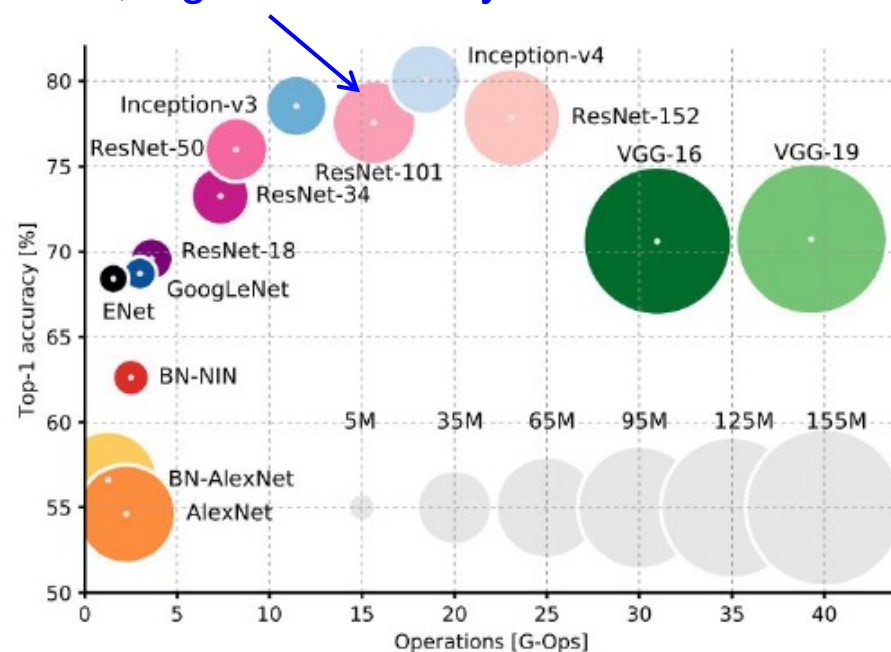


An Analysis of Deep Neural Network Models for Practical Applications, 2017.

复杂度比较

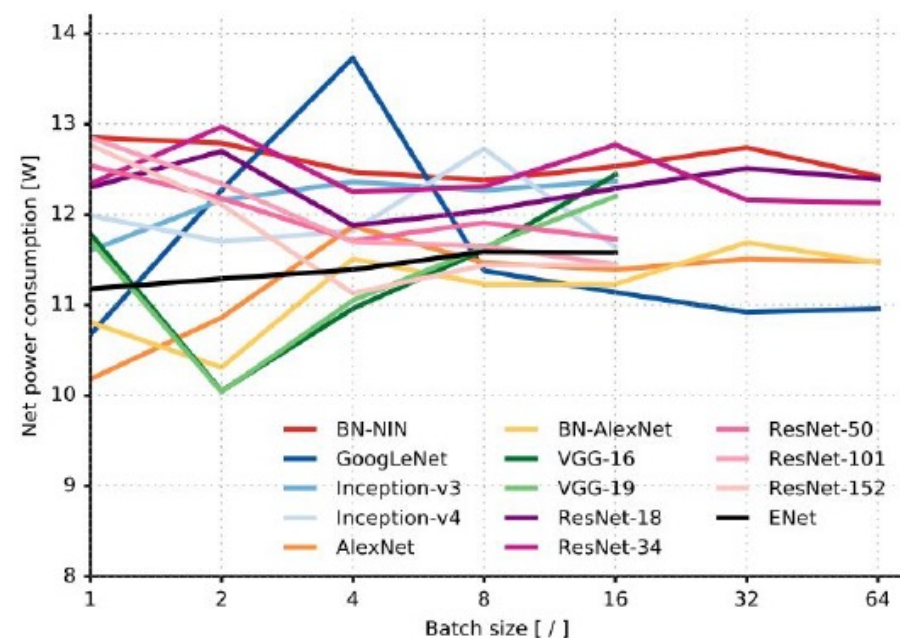
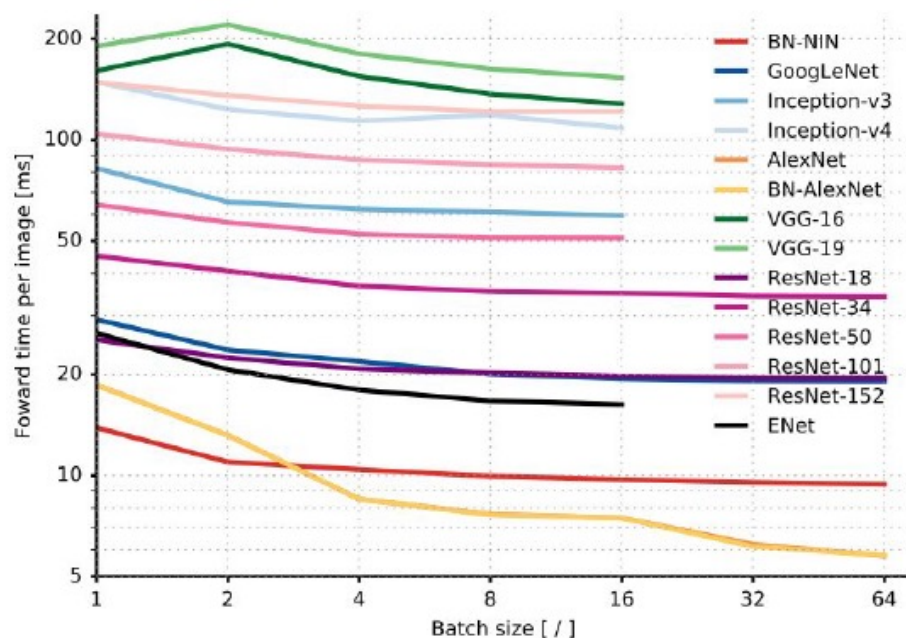


ResNet:
Moderate efficiency depending on
model, highest accuracy

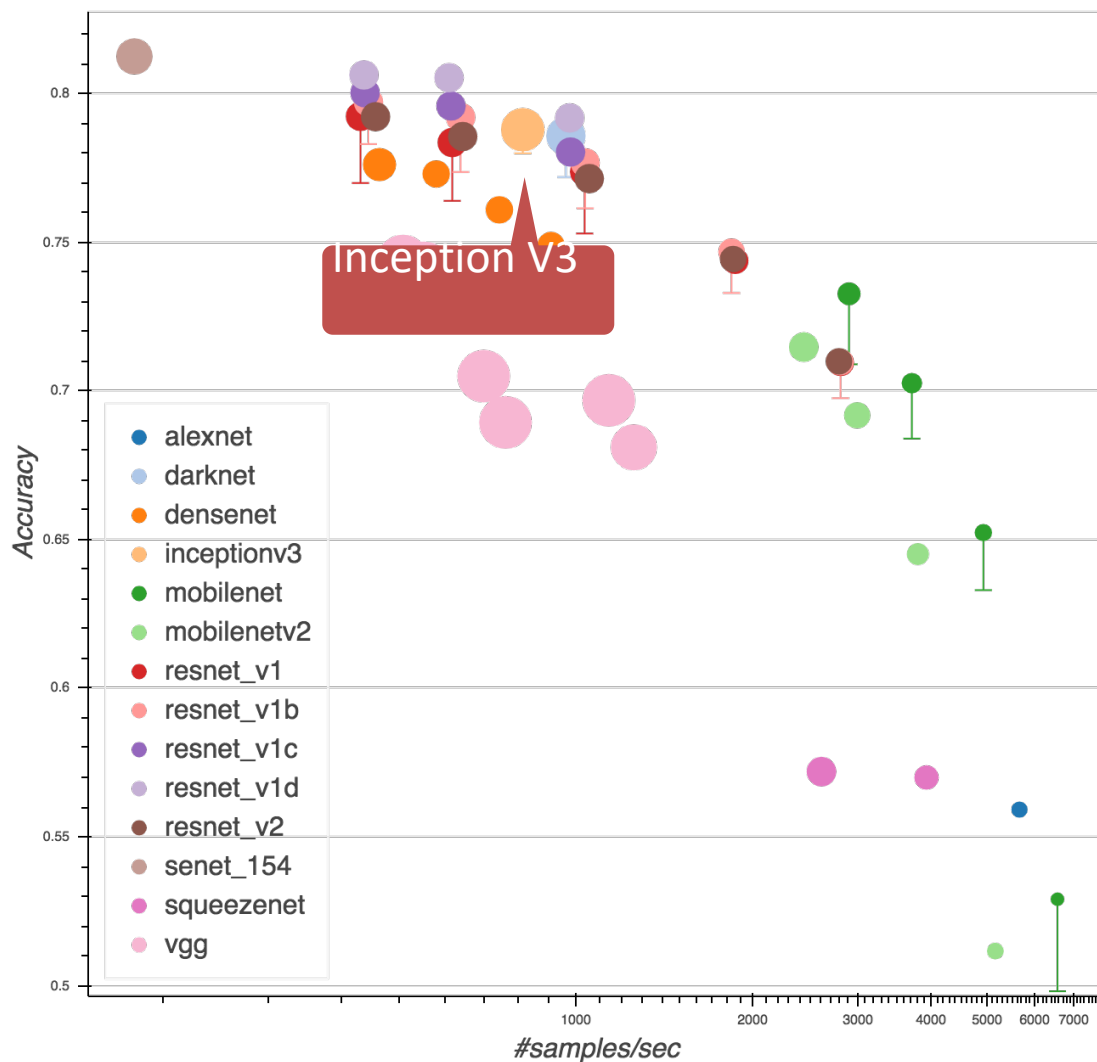


An Analysis of Deep Neural Network Models for Practical Applications, 2017.

正向传输时间和功耗



An Analysis of Deep Neural Network Models for Practical Applications, 2017.



GluonCV 模型“动物园”
https://gluon-cv.mxnet.io/model_zoo/classification.html